Web-Based Car Rental Management System with Real-Time IoT-Based GPS Tracking for 1816 Car Rental Co.

A Capstone Project presented to the
College of Computer Studies of
Pamantasan ng Lungsod ng Pasig

In partial fulfillment of the requirements for the degree of
Bachelor of Science in Information Technology

by:

Junio, Carl AJ G.
Martinez, John Edrian V.
Navarro, Rex S.

Federico G. Nueva, MT
Adviser

October 2026

ADVISER'S RECOMMENDATION SHEET

This Capstone Project entitled

Web-Based Car Rental Management System with Real-Time IoT-Based GPS Tracking for 1816 Car Rental Co.

by:

Junio, Carl AJ G.
Martinez, John Edrian V.
Navarro, Rex S.

And submitted in partial fulfillment of the requirements of the
Bachelor of Science in Information Technology degree
has been examined and is recommended for acceptance and approval

_____

Federico G. Nueva, MT
Adviser

October 2026

CAPSTONE PROJECT COORDINATOR
AND DEAN'S ACCEPTANCE SHEET

This Capstone Project entitled

Web-Based Car Rental Management System with Real-Time IoT-Based GPS Tracking for 1816 Car Rental Co.

by:

Junio, Carl AJ G.
Martinez, John Edrian V.
Navarro, Rex S.

After having been recommended and approved is hereby accepted by
the College of Computer Studies of
The Pamantasan ng Lungsod ng Pasig

_____

Noreen A. Perez, DIT
Capstone Project Coordinator

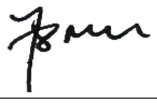
_____

Riegie D. Tan, DIT
CCS Dean

October 2026

CERTIFICATION

This capstone project entitled **Web-Based Car Rental Management System with Real-Time IoT-Based GPS Tracking for 1816 Car Rental Co.** prepared and submitted by Carl AJ G. Junio, John Edrian V. Martinez, and Rex S. Navarro in partial fulfillment of the requirements for the degree of Bachelor of Science in Information Technology (BSIT) has been examined and recommended for ORAL EXAMINATION.



Federico G. Nueva, MT
Adviser

APPROVAL SHEET

Approved by the Panel for Oral Defense on _____ with a rating of _____.

Greta M. Rosario, DIT
Panel Member

Noreen A. Perez, DIT
Chairperson

Accepted in partial fulfillment of the degree of Bachelor of Science in Information Technology.

Riegie D. Tan, DIT
CCS Dean

ACKNOWLEDGMENT

To begin with, we want to thank our Lord and Savior Jesus Christ for everything. He has been faithful then, now, and forever will be.

We would also like to thank our families, who did their best to support us in this process and help us concentrate on our capstone project. We were fed wonderfully with the meals that were prepared for us before and after the joint work.

Thanks to our adviser Mr. Nueva for his wonderful help and for his willingness to help us and share his valuable experience throughout our project. His advice was of great importance for the building of the entire system and for enabling us to find the simplest solutions.

We would also like to thank our panelists for their review of our project. Their questions helped us see the things we need to pay attention to and what we need to fix.

Finally, we would also like to express gratitude to our client, the study that we conducted, namely, 1816 Car Rental Company, and to its owner who was so kind to have entrusted us with this study.

C.A.J.G.J.

J.E.V.M.

R.S.N.

TABLE OF CONTENTS

List of Appendices vii

List of Figures viii

List of Tables ix

Abstract x

CHAPTER 1: Introduction 1

The Problem and its Background 1

Review of Related Literature 3

Significance of the Study 22

Statement of the Problem 23

Scope and Limitation of the Study 25

CHAPTER 2: Methodology 31

Research Design 31

Architectural Framework 33

Level 0 Data Flow Diagram 35

Algorithm Discussion 41

SE Paradigm 46

Population and Sampling 50

Data Collection Methods 51

Ethical Considerations 53

CHAPTER 3: Results and Discussions 59

Project Capabilities 59

Project Testing Results 79

System Evaluation Result 86

CHAPTER 4: Conclusion and Recommendations 95

Conclusion 95

Recommendations 97

References 101

Appendices 106

Curriculum Vitae 171

LIST OF APPENDICES

Appendix A: Operational Definition of Terms 106

Appendix B: Evaluation Sheets 112

Appendix C: Project Timeline 116

Appendix D: Work Assignments 119

Appendix E: Certificate of Appreciation 125

Appendix F: Source Code 129

Appendix G: Plagiarism Check Results 150

Appendix H: AI Generated Content Check Results 153

Appendix I: Thesis and Capstone Project Agreement (AI Disclosure) 156

Appendix J: Extended Abstract 164

LIST OF FIGURES

Figure 1: Architectural Framework 33

Figure 2: Level 0 Data Flow Diagram 35

Figure 3: Agile Development Approach 46

Figure 4: Owner Command Center 60

Figure 5: Fleet Inventory 61

Figure 6: Pricing and Availability 63

Figure 7: Booking Monitor 64

Figure 8: Live GPS Tracking 65

Figure 9: Renter Registration and Live Selfie Capture 67

Figure 10: Renter Requirements Review 69

Figure 11: Vehicle Details and Booking Form 70

Figure 12: Reservation Payment Confirmation 71

Figure 13: Vehicle Condition Checks and Documentation 72

Figure 14: Owner Review of Condition Findings 74

Figure 15: Operational and Financial Reports 75

Figure 16: Analytics for Decision-Making 76

LIST OF TABLES

Table 1: Selected Data Flow Examples 39

Table 2: Dent Confusion Matrix 83

Table 3: Scratch Confusion Matrix 83

Table 4: Crack Confusion Matrix 84

Table 5: Shattered Glass Confusion Matrix 84

Table 6: Broken Lamp Confusion Matrix 85

Table 7: Flat Tire Confusion Matrix 85

Table 8: System Functionality Result 87

Table 9: System Reliability Result 88

Table 10: System Usability Result 88

Table 11: System Efficiency Result 89

Table 12: System Portability Result 89

Table 13: System Maintainability Result 90

Table 14: Summary of System Evaluation Results 90

Table 15: General Weighted Means by Criterion 91

ABSTRACT

Title : Web-Based Car Rental Management System with Real-Time IoT-Based GPS Tracking for 1816 Car Rental Co.

Researchers : Junio, Carl AJ G.
Martinez, John Edrian V.
Navarro, Rex S.

Degree : Bachelor of Science in Information Technology

School Year : 2026–2027

Project Description :

In 1816 Car Rental Co., customers booked rentals via Facebook Messenger, while records of these transactions were maintained through Google Sheets. This created a difficulty in verifying the availability of rental vehicles and in coordinating records. This research study described and developed a web-based car rental management system with GPS tracking using an IoT platform for five rented cars. The platform centralized rent document reviews, reservation making, payment transactions, keeping of records, and vehicle condition checks and reports. The techniques of purpose sampling were used to measure the responses of 15 renters, one owner, and five technical evaluators. Respondents evaluated 15 indicators according to six criteria of software quality with the help of a four-point questionnaire. Equal weight of evaluators produced an aggregate rating Mean of 3.72 which showed Strongly Agree results for this software.

CHAPTER 1

INTRODUCTION

The Problem and its Background

Trade continues to be one of the pillars of modern economies and results in millions of commodities and products delivered to consumers every day. The current situation on the market is characterized by active operations and severe development. The number of sectors increases considerably every year. Logistics is one of the most important industries in the above-mentioned area. In this particular sector, logistics service providers face stronger requirements for the digitalization of their operations in order to remain competitive under such conditions (Cichosz et al., 2020). With the described growth, car rentals represent one of the principal processes.

The car rental industry experiences a massive demand since people always need transport means. This sector is expanding as the need for transportation is infinite. However, with the availability of car rental services, car owners cannot keep track of their vehicles. Akanda et al. (2022), in their research, demonstrated how GPS tracking can yield data about where the vehicle is and how fast it is moving. That did not contribute to the testers having information about how the technology performs when the vehicle is utilized by car renters since the investigators conducted tests on a prototype rather than rented vehicle.

Purchasing a private automobile is financially taxing for the consumer. Owning a car means maintaining it, paying for a yearly insurance policy, and also

factoring in depreciation of the vehicle's worth. In contrast to ownership, individuals would rather pay for having temporary access to a car. This tendency is indicative of the change in consumption patterns of consumers aimed at progressive switching from ownership to temporary access Henderson et al (2022). Consequently, this behavioral shift helps customer fleet companies to attract more customers. In order to meet consumer demands, such businesses should set up a strong digital system for daily customer bookings and processing.

At the time of conducting this research, the 1816 Car Rental Co. was a car rental business established in February 2025 in Marikina City by 2 people who were in charge of the daily operations, such as vehicle pick-up and delivery, cleaning, and service. The business development resulted in having 5 vehicles. Fb Messenger and Facebook groups related to vehicle rental were used for reservations, where the information regarding revenue and reservation was maintained on Google Sheets. This system created problems concerning confirmation of vehicle availability, double reservations, lack of rental agreement documents, and delays in answering customers.

In order to solve the problem mentioned above, the current research performed the development of a web car renting management system with IoT tracking capabilities. The developed system gave an opportunity to synchronize the reservations with the calendar and keep track of clients’ requirements. The dashboard presented the results of GPS measurements from the prototype.

Review of Related Literature

Car Rental Management Systems and Real-Time Fleet Tracking

In the past, businesses operating car rentals mainly relied on paperwork. But now, digital platforms have replaced that approach, making the process of bookings, fleet protection, and service easier. This has happened due to the fact that paper and manual processes are not sufficient since rental operations have become so busy. Golo and Encarnacion (2024) researched this issue in Siargao island in the Philippines. They found out that rental companies faced problems with booking errors, chaos in records, and scheduling that broke down easily. As a solution to that, they suggested a rental management system that will help to automate the booking and to streamline fleet scheduling and display the availability of cars for customers. The target groups of this study include rent operators, tourists, and office personnel, the business operators being the key beneficiaries of the system replacing paper work system. The results of their research indicated that the automatic reservation process can help reduce booking mistakes and provide customers with easier access to rental services.

The necessity for continued, structured booking is a matter not only for tourism destinations in particular areas. The authors Kashmim et al. (2023) introduced their vehicle agency rental online system, which solves the problem of the limitations in time imposed by some conventional rental systems and allows the clients to book the vehicles 24/7, manage the fleet automatically and compare vehicles. Additionally, the system allows changing and canceling a booking, as well as ensuring the proper background for administrators.

Furthermore, as the results of the survey conducted among agencies and customers indicated, the new method would help to boost the efficiency of work at both different organizations and companies without the need to check the booking process by means of phone calls or contacting the agencies directly, however, the study has not provided any data related to real applications of this system in life. Golo and Encarnacion concentrated on solving the current problems regarding complicated outdated processes in one place, whereas the authors argue that a successful rental system must operate 24 hours a day.

The combined results of these two studies provide clear evidence that any online rental platform is required to have such features as booking automation and 24/7 access. Nazer et al. (2025) support this finding by presenting a system that was implemented and tested. Their proposed web-based platform created for a vehicle-renting application in Kerala, India provides users with the possibility of searching for available vehicles, making reservations, and paying for their service. It also provides administrators with the opportunity to manage bookings, inventory of vehicles as well as payment records from a single interface. The testing of the platform showed that it made more than one hundred bookings and that the vehicles were utilized on the level higher than 75 percent of the fleet suggesting that the platform was able to perform the booking functions as well as the management aspects without help of any of the facilities of the future technology. However, according to the researchers, the platform does not have features such as using GPS for tracking the fleet and needs to be developed into

a more innovative product. This point corresponds to the study of Wong Wei Min et al. (2021), which emphasized the importance of vehicle location tracking.

Building on this gap, Wong Wei Min et al. (2021) focus on the security problem that appears once a vehicle leaves the booking stage. Their study identifies two problems as the reason for building a hardware-level tracking solution. One issue is the growing incidence of car thieves targeting individual car owners. A second issue concerns smaller car rental companies that worry that they would not be getting their cars back by the end of the rental period. In response, the researchers built a vehicle tracking system that combines an Arduino IoT platform with GPS and GSM modules, paired with a web-based application that lets owners monitor their assets in real time. It provides updates about location in two ways: through SMS and web dashboard. What makes it uniquely distinctive is that it has several functions incorporated that are normally not available together in the same system. Among these functions, there are login and user verification, the means for controlling the devices for tracking, and the means for storage of numbers to be informed immediately. It is likely the most useful for car owners and small companies renting vehicles, as with the help of this system they can monitor their vehicles even from afar. The analysis indicates that no affordable system solving the issue of theft and return of vehicles is available, as this function was never designed for the booking software. By using IoT hardware along with the web app, this platform ensures real-time tracking for small businesses that cannot afford large fleet management solutions.

Wong Wei Min et al. (2021) illustrate what it means to build a vehicle tracking system that relies on rental tracking. Alhwety and Elfadil (2024) then expand on this by conducting a systematic review of vehicle tracking methods, both active and passive. In this context, active tracking is viewed as the better option, as it provides real-time location tracking, geofencing to notify an owner that their vehicle has crossed the predetermined barrier, as well as data transmitted over cellular networks or satellites. As for the outcomes of the review, fleet operators with an advanced vehicle tracker were pleased to notice that they could track their fleet vehicles much more effectively and use geofencing to minimize thefts and misuses of their vehicles. Fleet and asset managers gain the most advantages from this technology, and it is pointed out in the review that real-time vehicle tracking with alerts has already become a standard for similar companies. However, the review raises one technical concern. GPS accuracy decreases in urban areas as tall buildings and structures obstruct the signal. This creates blind zones in gathering information when signals are crucial. Alhwety and Elfadil (2024) mention that research must address the issue of enhancing GPS accuracy in city blind zones and noted that none of the systems they examined has managed to find way around this problem.

Synthesis

Together, the five studies reviewed indicate that vehicle rental management consists of two separate yet interrelated issues. Firstly, there is an organizational issue because manual booking systems lead to failures in scheduling and limitations in service availability. Secondly, there is an issue of

security as giving out the vehicle to the customer creates a visibility gap that cannot be bridged by the booking system alone. Golo and Encarnacion (2024) and Kashmim et al. (2023) address the organizational issue by suggesting web-based platforms designed to automate booking, improve the accessibility of the fleets, and give users more control over their reservations. Nazer et al. (2025) tackle the same organizational challenge by presenting their working web prototype that allows users to take control of vehicle searches and reservations, with its functionality demonstrated through actual booking results and outputs from the admin dashboard. Wong Wei Min et al. (2021) and Alhwety and Elfadil (2024) address the security challenge, specifically, the former offers an already functioning IoT system of tracking aimed at addressing the issues of theft and non-return of the vehicle, while the latter provides an organized review of the advantages and shortcomings of existing tracking technologies. Four out of the five studies discussed involve the use of web-based interfaces that connect administrators, customers, and fleet data, with Nazer et al. (2025) providing an example of an admin dashboard that integrates the functions of reservation, inventory, and payment into one platform for use by administrators. In contrast, Alhwety and Elfadil (2024) review the use of tracking technologies in existing systems without implementing an application layer themselves. GPS technology is also a recurring point of reference across the five studies. The authors themselves indicate that GPS technology is absent in the booking system of Nazer et al., but it is being actively used for security purposes by Wong Wei Min et al., while Alhwety and Elfadil review its applications and known weaknesses.

In spite of these similarities, the five studies position themselves in rather different locations throughout the problem space. Golo and Encarnacion, Kashmim et al., and Nazer et al. create booking-related systems that perceive the vehicle only as an entity to schedule, compare, and confirm, but the scope of their systems does not extend beyond the point when the vehicle is handed over to the customer; Nazer et al. (2025) make this boundary clear by naming GPS tracking as a feature for future development that has not yet been implemented rather than a current capability. Wong Wei Min et al. begin where the other studies stop by creating a system specifically designed to monitor what happens after the vehicle leaves a particular location. Instead of developing a system, Alhwety and Elfadil review the complete area and identify a technical difficulty that has yet to be solved by active tracking technology: the problems of GPS signals caused by physical obstructions in dense urban areas, which lead to gaps in tracking data precisely when constant coverage is most needed. Alhwety and Elfadil, therefore, identify improving the accuracy of GPS in urban areas as a direction for future research and view the issue of signal loss not as a difficulty that was solved by any of the systems reviewed, but as a known limitation of the technology. None of the five studies mentioned also consider this problem based specifically on the conditions of a congested urban area in Southeast Asia.

There still exists a narrower gap in the current research, which has been addressed by Nazer et al. (2025) who created a complete date-range reservation system that features fleet management on the admin side. but the system does not track a vehicle's security once a rental is active, unlike the system built by

Wong Wei Min et al. (2021). Therefore, out of the five studies considered here, none of them used the combination of structured pre-booking, tracking for the owner in the course of the rental, and data recovery during connectivity loss.

The developed system addressed these gaps through a web-based booking interface and database-driven management dashboard, drawing on the approaches of Golo and Encarnacion, Kashmim et al., and Nazer et al. Regarding vehicle tracking**,** Nazer et al. (2025) pointed out that there is a need to establish GPS tracking in future projects. The system also implemented software that can gather and process GPS data. Tests were carried out on the proposed system. However, the GPS module was not used with vehicles or during tracking under company operations. Also, Wong Wei Min et al. served as sources for theoretical insights into the potential use of IoT development for developing rental management applications, but their findings do not form a part of this paper.

The system developed also addressed the problem of connectivity, which although related to, is different from the GPS accuracy problem which Alhwety and Elfadil (2024) discussed. GPS positional accuracy in dense urban areas is susceptible to interference by physical obstructions such as high-rise buildings, and local data buffering cannot correct inaccurate coordinates. The system designed was not professing that it resolves this situational accuracy problem. Rather, it sought to address the chances of missing the readings when there was interruption in the functioning of the cellular network. The LILYGO TTGO T-SIM A7670E R2 module provides integrated GPS, GSM and processing on a single circuit board making it easy to set up the required hardware for a prototype

although without guaranteeing against power instability and loss of signals. One of the intended functions of the flash memory buffering was to save the telemetry locally in case of connectivity loss and to send it later once there is connectivity again. It is worth mentioning that the provided prototype tests did not indicate how the developed routine of the system will perform in real company rentals. With these features, the approaches from the reviewed studies were adapted to the needs of a small-scale Philippine rental operator, although their performance in actual operations still needed to be checked.

Internet of Things Architecture

The Internet of Things (IoT) refers to the devices that are connected to one another embedded with sensors, computing software, and networks that can analyze and send data with limited human intervention. The idea behind this is the wide presence of digital devices that by cooperating with one another and communicating with humans achieve a common purpose (Lombardi et al., 2021). IoT architecture is composed of multiple components of protocols, sensors, actuators, cloud services, and layers, that plays an important part in making the desired task possible (Samizadeh Nikoui et al., 2021).

There's no universal layer count or structure that fits every IoT system, this comes down to how different the applications are from each other. Still, when one dives deep on the research, the three-layer setup (perception, network, application) is the one that is cited often as the starting point, several papers describe it as the fundamental architecture that later, more complex models were expanded from (Rashid & Mustafa, 2023).

The perception layer gathers information from its environment using an RFID, GPS module, and microcontrollers. This layer is responsible for collecting data such as location, movement, and environmental conditions. The network layer will then transport this gathered data over the internet to be received by a waiting IoT endpoint using wired or wireless connection. Upon receiving, the application layer will now deliver services and provide feedback to the end user based on the data gathered from the lower layers. Enabling support for different areas of work. (Rashid & Mustafa, 2023; Lombardi et al., 2021).

For vehicle tracking systems, this layered model seamlessly supports GPS and GSM-based applications. Islam Akanda et al. (2022) describe a tracking architecture where the GPS module (perception layer) calculates coordinates, and the GSM module (network layer) transmits this data to enable SMS-based access. This transmitted data is then rendered by an application layer that displays the vehicle's real-time position to the user. In a similar point, Saad Salih and Alsaedi (2023) implemented an architecture that integrates an in-vehicle hardware unit with a smartphone application for continuous monitoring.

Real-Time Protocol using MQTT

Message Queuing Telemetry Transport, or MQTT, as defined by (Shahri et al., 2022) is a lightweight publish-subscribe messaging protocol built specifically for constrained devices and low-bandwidth, unreliable networks, which makes it a natural fit for GPS trackers running over cellular data.

There are several ways an IoT device can get its data to the application layer, the two most compared are HTTP and MQTT. HTTP follows a

request-response pattern, where every exchange requires opening and closing a connection, which adds an overhead device reporting every few seconds which may be costly (Jara Ochoa et al., 2023). MQTT differs because it uses a publish-subscribe model instead, publishers (perception layer) and subscribers (application layer) do not communicate directly to each other, an MQTT broker mediates the two and handles the routing by information and is sent only to the subscribers who are set to receive that information. This makes MQTT a reliable option for continuous transport compared to HTTP, since the connection stays open and does not need to be re-established for every single data point (Jara Ochoa et al., 2023).

MQTT's reliability comes from a feature called persistent session. Quality of Service (QoS) is the mechanism MQTT uses to control what happens to data between publisher and subscriber in the events that a session drops connection likely due to signal disruption. QoS 0, there is no guarantee that the message will be delivered; at level QoS 1, the message can be said to be delivered but might have duplicates, while at QoS 2 level, it covers a handshake between the publisher and broker containing four phases to guarantee a message is delivered exactly once and has no duplicates.

System Development

React is a JavaScript library that uses modular components in creating user interfaces. With its virtual DOM, changes can be made to certain parts of the interface without having to reload the whole page. Piastou (2023) compared major front-end frameworks and found that this approach helped React optimize

rendering performance as the application became more complicated. There was, however, still a need for optimization when dealing with more complex changes in web state. These findings support the use of React in a rental system where car availability and booking status need to be updated dynamically in real time, without requiring the user to reload the page.

On the server side, Fastify is a Node.js framework that allows developers to create RESTful APIs and manage the logic behind the system. In their study, Pratama and Raharja (2023) tested different Node.js frameworks using VirtualBox, Docker, and Podman. Requests, latency, and throughput were used to measure performance. Fastify paired with Sequelize obtained the best performance score in Docker and Podman among the configurations included in the tests. This gives some support for choosing Fastify to handle requests that arrive at the same time, including reservations and administrative transactions. The results, however, apply to the configurations tested. They did not establish the performance of the developed system with its particular database and deployment setup.

For data management, PostgreSQL is characterized in the literature as an open-source relational database management system (RDBMS) known for its support of concurrent transactions through Multi-Version Concurrency Control (MVCC) and advanced indexing techniques (Salunke & Ouda, 2024). Salunke and Ouda (2024) performed a comparison between PostgreSQL and MySQL systems and found that the former executes select queries up to 13 times faster than the other. Besides that, the overall performance of PostgreSQL remained

stable when select and insert operations were performed simultaneously. At the same time, MySQL experienced a noticeable decline in its performance under the same conditions. This is important for the booking system because the handling of requests regarding availability of cars and recording a new booking can happen concurrently. The functionality of the database should allow this without a decline in performance that could contribute to conflicting reservations, including double-booking.

Another key aspect to mention about security literature is that it discusses JSON Web Tokens (JWT), which can be defined as a token-based user authorization and authentication method where the information about the user and its right of access is kept in the signed token (Bucko et al., 2023). Bucko et al. (2023) implemented a JWT-based technique upgraded by behavioral indicators, such as the number of password attempts, IP address stability, and user-agent type, and found that the proposed technique increased the assessed level of trust in authentication. Such an approach is important to use when there is a need to differentiate access that is given to various user roles, each requiring separate authentication and authorization.

Finally, cloud hosting models provide a possibility of deploying web applications in such a way as to support dynamic scalability and fault tolerance, which allow organizations to avoid maintaining and provisioning physical server infrastructure (Kambala, 2023). Kambala (2023) states that key components of the structure of a cloud-based application are the high availability, load balancing, and auto-scaling, which allow applications to handle fluctuating traffic while

maintaining consistent performance. It is also mentioned that a stateless application design supports scalability, since any server instance can accept requests without depending on stored session data.

Database and Security

The safety of user credentials is directly tied to how passwords are stored. Rather than being kept in the written-out form, the newest internet uses hashing techniques like Bcrypt, which make it hard for attackers to crack the password (Skanda, Srivatsa, & Premananda, 2022). In the research carried out by Skanda et al. (2022), it was found that Bcrypt was even more efficient than hashing methods based on SHA-256.

The management of API keys, database credentials, and other environment variables used by the developed system was in accordance with the credential security principles. According to a systematic review of application secrets management practices, Basak, Neil, Reaves, and Williams (2022) stated that keeping secrets in local environment variables, separate from the application's source code, is one of the best practices of securing secrets and preventing leaks, provided that files are excluded from versioning. This has provided evidence for the decision to keep sensitive data of the developed system away from the source code.

An additional consideration for credential protection would include providing secure transit for data. This includes encrypting data transfers over the wire through the use of Transport Layer Security (TLS), which authenticates the communicating parties involved in the data transfer between the application's

front-end and back-end (Jangam, 2023). Jangam (2023) further mentions that TLS 1.3 performs better than its predecessors through easier pairing and enhanced confidentiality, meaning that should the secret key be compromised, previous messages will still be secure. Therefore, the protection of sensitive information such as user credentials and bookings is very important in ensuring the effective functioning of the system.

The same principle applied to how GPS data moved through the developed system, as readings from the tracking prototype had to be sent to the application layer. Paris, Habaebi, and Zyoud (2023) implemented and evaluated TLS-secured MQTT communication in an IoT environment and found that MQTT alone transmitted data in plaintext, while adding TLS increased communication and energy costs on constrained devices. The system's tracking design provided for TLS-secured MQTT transmission, but the reported tests did not evaluate a connection to a rental vehicle's main power supply. Such an installation remained a future deployment consideration. Neither TLS nor the prototype tests established that a rented vehicle's location could not be altered or that the device would remain continuously available.

Finally, protecting the system from fake bookings require verification from the users that they are who they claim to be. In two experimental studies on peer-to-peer accommodation platforms, Marth, Hartl, and Penz (2022) found that regulation features such as information indicated that user identity, reviews, and content would be checked and monitored reduced the potential consumers' perceived risk and increased their willingness to interact with the platform,

whereas commonly used trust-building measures, such as reputation systems and payment-security badges, had no significant effect on risk perception or engagement intention. This conclusion affirms the necessity of a basic identification check step for registered lessees in the process: based on the research result, the visible checks of user identification are the means of notably reducing perceived risks and enhancing the system of usage for the purpose of reducing fake orders and promoting trust in the process of dealing with car providers and renters who do not know each other.

Any gathering of personal information must be done in accordance with the Data Privacy Act of 2012 of the Philippines (RA 10173), according to which the issues of transparency, legitimacy, and proportionality are vital (Martirez, 2023). Martirez (2023) analysed compliance with this act in the Philippines and concluded that the degree of adherence to the requirements of the act is average. It is especially important for the developed identity verification process within the provided system: the only information which will be needed for the identity confirmation of any renter should be collected and this could mean obtaining a valid ID in order to proceed with further operations.

4G Network Reliability

When using the MQTT protocol, the reliability of the network is important for transferring data. Although the protocol uses a publish-subscribe model of communications, it does not solve all the problems related to the physical means of transmission. In a situation when the link can be interrupted, the guarantees that are provided by MQTT at the application layer must compensate for the

problems connected with the network layer. This argument was supported by Seoane et al. in 2021 by running an experiment on the topic. During the experiment, it was determined that with the increase of packet loss from 0% to 20%, the bandwidth and the delay of the packet have risen considerably. This is why three levels of QoS are presented in MQTT; QoS 1 and QoS 2 provide message acknowledgment and retransmission through the utilization of additional methods and the involvement of higher costs (Seoane et al., 2021).

Within a system that transports GPS telematics through a cellular service there exists an inextricable trade-off with respect to the reliability of the 4G/LTE link itself. The strength of the cellular signal does not remain constant since it depends on the distance from the base station, the type of area, and the presence of obstacles in the environment. This was confirmed by Ayad et al. (2022) through their field measurements of the 4G system in urban, suburban, and rural areas. They found that the received signal strength decreased with the increase of the distance from the base station while the signal quality increased with the height of the antenna due to the reduced number of obstacles in the path of the signal. Because a moving vehicle continuously changes its distance and orientation relative to the nearest cell tower, comparable fluctuation in signal quality can be expected over the course of a single trip. Gentile et al. (2024) observed a parallel effect when benchmarking MQTT itself across wired, Wi-Fi, and 3GPP-4G connections: message latency over the 4G link showed noticeably greater variability than over Ethernet or Wi-Fi, which the authors attributed to the

comparatively less controllable conditions of a commercial cellular network relative to a locally managed wired or wireless one.

Based on the findings of these studies, it can be deduced that the reliability of 4G in real-time telemetry is a compromise between the network being reliable or unreliable; instead, it is a question of the variations that the QoS system of MQTT is created to deal with. A good network minimizes the frequency of the mechanism's operation, however, even if the signal quality varies, the fact of getting the message to the specified server without losing it in transit indicates that the QoS mechanism has been successfully activated.

Synthesis

The examination of the five technical aspects indicates that there is a connected foundation provided for the developed system and its technology choices. Lombardi et al. (2021) present a general description of the IoT. The components of the IoT architecture and the challenges to consider in its design are examined in the works of Samizadeh Nikoui et al. (2021). Rashid and Mustafa (2023) classify them into three levels: perception, network, and application. The works of Islam Akanda et al. (2022) and Saad Salih and Alsaedi (2023) present evidence of the use of GPS and GSM in the layers for the purpose of tracking vehicles. A prototype for the IoT monitoring of vehicle speed, which includes hardware, cloud processing, and web and mobile applications, is developed in Khan et al. (2022). All of these studies support the layered structure of the developed system. Nevertheless, none of the studies test the specific hardware and software configuration used in this project.

In this setup, MQTT provides the connection between the tracking prototype and the application layer. Jara Ochoa et al. (2023) found that MQTT used less power than HTTP in their cold-chain monitoring system. Their finding supports the use of MQTT in IoT applications where conserving resources is a concern. Palmese et al. (2022) show that MQTT-SN QoS selection affects message delivery and latency under changing network conditions, while Reiher et al. (2022) demonstrate that an MQTT-based interface can achieve low latency in a specific 5G edge-cloud vehicle application. These results support the application of MQTT for near real-time telemetry. However, they do not provide a predetermined delivery time, nor do they guarantee uninterrupted communication. Seoane et al. (2021) indicate that adverse network conditions as well as security features can increase communication overhead and delay. Ayad et al. (2022) and Gentile et al. (2024) have just as much shown that performance of cellular varies according to the type of connection and transmission environment. Thus, MQTT QoS can improve the handling of recoverable message loss, but it is unable to deliver current location updates while the cellular connection is completely absent.

From the perspective of the researchers, the literature discussing system development and databases supports the technologies chosen for the web application. Piastou (2023) highlights the use of React in the interfaces needing fast and efficient updates. Pratama and Raharja (2023) indicated that Fastify with Sequelize in Docker and Podman proved to have the highest performance in the comparison of setups made in their work. Thus, this confirms that Fastify can be

used as the server-side technology; however, this should be stressed that the findings are valid only for the specific environment used in the study and do not make full assurance of the performance outcomes achieved using the configuration of technology in the developed system. Kambala (2023) provides architecture recommendations on achieving scalability and availability as well as fault tolerance for applications in the cloud. Salunke and Ouda (2024) show that PostgreSQL showed stable performance in concurrent select-and-insert operations, which are key for systems that need to retrieve the information about the vehicle availability as well as record the bookings. However, these sources do not give the results of the performance achieved by the full stack of React, Node.js, Fastify, and PostgreSQL in the conditions of the developed system.

The literature on security offers a similar technological foundation, but it makes no guarantees regarding the safety of any system that has been developed. For instance, Bucko et al. (2023) show the deployment and improvement of the technology based on JWT. Meanwhile, Skanda et al. (2022) provide support for Bcrypt as a protection mechanism for passwords. Both the studies by Jangam (2023) and Paris et al. (2023) contribute to the discussion of safeguarding transmitted data by means of TLS, yet Paris et al. correctly note that encryption results in additional energy and transmission cost. Additionally, Republic Act No. 10173 prescribed that all processing of personal data should be carried out in compliance with the principles of transparency, legitimate purpose and proportionality. Still, Martirez (2023) shows that organizations might fail to comply with these principles even if they are aware of them. All these findings

can be used to justify the authentication, encryption and data-collection policies employed by the proposed system.

The assessed investigations indicate the technological approach where vehicle data acquisition is performed at the hardware level, data is sent via MQTT over cellular communication, and data is processed, stored, and presented using a web application. However, such investigations were conducted with components of other systems and in other working conditions. The combination of LILYGO GPS hardware, MQTT, 4G communication, Node.js and Fastify, PostgreSQL, React and cloud deployment was not tested in the entire form in the conditions of actual vehicle route operations for 1816 Car Rental Co. The prototype developed includes a flash-buffering procedure, which provides data keeping during the interruptions and sending it when communication is available again, but it doesn't correct errors in GPS coordinates. The research did tests on coordinates checking, speed measuring, MQTT data transferring, and network connection checking, but it didn't provide the findings about the system reliability during real rentals.

Significance of the Study

Through the implementation of a centralized online Car Rental Management System with the incorporation of Internet of Things (IoT) GPS tracking capabilities, the following stakeholders of 1816 Car Rental Co are being positively affected by this research study:

Owner and Staff

The newly created system managed to consolidate bookings in a shared calendar available using owner and staff interfaces. In order to complete the verification process, renters provided government ID and live selfie online. Those who sought self-Drive access submitted driving documents. The dashboard allowed fleet monitoring by providing up-to-date GPS coordinates and speed findings, recorded routes, and alerts about the service area. The researcher performed all the tests with the prototype only while walking across the campus of PLP and using the motorcycle around Pinagbuhatan.

Renters

The system, in accordance with its functions, permits the users to access the information pertaining the vehicles that are currently available for hiring along with the provision of verification of documents prior to renting. This ensures that the reservations will reflect immediately on a common calendar and the customers will feel secured knowing that a confirmed reservation wouldn’t interfere with the making of other arrangements.

Statement of the Problem

At the present time, 1816 Car Rental Co. processes inquiries from its customers via Facebook Messenger and accepts bookings from people through interactions on the social platform. The requirements for the renters are warranted manually, payments can be processed in a separate way, and information about the location of the vehicle and its condition is preserved

through different approaches. Since the aforementioned actions is not embodied in a single booking request, the holders of this business are required to establish relationships between their customers and vehicles, scheduling of rentals, payments, and rental status manually.

This study's goal is to create a centralized vehicle rental management system to streamline the process of booking, executing transactions, and overseeing cars at 1816 Rent Company.

The specific objectives include queries that need addressing to achieve the above goal and which include:

1. What system can be developed for 1816 Rental Company to centralize its car rental management operations?
2. How can the proposed system improve the rental owner’s management of vehicle information, availability, booking schedules, payment confirmations, and real time monitoring of vehicles during active rentals?
3. How can the proposed system address the challenges affecting the secure and efficient completion of renter transactions, from registration and review of submitted identity documents to booking, payment, and confirmation?
4. How can the proposed system provide a consistent basis for documenting and comparing vehicle conditions before handover and after return to support a fair and documented review of vehicle condition?

5. What operational and financial reports can be generated by the proposed system to support the monitoring of bookings, payments, vehicle usage, fleet activity, and revenue?
6. What operational insights can the system provide to the owner through data analytics to support decision-making?

Scope and Limitation of the Study

The purpose of this research was to create a vehicle rental management solution for the 1816 Car Rental Company. The system merged the following aspects of the rental establishment into one application: requirements of renters, reservations, schedules of vehicles, payments of reservations, fleet records, certification of condition, GPS data and operations records. The application was adapted to the existing fleet of five vehicles available for the 1816 Car Rental Company. All of the administrative activities were assigned to the owner of the application. There was also the staff function of the application created to accommodate future employees of the company appointed by the owner. Access to the information in the system was limited according to the roles.

The renter will be able to get registered in the system and prepare the documentation for the owner of the system or the staff to review. Only renters whose documentation has been confirmed can proceed with the reservation.

The reservation process describes a vehicle chosen, date, destination, type of rental and charges for the rental. The bookings made in advance and ones waiting for a payment are considered in the vehicle availability calculations.

Each schedule created is checked with the existing ones to avoid booking conflicts involving the same vehicle and rental period.

After the reservation process is complete, the chosen schedule is kept for 10 minutes with a status of payment pending. The hirer is redirected to PayMongo payment gateway to pay PHP 1,000 as down payment for the reservation. Before confirming the payment, the system checks the signed PayMongo notification with the booking. Once payment is confirmed, the booking status changes to confirmed and the renters see it as Booked or Scheduled. A record of the transaction and invoice is created. Release of the vehicle requires the confirmed payment and completion of pre-handover condition record.

The developed system included a vehicle damage detection module which is based on machine learning to facilitate pre-rental and post-rental inspections. The module had the ability to analyze uploaded vehicle images and provide claims of possible visible damages to the vehicle body, such as dents, scratches, cracks, broken lamps, shattered glass, and flat tires. The damage detected by the system was displayed in terms of the location and the related confidence score to facilitate the process of reviewing the vehicle condition by the authorized persons. All the images and findings of pre-rental and post-rental inspections were saved along with the booking documents concerned. Damage assessment was made by the Owner of the vehicle only after examining the findings of the inspection by the Owner. Generally, the renter was required to review and confirm the final report before renting or releasing the vehicle. The system rules

provided for authorized direct handover override, owner or employee returns override, and automatic confirmation after 24 hours.

The system developed is used for GPS monitoring for active rental. Whenever the established module gives the readings, the system records such data like location coordinates, speed, time of recording, the status of the device. This gave the possibility of viewing the last position, showing location history, finding the route in the maps according to recorded GPS points. The system has also provided for the possibility of being able to receive delayed readings once the connection has been established.

The platform permits requests for refunds due to disasters. If the disaster happens after the booking has been completed but before the vehicle pick-up, only the owner of the platform has the authority to state or update the appropriate period. Renters who have completed their bookings in this time period may apply for a refund.

The platform uses rental history and operational information collected in the system for preparation of management reports and statistics. Reports reflect information about bookings and payments made, vehicle usage, fleet operations, and revenue for the periods selected for report generation. Statistics reflect trends and comparisons for the selected dates, vehicles, payment methods, and booking times.

During registration, renters are expected to present valid identification documents. Optical Character Recognition (OCR) technology in the system shows the IDs but also requires the renter to undergo guided live head-turning

verification. Selfies together with prompt, timestamp and motion score obtained from this step are saved for manual verification. While active checking entails rejection of static IDs, it lacks face recognition capabilities that make the procedure immune to replay and deepfake attacks. The system does not access the Land Transportation Office (LTO) database, the National Statistics Authority, or any other database that can confirm if the ID is genuine or untouched. Identification documents and ID details extracted by the system, live selfie, and booking requirements are reviewed by the owner or the employee to make the decision on the eligibility of the renter.

The availability of vehicles solely relies on booking records registered via the given platform. Booking inquiries received via Messenger, phone calls or by other means cannot reserve any vehicle until an actual booking is completed via the given platform. If there are gaps in the records, the displayed timetable may not be accurate.

Paymongo can receive and confirm only the downpayment of PHP 1,000. The rest of the rental fee is registered manually and must be settled outside the Paymongo flow before the vehicle is returned. Payments not made in reserve for holding a vehicle for 10 minutes may stop being effective, and the deposit paid only via the given platform may be subject to cancellation following the no-show or renter-release terms. All the other cancellations that took place previously are still done manually. Refunds can only be claimed in case of an eligible booking, but the calamity period must be declared by the owner. The refund can only be

approved, denied or settled by the owner, while a refund transfer actually occurs outside the platform.

The company utilized software to manage its fleet of five vehicles. The testing of GPS hardware entailed only the researcher-operated prototype activities at the PLP campus, where walking and motorcycle riding was done at Pinagbuhatan. The testing focused on GPS coordinates, speed, MQTT, and networking. The module was not installed in vehicles rented for research, nor was the fleet-wide GPS evaluation accomplished in the course of the rentals.

The GPS monitoring relies on electric power, satellite, and connectivity. In case of a connection interruption, the vehicle's location and route information would not be updated. The information collected might become part of the route history only if it is stored locally and then released once the connection is restored.

The software has been set up for approved locations within the Marikina-La Union operational region. The maximum distance permitted by means of the straight-line geofencing is 265 kilometers. It should be noted that the transit corridor through Central Luzon and the boundary of the Province of La Union does not mean an assurance of making it through 265 kilometers of travelling by road. Therefore, travelling north from La Union, east to Benguet in the outlying area, the whole of Bicol Region or inter-island destinations is not covered by the service structure mentioned above. When reserving a car, the Renter indicated the destination, which was checked with regard to the specified geofenced area. The Owner/Staff could see the information on the reservation as

well. Apart from that, the GPS alerts are used for a more accurate destination approval. Use of the route history is recorded in GPS points, which means some gaps in the route recording may be present.

The records of vehicle conditions and machine-learned detection are only for supporting evidence and need to be reviewed by Owner before they become final. The model cannot identify the cause or the extent of the damage, the liability for damages caused by the renter, the costs of the repairs, or responsibility for damages. The accuracy of the detection can be influenced by the quality of the photographs and environmental conditions. The necessary angles of the car should be captured, and there can be some extra pictures and 3 walkaround videos of 50 MB each uploaded optionally.

The analytics depend on the information in the system and its quality in terms of completeness and correctness. The forecast prediction, external market analysis, formal accounting, profit calculation, and automated management decisions are beyond the scope of the analysis.

CHAPTER 2

METHODOLOGY

This chapter described the methodology used to develop the Web-Based Car Rental Management System with Real-Time IoT-Based GPS Tracking for 1816 Car Rental Co. It covered the research design, system architecture, algorithms, development process, participants, data collection instrument, evaluation procedures, and ethical considerations.

Research Design

The study used the descriptive-developmental research design as described by Barroga et al. (2023). Descriptive research is defined as the approach that studies the situation, groups, and variables in a no-interference manner. The design and developmental research, according to Ishak et al. (2021), can be viewed as a systematic and iterative process of developing, testing, and improving a product. In the present study, the descriptive part investigated the rental practices of 1816 Car Rental Co., while the developmental part dealt with the development, implementation, assessment, and revisions of the vehicle rental and tracking system.

In the course of the research, the descriptive part described the way the company deals with renting including requirements for the renters, availability of the cars, reservations, payment, car handover, car returns, record keeping, and fleet monitoring. To achieve this goal the authors had to collect data on the methods used within the company and the possible issues concerning the

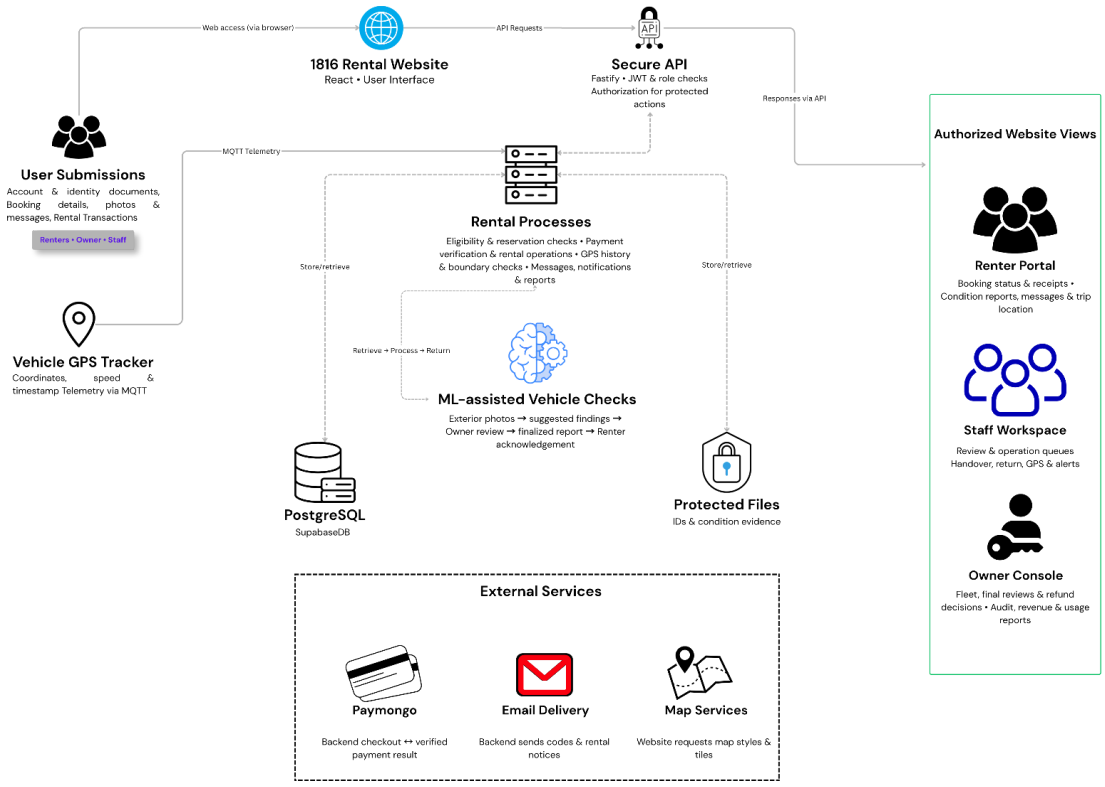
operation of the system through an online interview with two owners of the company.

The development phase utilized the identified specifications to facilitate the development of the web-based rental and tracking system. The process involved requirement analysis, architecture and database design, interface development, system integration, testing, evaluation, and adjustments. The developed system offered role-based access, application of requirements by the renters, booking of the vehicles, payment processing, maintenance of the conditions, transfers, notifications, and IoT monitoring. The software performance and functions were evaluated and tested.

Architectural Framework

Figure 1

Architectural Framework



As shown in Figure 1, the structure used the IPO Framework. The elements in this instance would be user submissions and inputs from the GPS tracker. The Renter Portal collected the user account details, ID proofs, booking details, vehicle snaps, and user messages. The Owner and Staff screens contributed to the decision-making process, as well as updates on the rentals. The tracker gave inputs like location and speed and the timings of the readings.

The technique was formed using the 1816 Rental Website, Secure API, and Rental Process. The clients would use the browser to open the website to

input information and check the status of their rentals. The Secure API checked for the user's role before allowing them to access any records. The renter, thus, had only limited access to their booking records, while the Owner would be able to get the rates and business reports.

The procedure of rental verification dealt with verifying the conditions prerequisite for the transaction, vehicles, reservations, and payments. Furthermore, the rental process contained information about delivery and collection logs, communication and notifications. GPS data constituted the history of position and was used to check if the area was approved for travels.

The function of Vehicle Checks using machine-learning technology checked images taken before the vehicle delivery and at return time with an aim to identify possible damages to the exterior of the vehicle. After that, the Owner examined the outcomes, made amendments, and verified the report.

The PostgreSQL database kept information about accounts, reservations, payments, vehicle inspections, and GPS movements history. The function of the secure storage fulfilled the task of keeping ID and images of the vehicle in the secured mode. Third-party services assisted with particular procedures: PayMongo provided the payment page, Email Delivery supplied proofs of rental verification, and Map Services provided maps for routing.

The results were showcased using Renter Portal, Staff Workspace, and Owner Console. The Renter Portal provided booking information such as status of bookings, receipts, confirmed condition reports, messages, and active-rental location details, if applicable. The Staff Workspace helped with reviewing,

monitoring the handover and returns, GPS information, and alerts. The Owner Console supplied fleet information regarding final condition assessments, refunds, audits, and payment reports.

Level 0 Data Flow Diagram

The Level 0 Data Flow Diagram (DFD) outlined the 1816 Car Rental Management and GPS Tracking System. It represented the platform as one central process and illustrated its exchanges with seven external entities: the Renter, Rental Owner, Staff, GPS Tracking Device and Service, Payment Gateway (PayMongo), Email Delivery Service, and Map Service.

Figure 2

Level 0 Data Flow Diagram

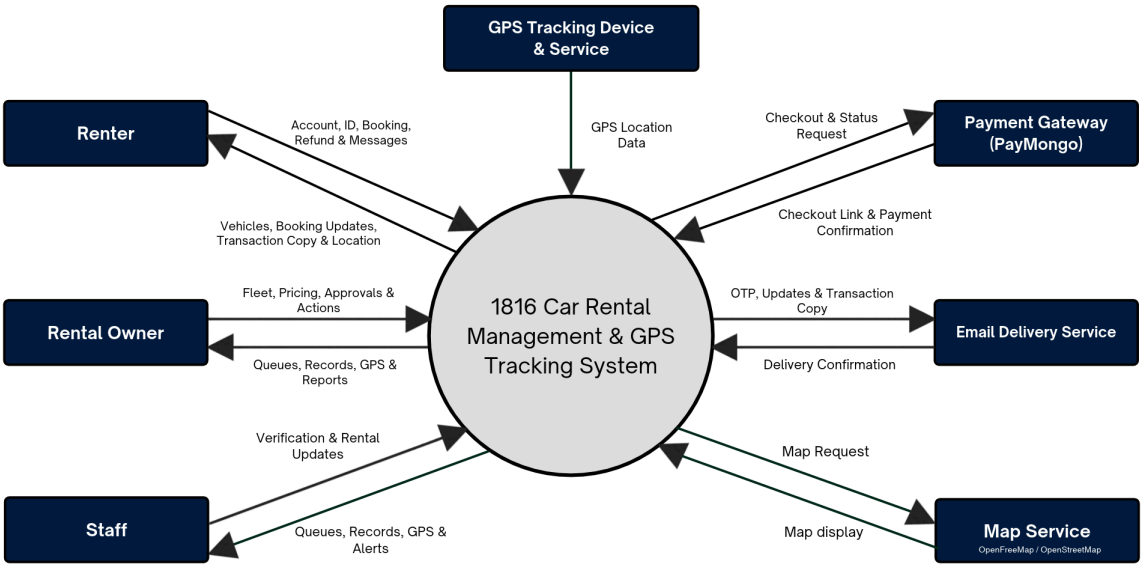


Figure 2 presented the platform as one central process that accepted information from users and connected services, applied rental and access rules,

and returned the records, updates, confirmations, and displays required by each external entity.

1816 Car Rental Management & GPS Tracking System (System Process):

The system in Level 0 is described in Figure 2. According to this level, the system boundary exchanges were emphasized over internal operations of screens, databases, and background services. The platform was represented with a big circle and the roles, which played a part in the everyday performance of the system were represented with seven boxes around the circle.

The Renter, Rental Owner, and Staff entities exchanged information with the system, whereby data is supplied and output is received respectively. The GPS Tracking Device and Service provided with speed and location data, which is delayed and marked as backfilled; Furthermore, the platform did not provide movement instructions to the tracker. PayMongo provided with checkout creation and status request and returned a checkout link, or payment verification. The Email Delivery Service was used for OTP, account and booking messages as well as reminders and receipts.

The relationships illustrate the platform as the focal point regulating the activities of users with respect to vehicle monitoring, money transferring, mail handling, and displaying maps. The arrows indicated the flow of information in the data exchange process, whereas the simplified captions defined the data without illustrating its reproductive stages.

External Entities:

- Renter: The platform facilitated the provision of account information, identification documents, Self-Drive documentation, booking details, and messages. It provided updates on vehicle booking issues, documents, issue of receipts, and information on the destination of the rental vehicle whenever available.
- Staff: The workstation assisted with identification decisions and Self-Drive process updates that included determining vehicle condition, delivery, pickup, and payment methods. It provided information related to rental operations, messages, and notifications.
- Rental Admin: The console supported reports on fleet and pricing updates, refund decisions and settings. It provided data on pending, operational processes and audit history of rentals.
- GPS Tracking Device: The location and speed of activities were transmitted through the MQTT protocol. The system saved all tracking information until the session was completed.
- Payment Gateway (PayMongo): The integration has sent checkout-creating requests and status requests to receive checkout links and verify payments via either signed webhooks or checkout session status responses.

- Email Delivery Service: The integration sent OTPs, decisions about verifications, booking updates, reminders, and receipt emails, and received results for each sending request.
- Map Service: The integration requested and received style and tile info for the renter and management maps. The routes were derived from the system GPS telemetry and not provided by the map provider.

Data Flows:

- System Arrival: The platform took in account and document submissions and booking data as well as refund requests and messages from the Renter interface; fleet data, pricing data, approval information, and management data from the Owner interface; verification and updates about rental operations from the Staff interface; telemetry data from the GPS Tracking Device and Service; checkout links and payments status from PayMongo; and results from Email Delivery and style and tile from the Map Service.
- Outgoing from System: The platform provided vehicle information, booking updates, receipts, finalized condition reports, notifications, and active-rental location data to the Renter interface; queues, records, GPS information, audit history, and performance reports to the Owner interface; and queues, records, GPS information, and alerts to the Staff interface. It sent checkout and status requests to PayMongo, messages to Email Delivery, and map-display requests to the Map Service.

Selected Data Flow Examples:

Table 1

Selected Data Flow Examples

From	To	Data Flow	Example Description
Renter	System	Account, ID, Booking, Refund & Messages	The Renter interface accepted account details, identity or Self-Drive documents, booking details, refund requests, and messages.
System	Renter	Vehicles, Booking Updates, Transaction Copy & Location	The system provided vehicle information, booking status, receipts or transaction records, finalized condition reports, notifications, and active-rental location information when available.
Staff	System	Verification & Rental Updates	The Staff interface supported verification decisions, condition checks, handover, return, payments, and vehicle-status updates.
System	Staff	Queues, Records, GPS & Alerts	The system provided the Staff workspace with operational queues, renter and booking records, live or historical GPS information, messages, and alerts.
Rental Owner	System	Fleet, Pricing, Approvals & Actions	The Owner interface supported vehicle and rate updates, verification and refund reviews, settings changes, and other owner-only actions.

From	To	Data Flow	Example Description
System	Rental Owner	Queues, Records, GPS & Reports	The Owner console displayed pending work, operational records, GPS data, audit history, and revenue or performance reports.
GPS Tracking Device and Service	System	GPS Location Data	The tracker transmitted location, recording time, speed, and backfill status through MQTT. The system stored the data and associated it with an applicable tracking session.
System	Payment Gateway (PayMongo)	Checkout & Status Request	The system requested hosted-checkout creation or the current checkout-session status from PayMongo.
Payment Gateway (PayMongo)	System	Checkout Link & Payment Confirmation	PayMongo returned the checkout link and supplied verified payment information through a signed webhook or status response.

Algorithm Discussion

The workflow utilized for the handover and return process involved the use of photographs with which the condition of the vehicle can be recorded. The damage detecting module accessed the photos and then indicated the portions of the photo that needed to be reviewed by the Owner. This module relied on the pre-trained CarDD YOLOv8s model that recognized the following types of damage: dent, scratch, crack, broken glass, damaged lamp, and flat tire. YOLO is an acronym, meaning "You Only Look Once," and being responsible for the method of inspection of the photo in one go to find the damaged parts of the vehicle. Also, the model used instance segmentation to monitor the shape of the damaged areas. The system was based on abdullahg7/cardd-yolov8s:v2.0 model implemented in the Open Neural Network Exchange (ONNX) format. The processing of the issue was performed using the ONNX Runtime software.

Image Inspection and Letterbox Preprocessing

The initial step in conducting the damage assessment was to analyze both the size as well as quality of images. A warning was issued in case of images where resolution was below 640, due to which minute details become hardly visible in images. It was calculated to understand the entropy of images, which can be described as a measure of the diversity of pixels present in the picture; if images showed the 3rd entropy level or below, a low quality warning was issued as well. Regardless of this warning, it is was impossible to declare that the picture was of no value. The checks were used to establish whether the images

could be of assistance as evidence in reaching the decision on whether the vehicle suffered any damage or not.

The system made necessary adjustments to the size of the pictures to the default size of 1024 pixels by 1024 pixels for models. However, stretching a wider image might skew the automobile and its features, so the system resized the image in a proportional way followed by a gray padding using letterbox pre-processing. For an image of width W and height H, the scale was calculated by choosing the minimum value between the ratio of 1024 divided by the width and the ratio of 1024 divided by the height ($s = \min(1024/W, 1024/H)$). This value was used for resizing the image without changing its proportion. The finalized dimensions were rounded to integers while the padding moved the image to the center of the picture. For the padding a value of 114 was applied to channels of red, green, and blue colors. The transparency channel has been removed and the values were divided by 255.

YOLOv8s Detection and Classification

Once the data was processed, YOLOv8s produced indications showing areas that were presumably damaged. Every indication included a box that covered the area in question along with a score for the six damage types. YOLOv8s then picked the highest score type and took it as confidence value. Confidence refers to the extent to which the model supports the predictions rather than guarantee the correctness of the predictions. All indications with a score of 0.25 or lower were filtered out. For example, when a suspicion had a

score of 0.18 it was filtered out as opposed to a case where the score would be 0.80, which would go for further assessment.

The center position of the box, width, and height were used by the model to define the box. The system then calculated the corner positions by using x1 = xc - w/2, y1 = yc - h/2, x2 = xc + w/2, and y2 = yc + h/2, where xc and yc were the center coordinates and w and h represented the width and height of the box. The corner positions here denote the positions in the resized Image. To convert the corner positions back to the original image, the system used x = (x' - px)/s and y = (y' - py)/s to perform the reverse operation as per the values of x', y', px, py and s. The positions defined by the model were again checked if it posits inside the photograph boundary. All such boxes having width or height equal to one pixel or less are dismissed.

Class-wise Non-Maximum Suppression

One scratch can produce many bounding boxes within the same area. Non-Maximum Suppression (NMS) was applied to discard boxes that had significant overlap with other candidate boxes and to keep only the box that had the highest confidence level. The NMS algorithm started from the box that had the maximum confidence and started comparing other boxes with the boxes that were already selected. The comparison was done for boxes that belonged to the same damage category so the overlap of a dent box with a scratch box did not eliminate scratches.

Another important aspect that needed to be taken into consideration was the overlap value calculation. The overlap was calculated using the Intersection

over Union (IoU) method. The intersection indicated the common area of two boxes whereas union showed the total area marked by the two boxes together taking into account the overlap only once. Thus, AoU = overlap area / total area, or AoU = area of box 1 ∩ area of box 2 / area of box 1 ∪ area of box 2. As an example if the common area was 75 pixels and the total area was 100 pixels, the IoU was 0.75.

With the default IoU threshold set at 0.45, the process of getting rid of lower-confidence boxes commenced as soon as their overlap with the box of interest reached or increased from the value specified. While doing this, it was also necessary to perform the check for the box being inside another one, since even the small box may have a low IoU, referring to the same mark together with the big box. In this case, the area of overlap was divided by the box of smaller size. The value produced was at least 0.70 to delete the one with lower confidence according to the default settings. Thus, all these measures brought about the decrease of values identified multiple times, although a part of the results had to be checked by humans.

Segmentation Mask and Final Detection Output

When it eliminated possible duplicates, the system evaluated the form of all the deferred damaged region by means of a segmentation mask. In particular, a rectangle-shaped bounding box encapsulated the selected area, while the mask presented every pixel belonging to that region of the damage. Thus, the system used their common prototype masks combined with the coefficients specified for every sole candidate. Each coefficient regulated the contribution of

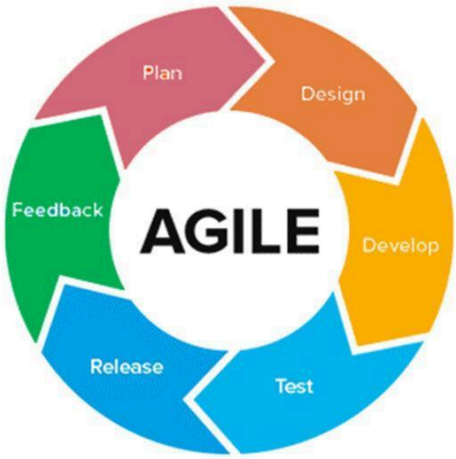
every pattern to its mask. The equation used was $M(x,y) = \sum ckPk(x,y)$. In this formula, the Pk was the value of a prototype in a particular point, and ck was the coefficient. The added values determined in the summation operation were referred to by the sign of summation ∑.

The mask boundary was mapped onto the original photo. This boundary was stored in normalized coordinates, which ran between 0-1. Such coordinates demonstrate the positions in proportion to the image. Thus, the results obtained retain their position regardless of the projected sizes. The boundary is represented via a convex hull. This shape could be made out of not more than 64 points. Although this representation is more convenient for storage, it may also involve some areas beyond a crooked mark. Each completed result comprises the type of damage, confidence score, normalized bounding boxes, and polygonal representation if such a valid mask was available. The area ratio is established as well. This ratio demonstrates the relevant box and not the exact amount of damage. The algorithm defined neither the moment of damage nor the liable person.

SE Paradigm

Figure 3

Agile Development Approach



The Agile approach was implemented in the development of the system of 1816 Car Rental Co., as seen in Figure 3. It was considered that the development can be divided into a set of manageable modules that can be tested, reviewed and corrected at the early stages before their dependent modules are developed.

Such an approach suits the functionality of the whole system. And many factors happened to have influence on the development of each module of the system. The functional requirements of the company and their evaluation work at development of each of the modules.

The entire development process comprised of 6 stages: Planning, Designing, Developing, Testing and Evaluation. In fact, these stages were applied to main parts of the system and not to the system as a whole.

1. Plan

While carrying out the project, the researchers identified booking problems, customer details that are kept separate from one another, manual payment confirming process, and vehicle position tracking issues as 1816 Car Rental Co's operational difficulties.

The researchers defined these problems as requirements and prioritized them according to their dependence on each other. In other words, essential services, like customer account handling service, vehicle sign-up and scheduling functions have to be working before actual services that depend on them can start working. Such requirements involve overlap protection, restricted access to renter's details, and provision of the current GPS readings. The GPS prototype testing was carried out only in researcher-controlled operations at PLP campus and Pinagbuhatan, as it was mentioned in Ethical Considerations.

To find these problems, the researchers conducted an online interview with the owner of 1816 Car Rental Co. The interview was aimed to find out the existing booking process at the company, and discover the requirements of the customers, payment process, car release and return, record keeping, and the problems faced by the company. The information received in the interview served as a basis for the requirements determined during the research.

2. Design

In the design phase, researchers created a user interface respective to the roles of the Renter, Staff, and Owner. Each interface was developed to show the

functionalities that correspond to each role. The renters could use the Renter Portal to browse vehicles and manage their bookings. The other management interfaces had functionalities related to preparing review, rental history, vehicle information, GPS information, and reports.

The researchers developed a database related to users, vehicles, bookings, payments, documents, inspections of vehicles, and information gotten from GPS. For tracking of data, readings traveled from their source to cellular network and through the MQTT broker. Once the data was transmitted to the backend, it was stored in PostgreSQL to allow displaying of available readings.

The security system was designed to allow authenticated access and permissions based on the roles. The personal documents, payment details, and evidence of vehicle condition could be accessed only by the authorized persons.

3. Develop

Features had been created by the team to bridge among the interface, backend, and database.

The booking feature comprised of a reservation form, schedule validation, and storage of booking records. The remaining functions are payment verification, review of renter requirements, handover and return, monitoring of delays, notifications, geofencing, and descriptive analytics.

4. Test

The process of development involved testing that covered registration, login, role permissions, document uploads, booking schedules, payments, and changes in booking statuses.

Booking testing involved conflicting schedules to check overlaps. Access-control testing checks involved restrictions on documents and location of private renters.

The GPS prototype tests involved only the verification of the use of coordinates, speed measurement, MQTT transmission, and connectivity while the researchers walked over the PLP campus and rode a motorcycle in Pinagbuhatan. The module was not fixed on the renter's vehicle. The evaluation of the system utilized role-appropriate scenarios performed by the owner, renters, and technical evaluators, and it was not an evaluation of the entire fleet.

5. Deploy

The deployment was made in a safe location. Preparations included web application, backend systems, database, file storage, security settings. The whole automated process was checked from registration and submission of documents to payment and handover.

6. Review

The features that were completed underwent evaluation according to the rental criteria provided by the company. It was possible to determine which

features needed additional focus due to input from the company's owner in the operation and evaluation process.

During the evaluation of system performance, a structured questionnaire has been utilized for the purpose of verification of software quality in accordance with completion of role-based tasks. Separate damage modeling test allowed comparing the expected existence of damage class with the reference tags in the CarDD test images. All actions mentioned above supported the evaluation results that are presented in Chapter 3.

The research team has reviewed the development process and the data received from the evaluation to recognize the strengths and weaknesses of the process. The findings made are used for revision and recommendations concerning further work on the project.

Population and Sampling

The system was evaluated by individuals who used, operated, or assessed its functions. Because it was developed for one small-scale rental business, the evaluation involved a limited and identifiable group of participants.

Population

The evaluation participants were grouped as follows:

Owner. One owner of 1816 Car Rental Co. evaluated the system from the perspective of rental operations and management.

Renters. Fifteen renters completed role-relevant tasks and evaluated the renter-side functions.

Technical Evaluators. Five IT professionals/engineers evaluated the system's technical functions and software quality.

Sampling Technique

The research applied purposive sampling to choose participants whose knowledge of rental business, prior use of the system, or expert skills allowed them to give pertinent feedback.

The evaluation included one owner, 15 people who leased apartments or rooms, and five technical evaluators, totaling 21 respondents. Purposive sampling led to relevant feedback related to the specific system of the company, rather than to population estimates or basis for generalizations.

Data Collection Methods

To collect the evaluation data, the researchers designed a structured system evaluation questionnaire that was administered using Google Forms. The system owner, user, and technical assessors completed their tasks according to their respective roles in the system and then filled in the questionnaire concerning the system’s behavior.

Instruments

The researchers implemented only one structured system evaluation questionnaire as their data collection instrument. It consisted of the same statements and response categories for each of the three evaluator groups.

The questionnaire consisted of 15 indicators that belong to the different system quality criteria, specifically functionality, reliability, usability, efficiency, portability,

and maintainability. The indicators covered practicality, accuracy, security, fault tolerance, recoverability, comprehensibility, learnability, comprehensibility, attractiveness, time characteristics, resource effectiveness, installation, substitution, and technical analysis capabilities.

The scale utilized in this questionnaire was four-point: 4 - Strongly Agree, 3 - Agree, 2 - Disagree, and 1 - Strongly Disagree. The means were interpreted as Strongly Agree for 3.50-4.00, Agree for 2.50-3.49, Disagree for 1.50-2.49, and Strongly Disagree for 1.00-1.49. Open-ended questions were posed so that evaluators could identify which function worked best and what task or screen was difficult and what could be improved.

Procedure

The evaluators received the system to perform the scenarios assigned to their evaluator type and thereafter completed the questionnaire that comprised 15 statements in Google Forms on the basis of their observations of how to system behaved during the tasks. The results were averaged for every subcriterion separately for the three evaluator groups. To find the Composite, the three group means were averaged equally; in this way, each group accounted for one third of the result irrespective of their sizes. The General Weighted Mean for every criterion was calculated as an average of its sub criterion Composites. The six criteria means were averaged equally to find the Grand Overall Weighted Mean. The intermediate means were not rounded and the reported means were rounded up to two decimal points and provided with qualitative evaluations.

Ethical Considerations

Throughout the development and assessment, an ethical framework was established that encompassed ideas of transparency, responsible use of information, and participant's safety. Machine learning played an assisting role in processing images of vehicles. Still, the verification, decision-making process, and final conclusions were made by people. The ethical requirements of the conducted research can be defined by:

Informed Consent and Voluntary Participation

Before engaging in any interviews, testing, and evaluation, subjects must be informed about aims and background of the research and the use of the gathered information. Participation in the study is to be voluntary and individuals always have the right to refuse to answer any questions, as well as to withdraw from the study at any point.

The registration included the clause concerning confidentiality and consent for the transmission and analysis of the submitted data. The privacy regulations covered the collected data, its purposes, access to information, and protection of its confidentiality. The reporting requirements excluded all unnecessary details and names of the respondents.

Data Privacy and Confidentiality

The research was carried out in accordance with Republic Act No. 10173, better known as the Data Privacy Act of 2012. The data scope of the system included account information, identification cards, and data on bookings and

payment history, messages, evidence of conditions, and GPS data. The data-collection policy required that data be collected in a transparent way for legitimate purposes and in proportion to the intended use.

The data-usage policy required that only the relevant information be collected for the purpose of verification of rent charges, making and confirming bookings, confirming payments, and returning the rented item. The policy did not allow for the use of collected data in any other way without obtain permission to do so.

The secured file storage included a technology capable of hashing passwords, validating access, and controlling access rights based on user roles, thus preventing unauthorized access to or disclosure of the information stored in the file repository. Moreover, confidential files could only be accessed by authorized persons.

Human Review and Fair Decision-Making

There was no use of artificial intelligence or machine learning in the verification of the identities, authorization of reservations, identification of violations occurring on the part of renters, determination of liability, determination of the costs of repairs, and development of charges. The condition model of automobiles gave only basic results. After receiving the results, the Owner could act in regard to the finding provided by the model and any extra data entered manually.

The system had no links to LTO records and did not provide automatic verification of the identity documents in the process. The OCR-based processing of data could suggest various document details for evaluation but did not assist in authentication.

The recordings done by the system were not sufficient for reaching an automatic decision on the legality of the actions. The protocol concerning the decisions made crucial points such as cases with rejected documents found, violations suspected, vehicle damages and payments by using only the available pieces of evidence.

Responsible Use of GPS and IoT Data

The regulation restricts the use of GPS technology to purposes of ensuring the security of a fleet of vehicles and ensuring the correct management of the rentals. It further discussed the inaccuracies that the readings of GPS devices might be subjected to because of factors like poor GPS signal reception,

power failures, etc. The telemetry data should be therefore considered auxiliary information but not be the basis for making accusations of wrong use of the vehicle.

During the pilot phase of the project, the IoT module was only slightly tested by the developers by walking around the territory of the PLP campus as well as by riding a two-wheeled vehicle in Pinagbuhatan. The unit had not yet been used in any vehicle rented by a customer. Testing was limited to clarification of the location coordinates and the speed of the transport and the transmission of the signals. The researchers did not collect any additional personal information during these processes.

Responsible Use of Vehicle-Condition Evidence

The workflow of inspecting has offered photographs, video images, odometer information, fuel consumption, and any other information necessary to confirm the condition of the equipment prior to handover and upon return. For this purpose, rules specified that irrelevant people, private talk, and property surrounding the equipment had to be excluded from photographs.

Thus, condition records and machine-learning results acted as supporting evidence on the matter. However, they do not automatically define the cause of damage, liability, repair cost, and charges. With help of the renter procedure, the report may be reviewed and accepted. The process of dispute reviewing involves the Owner or the Staff in the review of the available proofs.

Payment and Third-Party Services

The PayMongo system, used for reservation payment, was integrated into the platform. It did not keep any full card data or full electronic wallet information.

To meet privacy requirements, limited processing was done by external services used for rental service purposes. The services include PayMongo, OpenStreetMap and OpenFreeMap tiles, MQTT broker, PostgreSQL hosted by Supabase, and email delivery services Brevo and Gmail. Authorized usage of these services was limited to the services necessary for servicing of the platform only.

Data Retention and User Rights

In the case of retaining data, the retention policy stated that personal data should be retained no longer than is necessary for the performance of rental operations, research, legitimate business purposes and complying with law requirements. The study did not specify retention policies or procedures of deletion or anonymization of the data after the study is completed.

The privacy policy was formed by identifying the ways of making requests of access, amendments and other rights as foreseen in the Data Privacy Law. By the time of completion of the study, the policy did not include procedures needed for handling requests in this area. As to the incident handling policy, it contained requirements in relation to documenting the incidents and notifying agencies and stakeholders.

The principles stated the requirements of the study with respect to the rights of the participants and proper use of personal or location data. They indicated that an individual should perform the evaluation of decisions made and that the use of the system is limited to the purposes of fulfilling rental operations.

CHAPTER 3

RESULTS AND DISCUSSIONS

The current chapter summarizes the findings and discussions regarding the introduced system. The current chapter contains the relevant functionality of the proposed system, the findings from testing the vehicle damage model, and the evaluation of the system.

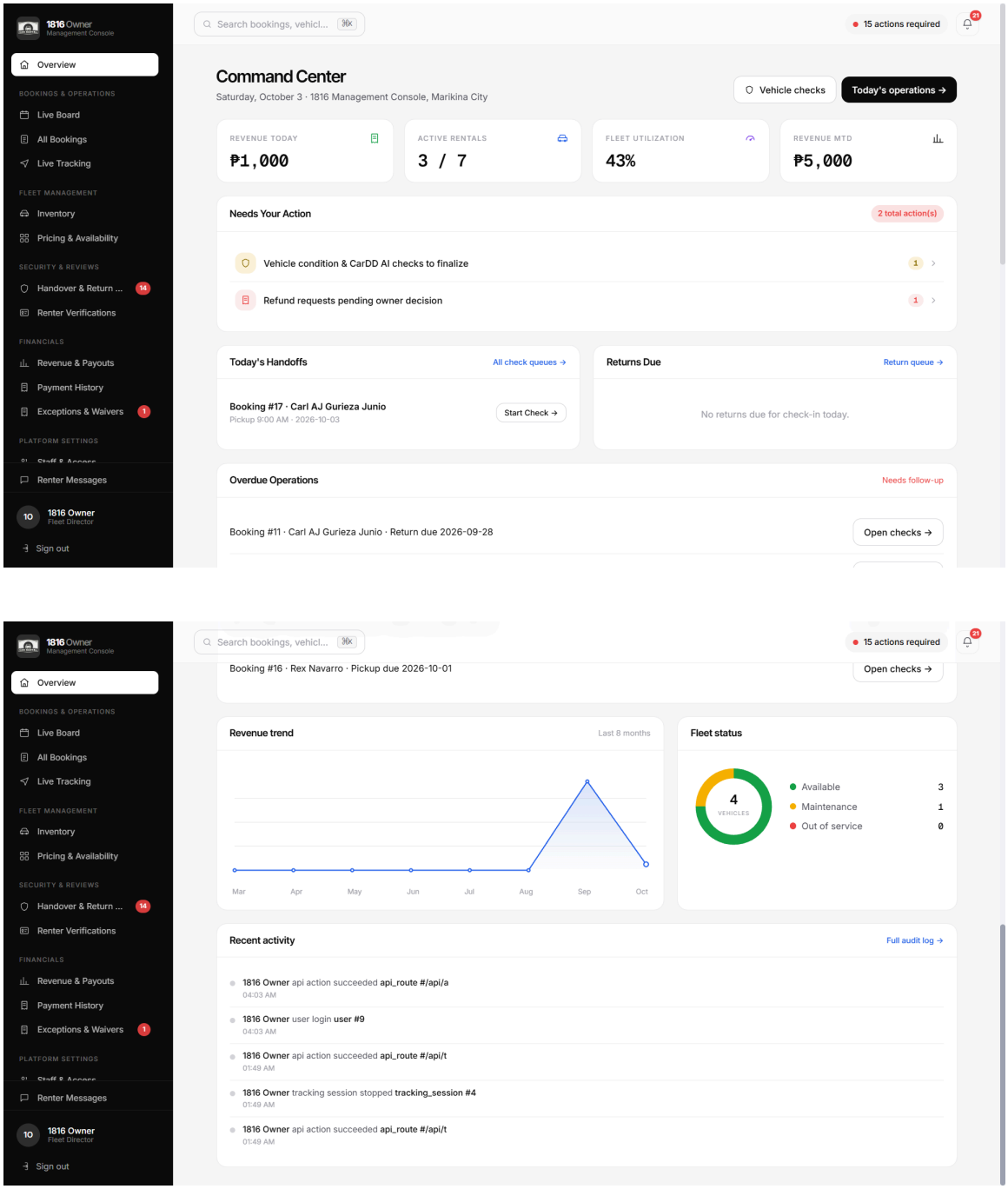
Project Capabilities

The system integrates three important aspects, i.e., the Renter Portal, Owner Console, and database, in one web-based system. The system gives renters an opportunity to create their personal accounts, provide booking requirements, check availability of vehicles, make reservations, and carry out payments through the PayMongo system. The system checks the booking dates in order to ensure that the bookings are not overlapping and stores the rate used for each booking.

The Owner Console allows the owner and staff to examine the booking requirements of renters, manage bookings, update vehicle and pricing data, take photos of vehicle condition during the handoff process and returning process, track the locations of vehicles, and review reports. The damage detection model provides its suggestions based on the images of the damage received.

Figure 4

Owner Command Center

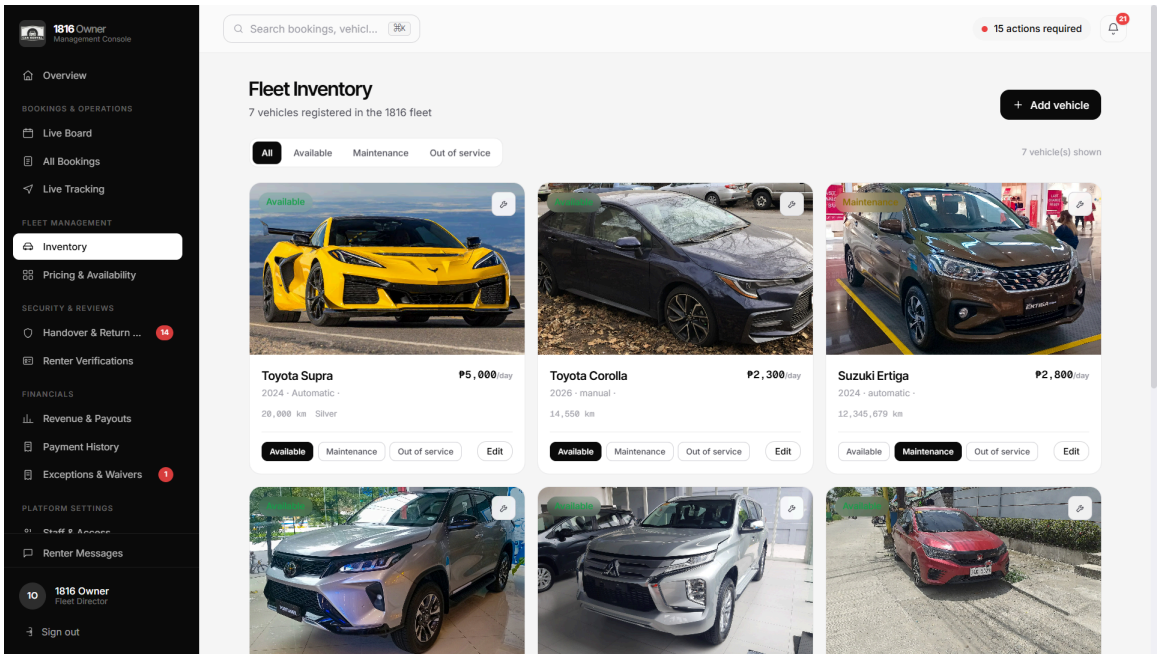


The Owner Command Center, is illustrated in Figure 4, and serves as the base point for the owner to oversee rental operations. It comprises four summary

cards which provides information about total revenue earned for the day, for the month to date, number of active rentals compared to number of vehicles in the system, and fleet utilization which is calculated from the number of active rentals.“Needs Your Action” queue counts pending renter document reviews, vehicle condition and CarDD assessment checks, refund requests, and reservations awaiting payment; selecting an item opens the corresponding work area. Today’s list of handovers and returns is displayed separately. The overdue pickups and returns appear as follow-up items in the list if they exist. The revenue chart shows the amounts earned in the previous months on the top half of the screen, while fleet status section distinguishes active vehicles in terms of available, in maintenance and out of service. The Recent Activity section shows all the actions performed in the system along with the link to the audit log.

Figure 5

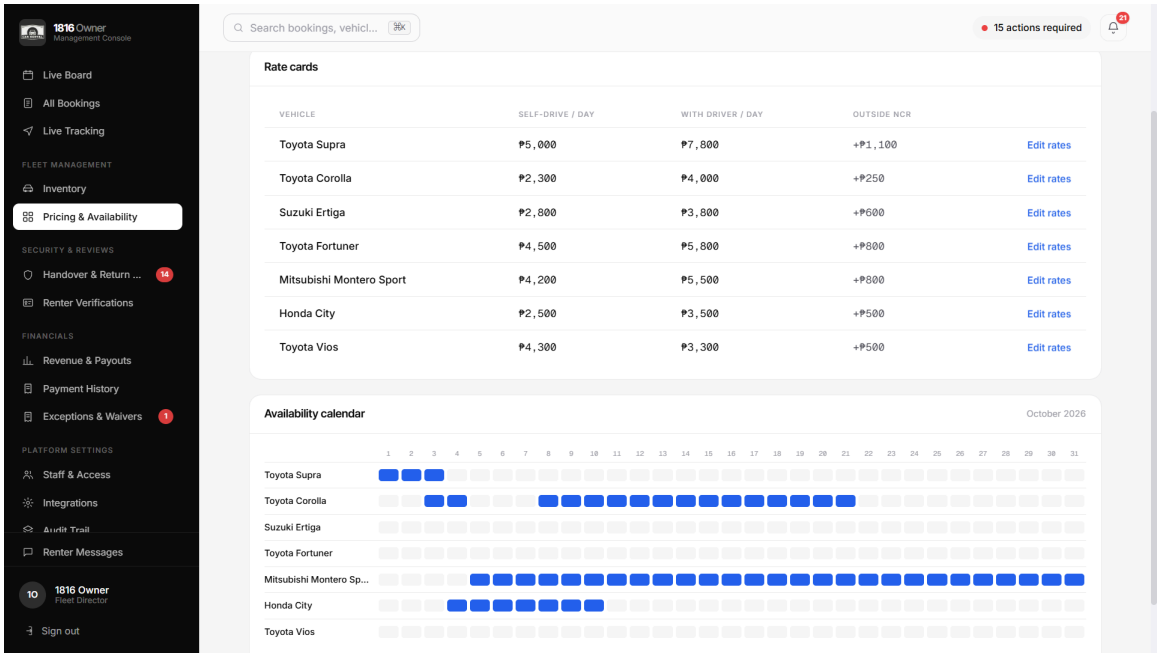
Fleet Inventory



The Fleet Inventory Page shown in Figure 5 depicts the implementation of the fleet management system, wherein the user analyzes the individual cards to review the details of the respective vehicles across the fleet. In this case, the vehicle card includes information such as the vehicle's image, parameters like the make and model, a few distinctive characteristics like the year and plate number, transmission type, odometer count, daily rental rate, current status, etc. Status filters help in finding a list of vehicles, namely all vehicles, available vehicles, vehicles under maintenance, or vehicles that are out of order, and the abbreviation reflected in the above area varies according to the filter selected. In addition, the user can select the 'Add vehicle' feature for entry of new data while being free to make an edit of the existing records using the 'Edit' button. Status indicators enable the user to check whether the vehicle is available, under maintenance, or out of order. The maintenance status indicator can provide historical information about the vehicle and a form for entry of the service type, date of service, cost, odometer status, and notes on the next service.

Figure 6

Pricing and Availability

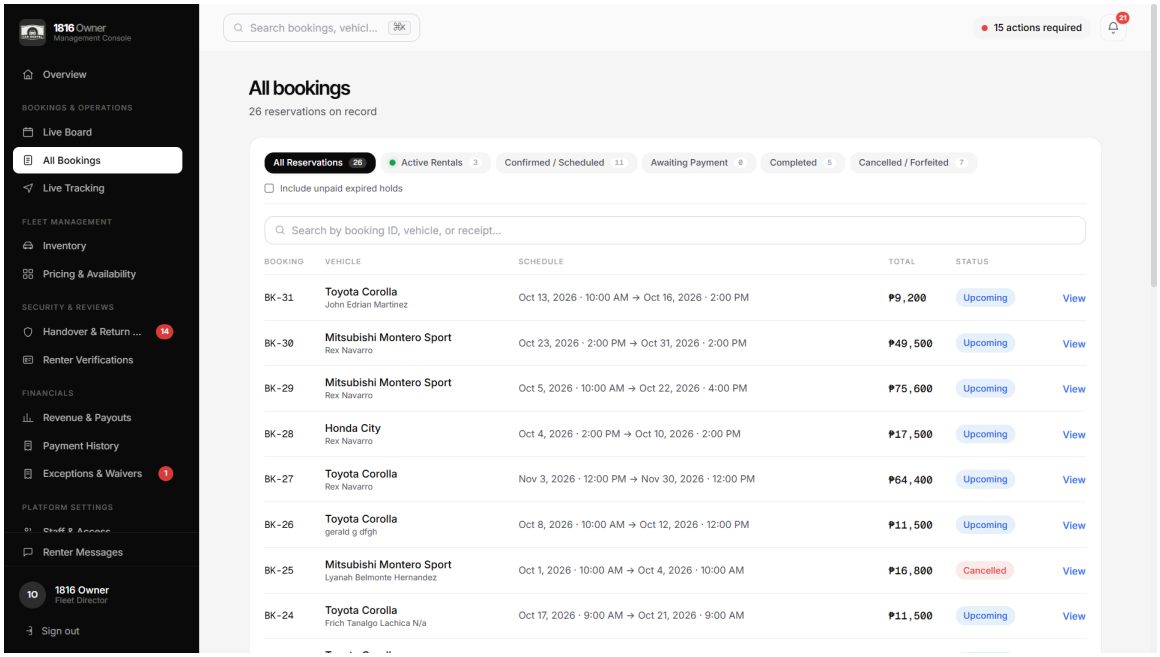


The Pricing and Availability page is as indicated in Figure 6 and is utilized for managing car charges and viewing the monthly itinerary. The rate chart shows the daily self-drive rate, with-driver rate, and off-NCR charge for the car. If Edit rates is clicked a digital form will open to allow for changes in costing of that particular car. The page notes that the modified rate is valid for any new booking whereas the previous booking holds on to its initial rate. Below the chart lies a calendar giving the month's itinerary by allocating one row for each vehicle and column for each date. Colorful boxes indicate the different dates for the reservations, for maintenance, for availability, and for the vehicles not in use. The calendar uses booking and maintenance data obtained from the database records that have been generated by the system. When a customer wants to

reserve a car for certain days, the booking process checks if the car is available or not; the page provides the owner with a visual calendar.

Figure 7

Booking Monitor

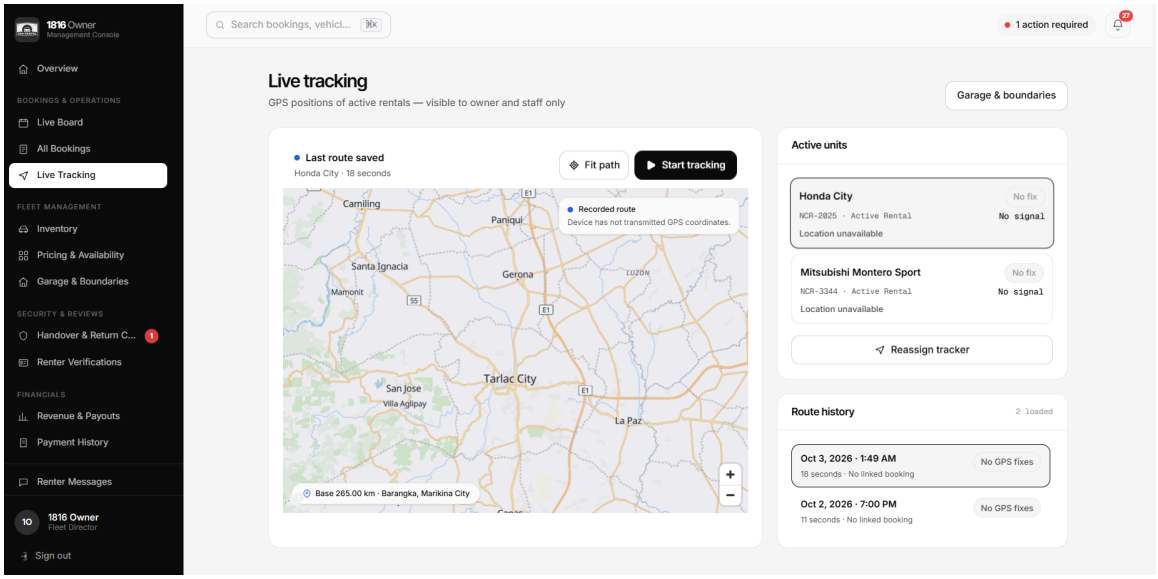


The All Bookings page is depicted in Figure 7, which is designed to present bookings data according to their current phase. The data are grouped into different tabs showing total bookings volume, active rentals, bookings that have been confirmed or are scheduled for future date, bookings needing payments, bookings completed and bookings which have been cancelled or forfeited. The function of including unpaid expired holds is realized through a checkbox, while the search function is activated through the search box entering booking number, vehicle name, customer name or receipt reference. The data per booking details booking reference, vehicle description, customer name, pick-up and return schedule, amount of booking and status of booking. To get

further information about the reservation activities, one should select the View function to open the reservation workspace with the details about the rental process and details of the rental offer provided to the client. The Full History tab includes the data on booking lifecycle, payment history, inspections, and messages about the reservation subject to category types and the time of occurrence. Actions depend on booking status, including noticing about outstanding balance and taking any decisions consequential to the booking

Figure 8

Live GPS Tracking



The process of already depicted live tracking is explained in details in Figure 8, where the owner or staff who has the active tracker selects a vehicle. When there is already a GPS fix, the map is going to display its route and location of the vehicle. There is an active panel that can show if GPS works, if the vehicle is moving, and if its speed is known. The operator will start the tracking by clicking on the start button to log the trip and will stop the tracking by

pressing the stop button. The result of stopping the tracking is that the session is still there and is stored in the telemetry data. During the tracking, the vehicle's location updates every 10 seconds. Fit path can show the current route of the vehicle. Below the sorting of active vehicles, there is also the route history where the last 20 trips of the vehicle are pictured. The date and time appear for the entry together with the time spent in entering the entry, as well as whether the GPS fix was recorded. When you select an entry, it will be running concurrently with the tracking session. Both entries being viewed will indicate "No GPS fix." The vehicle that the entry was selected from will also indicate "No fix," "No signal," and "No location." The current entries will still show in the database, but there will be no map path because no GPS fix was recorded for the current entries. The disappearance of the GPS coordinates should not be confused with Offline mode.

Figure 9

Renter Registration and Live Selfie Capture

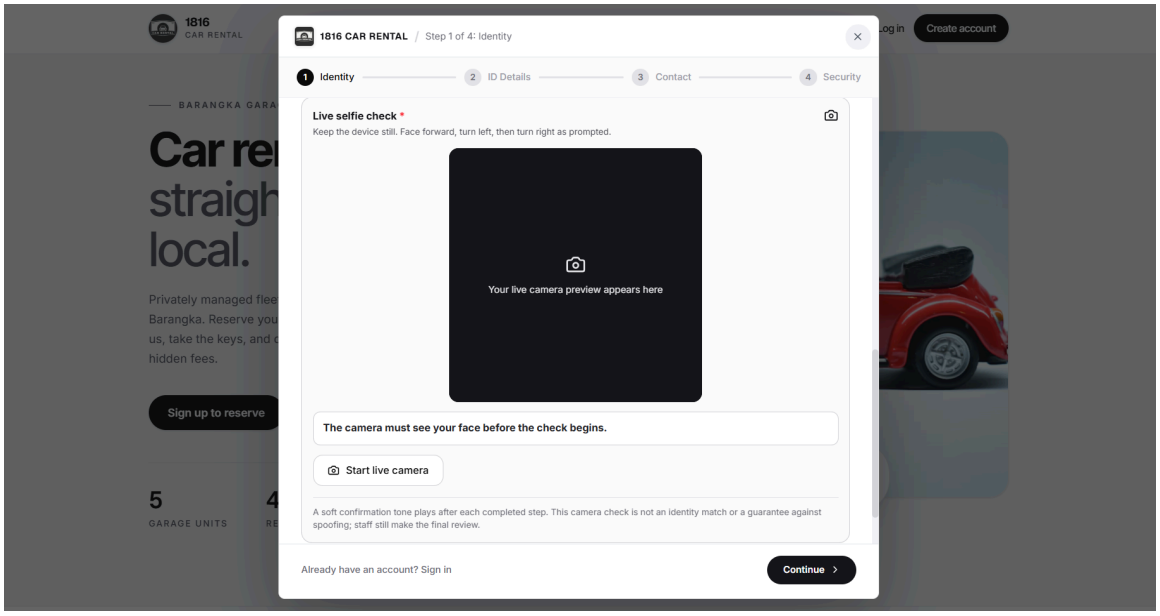
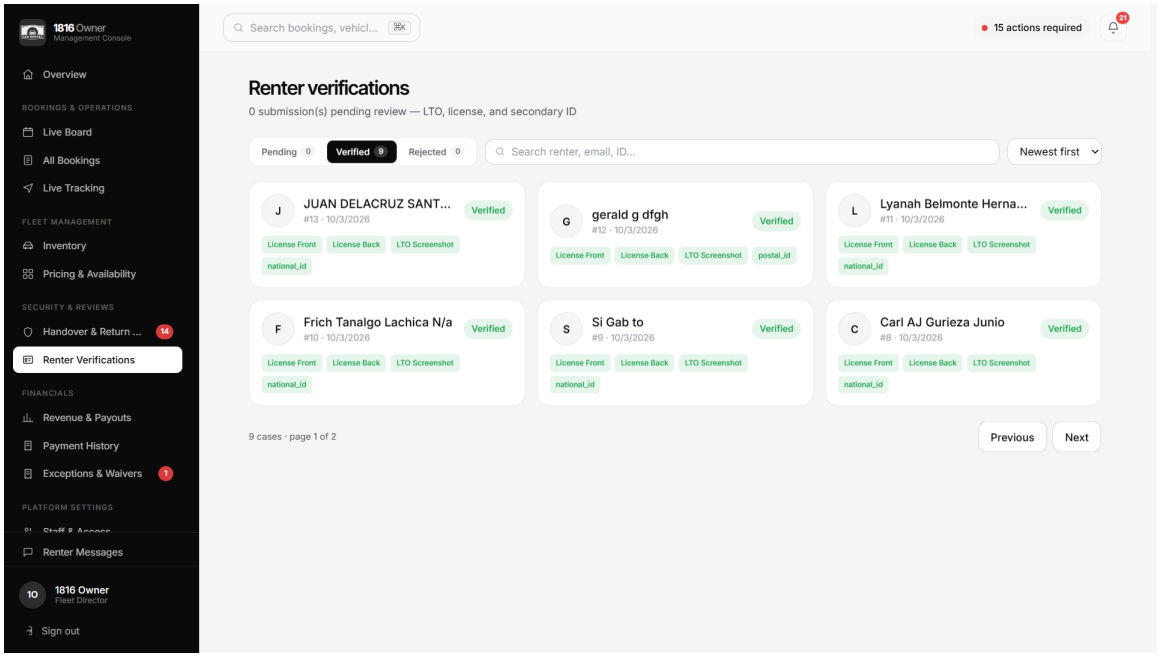


Figure 9 depicts the live-selfie verification process in the identity component of the renter's four-phase account-creation procedure. After accepting the terms of identity verification, the prospective tenant chooses a rental mode and a type of identification document before uploading a crisp frontal photo. Self-Drive registration calls for an up-to-date driver's license with the front and back, while Chauffeured registration lets the user submit any other authorized government-issued identification. The form allows for the extraction of identity information, which the applicant will verify and modify in the following Identity Information stage; information gathered is not automatically confirmed. During the live-selfie check, the user enters the camera mode, grants permission for camera use, and initiates the recognition of the face. The verification requires the use of facial identification points for the implementation of the sequence:

facing front; looking left; directly returning to the frontal position; looking to the right, and going back to facing front. Each step is done statically requiring compliance with audio confirmations whenever it is available. After the completion of the process, an automatic square selfie is taken. In cases when the face and the necessary turns are not detected. The Contact stage gathers data regarding electronic mail, personal residency, telephone, and emergency contact details. Security stage mandates password and its confirmation, acceptance of the Terms and Conditions and consent for privacy and identity processing, as well as a six-digit email verification code. The registration process activates the account and sends the identity data for further review of the owner or employees. The camera verification aspect does not provide automatic recognition of the face and does not safeguard against abuse of the identity. In addition, the process of obtaining Self-Drive access requires a separate approval procedure for specific driving documents.

Figure 10

Renter Requirements Review



In Figure 10 is depicted a sample of the Renter Verifications page that the owner or a member of the staff uses to look closely into the documents, presented by the renter. Tabs for the pending cases, verified cases, and rejected cases are shown in the upper part of the page, while the search and sort controls help in finding the required info. Each card indicates the renter, status of his/her submission, and evidence available regarding this case. Opening the case gives access to the pictures of the government ID of the renter, his/her registration selfie and the results of the movement check. There are also images of the driver's license with the information of expiry, showing that if the renter has submitted the Self-Drive documents, those images are also available within the case. The reviewer has to look at this data and make a manual decision for the pending scope. Approval of identity means the possibility of making Chauffeured

reservation, but Self-Drive authorization requires another approval of the driving documents and valid license. The pending scope can be rejected by the reviewer with the reason and can be recorded by the system.

Figure 11

Vehicle Details and Booking Form

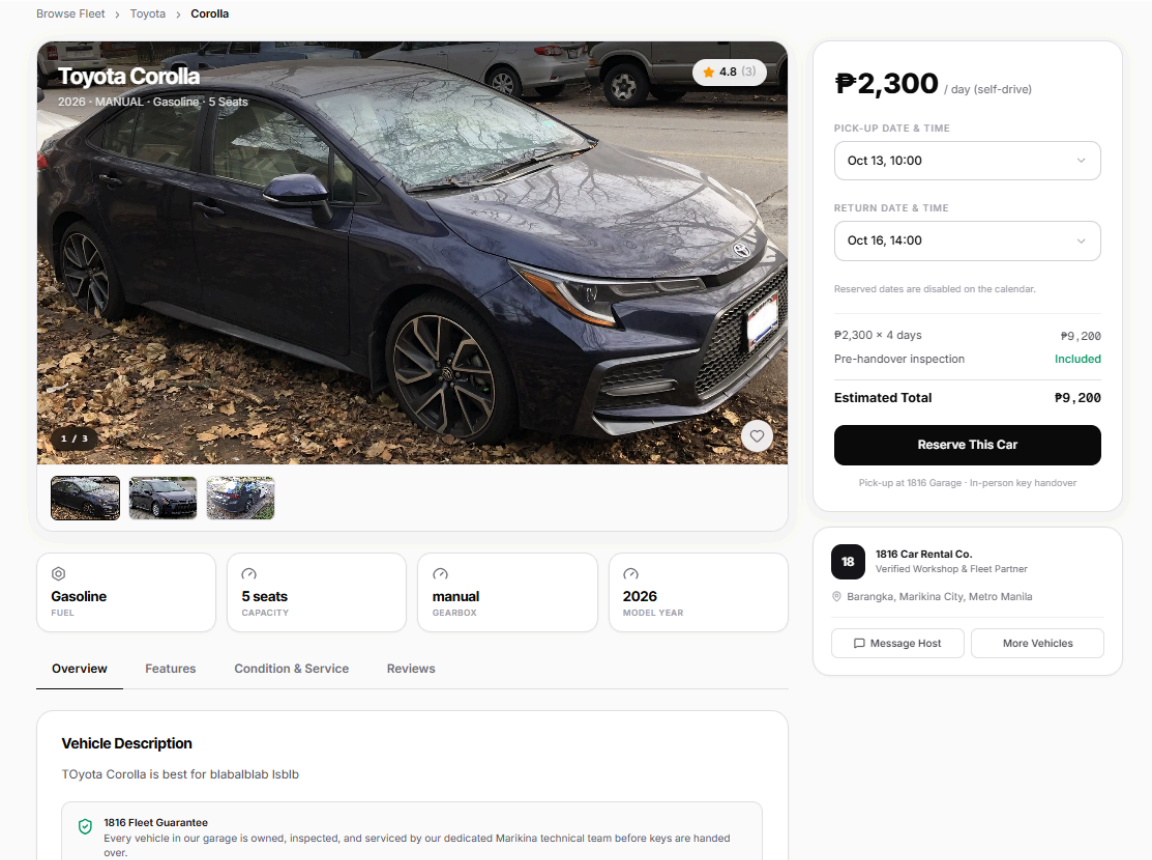


Figure 11 describes the details of a specific vehicle as well as the initial point for a booking by the customer. The image gallery in the information section displays the images of the vehicle and specifications available for the vehicle while the booking panel indicates the price which applies to one day of rental. The customer selects dates and times of pick up and drop off from an availability calendar which shows unavailable days. The page also calculates an estimated

rental amount based on the selected period before the booking process starts. During the booking process, the customer chooses Self-Driven or Chauffeured services and provides destination information; the system checks its approval and requests a valid time of at least one day. The check-out process includes entering the contact information, indicating the choice of pick up or delivery, and signing the terms. When the customer starts making payment, a special temporary reservation is made, the booked price is saved in memory as a screenshot, and the availability of the vehicle is verified.

Figure 12

Reservation Payment Confirmation

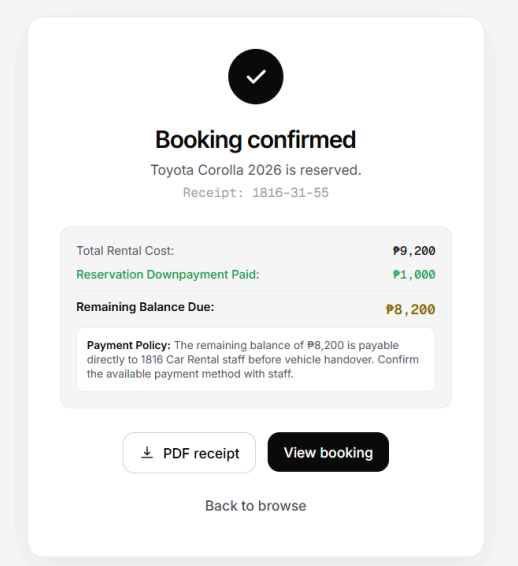
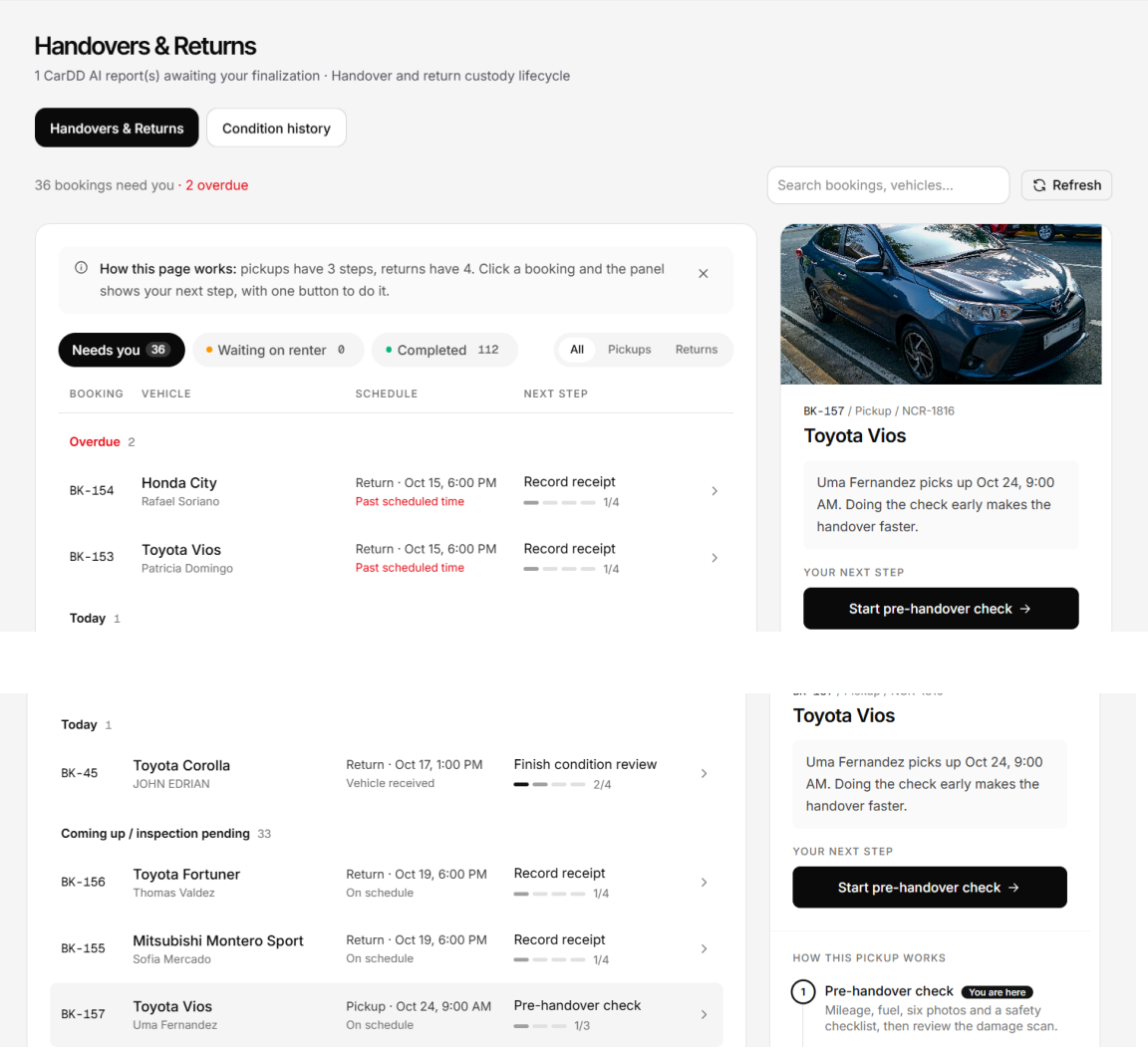


Figure 12 illustrates the confirmation of booking after successful validation of the deposit paid as a part of booking. It provides the information once again about the vehicle reserved together with number of receipt, total cost, details of the deposit paid and remaining amount due. The note related to the payment states that the balance should be cleared by the renter to the employee of 1816

Car Rental. The page also provides the possibility to download transaction copy in PDF format, view the information about the booking and see its status or simply continue doing the brows. Merely resulting from the payment provider's return is insufficient to create a confirmation page as all payment details must be confirmed by the system and booking status must be checked.

Figure 13

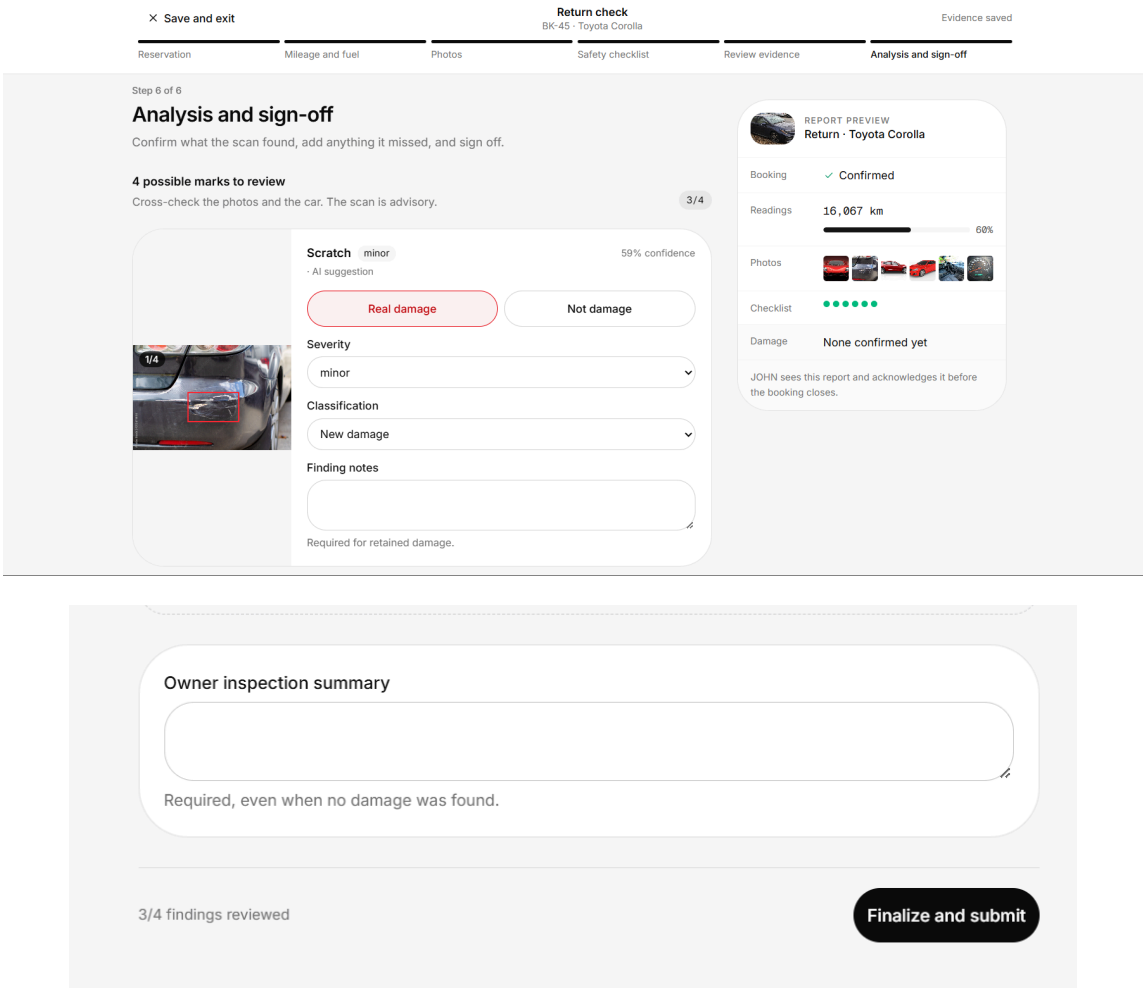
Vehicle Condition Checks and Documentation



The Handovers & Returns workspace is depicted in Figure 13, where vehicle conditions are recorded and vehicle pick-ups and returns are coordinated by the owner and employees. Bookings are categorized in three queues by their status, namely, the Needs you, Waiting on renter, and Completed queues. In addition, the All, Pickups, and Returns filters narrow down the bookings. The search feature is able to find bookings and classify them as overdue, current-day, or future ones. Each row in the booking feature contains information about the vehicle, the booking party name, the time of the booking, next required step, and the stage of the booking process. The detail panel provides pictures of the car, information about bookings, workflow tips, and buttons for the next step. The process of pick-ups includes three steps: preparing, informing of acceptance and releasing the vehicle, whereas returns consist of four steps: searching for the receipt, checking the vehicle, signing the receipt and closing the matter. Searching leads to the realization that the vehicle has been taken off from the balance. The collection of evidence triggers the collection of external photos for the assessment of the damage sustained by the CarDD. At this point, the owner examines and concludes the report that will undergo formal presentation to the tenant for confirmation. The Condition History section has all the inspections recorded up to date.

Figure 14

Owner Review of Condition Findings



The owner's report on vehicle condition assessment can be found in Figure 14. After following the required angles, it shows numerous photos evaluated by the owner of the checked car. When the automated CarDD is enabled, spotted problems will be outlined in the form of an area clearly marked with the name of the defect and its score indicating the credibility of the analysis. However, it is not to be regarded as the ultimate conclusions but a basis for

further investigation on behalf of the owner. The owner is to come up with the decision whether to acknowledge the defect suggested by the device or reject the diagnosis, and various aspects should be taken into account for records on the problems determined. The owner has a chance to add a manually noted problem that was missed by the analysis whereupon the level of the damage and type of defect will be reported. All defects retained in the report must be accompanied by notes and a final summary must outline the general condition of the car to complete the report. In case the automated analysis fails, the workflow makes it possible to conduct the assessment on a manual basis.

Figure 15

Operational and Financial Reports

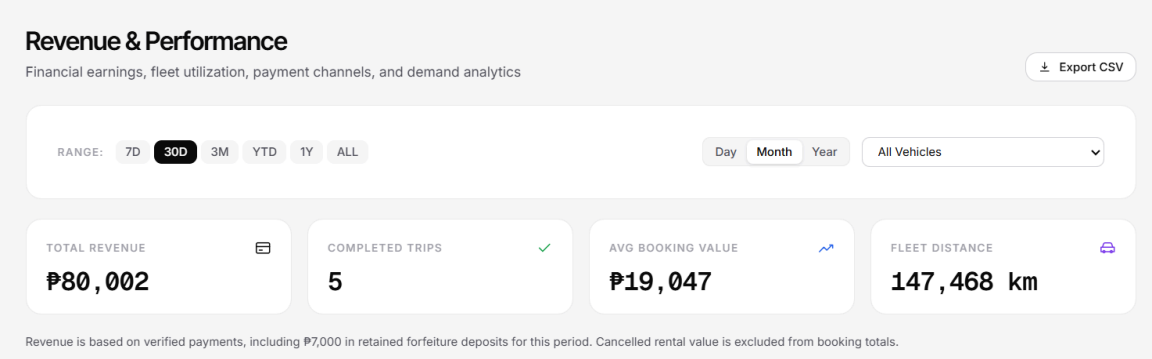
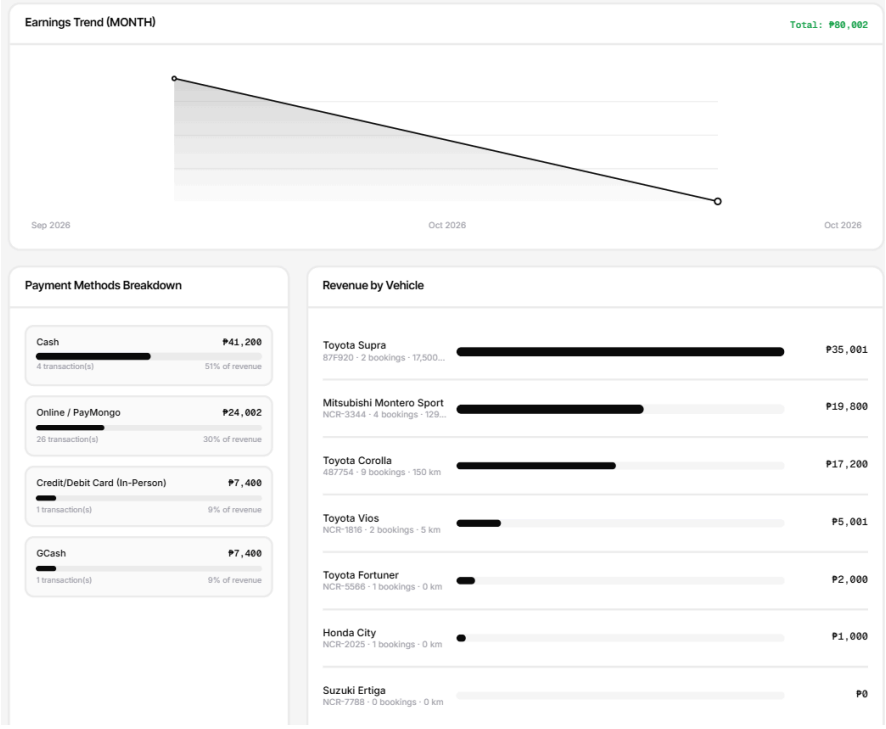


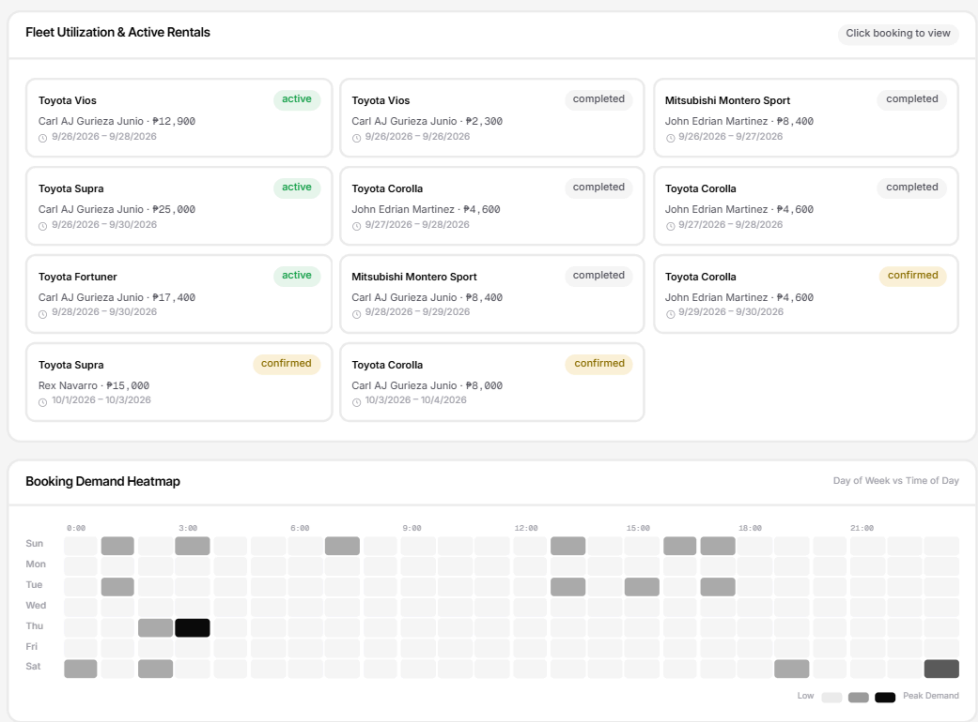
Figure 15 illustrates the report overview on the top of the Revenue & Performance page, which is solely accessible to the owner of the fleet. The owner is able to choose the range of the report, decide how the trend should be classified (being either daily, monthly, or yearly), as well as controll the process for either one vehicle or the fleet in general. The owner receives the information about total revenue, finished journeys, average value of the booking, and the distance covered by the fleet for the specific findings thanks to four summary

cards presented. The total price is counted based on the payment which is confirmed through payment documents only (meaning that any part of money that can be retained as forfeitive deposit and mentioned in notes is included in revenue counts). The revenue value is taken into account that has been paid for and not the booked service in order to avoid the false information about the rent. In case the owner wants to get some other figures, the owner has to press on the button again and load the information in a summary format.

Figure 16

Analytics for Decision-Making





The detailed analytical sections of the same Revenue & Performance page can be found in Figure 16. Verified payments are categorized based on the selected day, month, or year when displayed on the Earnings Trend graph, which shows how revenue changes during the desired period. Payment Method Breakdown portrays all the channels utilized and presents their transaction numbers, overall revenue revenue, and shares of revenue made through these channels. The Revenue by Vehicle section compares vehicles based on their revenue made during a certain period and provides additional details like booking information and distance. In the Fleet Utilization & Active Rentals section, the following reservations can be found, such as confirmed, ongoing, and finished reservations overlapping with the selected date range. In addition, the Booking Demand Heatmap is also included in this section with the relevant information displayed based on weekdays and hours on which each booking was made. The

same filters can be applied in this section, therefore enabling the owner to compare the periods or vehicles used.

Connection of Project Capabilities to the Research Questions

Research Question 1. Figures 4–16 depict the interconnected rental workflow created to resolve the different Messenger inquiries, spreadsheets, and operational records mentioned in Chapter 1. Owner view and renter view signify joint vehicle, booking, payment, and condition records.

Research Question 2. Figures 5–8 concerned with difficulties owner faces in keeping current information about vehicles, prices, availability, reservations, and locations. The combination of inventory, calendar, and booking monitor makes structure of the records; Live Tracking provides actual location if the tracker has a valid GPS fix.

Research Question 3. Figures 9–12 indicate disconnected stages from renter registration to final confirmation. They illustrate documents submission with human review, schedule selection with restrictions, verification of the payment, and confirmation processing after payment is completed.

Research Question 4. Figures 13–14 show the necessity of relevant condition records as well. In addition to pre-handover and return inspections, they contain images, readings, checklists together with notes and recommendations regarding damages, which were corroborated by the owner.

Research Question 5. Figure 15 shows the necessity to keep track of bookings, collections, car usage and income sources. It generates the filtered

reporting showing the bookings being recorded and payments being confirmed, whereas data is exported with the required view retained.

Research Question 6. The requirement for operational understanding is dealt with in Figure 16, which helps in forming patterns and benchmarks from past activity records. Although the descriptions help the owner in their evaluation, it does not show whether the decisions or operations were positively impacted as a result of the management decisions made.

Project Testing Results

Booking and Payment Workflow Checks

The researchers and scientists evaluated the rules for booking and payment separately from the process of evaluating damage and image. The targeted sequence of 14 backend evaluation passed successfully. It was determined that one pickup time had already passed while the data for Manila schedule is valid; one piece of invalid clock information had been rejected. The overlap lookup reveals that the same automobile had already been booked and was still unpaid. The application of the PostgreSQL exclusion also ensures that there are no overlaps between committed or held bookings for a given automobile. The results confirm the design of the booking validation system regardless of the fact that the particular exercise has not measured the incidence of simultaneous bookings with respect to the operation of the database yet.

The payment processing system allowed only valid PayMongo test-mode callbacks and denied all other types of signatures. In its payment confirmation

process, the amount paid, the currency used, and available seats are verified before updating the state of booking from “Pending” to “Confirmed,” the payment failure resulting in an error being logged. The life-cycle test was conducted by using a fictitious signed callback rather than a real PayMongo transaction. All these tests confirm the expected behavior. The use of the live payment gateway and failure timing remained unverified.

Vehicle Damage Model Tests

Confusion matrices compare CarDD annotated damage classes from test photographs to results generated from the CarDD’s YOLOv8s ONNX pre-trained model. The provided test split has 374 images with 785 annotated damage areas. Each single annotation provides the damage area type in addition to reference labels, thus allowing for an assessment against a benchmark. The annotation files were verified against the CarDD image_info.xlsx metadata records with 374 test images sharing the image ID, dimension, instance counts, and categories, the provided counts refer to the number of images.

In context of this approach, "Actual Positive" refers to presence of that particular class in the picture. "Actual Negative" indicates absence of that class. In total, the number of available labels in the test set include 157 pictures with dents, 183 with scratches, 48 with cracks, 71 with shattered glass, 65 with fractured lights and 31 with flat tires. When the application annotates an instance of a given class in an image with a confidence higher than 0.25, such image can be described as "Predicted Positive". Otherwise, it can be considered "Predicted Negative". The evaluation was based on processing of images in 1024 × 1024

letterbox configuration applying the class-wise NMS principle with an IoU threshold of 0.45. The predictions for images processed in this way were saved for each object class as well.

Comparing the actual and predicted results places each image in one of four groups. A True Positive (TP) means that the damage class is present and the model detects it. A False Positive (FP) means that the model reports the class even though it is absent from the reference labels. A False Negative (FN) means that the class is present but the model does not report it at the stated confidence threshold. A True Negative (TN) means that the class is absent and the model also reports no detection of it. A photo that shows scratches but not dent is Actual Negative for dent. If the photo has dent according to the model, it will add towards false positives in dent table.

Each table summarizes the four counts. Actual results are shown at the top, and predicted results are at the left side. The first number is TP, second number is FP, third number is FN, and finally, the last number is TN. The four numbers must add to 374 because each test image gives one result for the class examined. The sum of both TP and FN also matches the number of images that have that class in their data set. The two tests confirm the numbers in all the 6 tables.

The accuracy calculation is (TP + TN) / 374 x 100. In the case of dent, the valid attempt produced 135 true positives, 17 false positives, 22 false negatives, and 200 true negatives.Its 335 correct results give (335 / 374) × 100 = 89.57 percent. The accuracies correspond to 87.17, 93.32, 97.33, 94.12, and 97.33 for

scratch, crack, shattered glass, broken lamp, and flat tire respectively. All numbers are rounded up to two decimal places.

The results given relate to image presence results indicating whether the given damage classification has been reported in image(s) or not, but do not prove that all the damaged parts were found or that their boxes or masks were correctly located and shaped. Those details require a separate comparison of predicted areas with the annotated areas, using measures such as Intersection over Union (IoU). The saved object-detection evaluation and its multiclass confusion matrix should therefore be distinguished from the six image-level tables presented here.

Table 2

Dent Confusion Matrix

		Actual	
		Positive	Negative
Predicted	Positive	135	17
	Negative	22	200

In the dent confusion matrix shown in Table 2, positive means the image contains or the system detects dent. Negative means the image does not contain or the system does not detect dent. Accuracy is 89.57 percent.

Table 3

Scratch Confusion Matrix

		Actual	
		Positive	Negative
Predicted	Positive	168	33
	Negative	15	158

In the scratch confusion matrix shown in Table 3, positive means the image contains or the system detects scratch. Negative means the image does not contain or the system does not detect scratch. Accuracy is 87.17 percent.

Table 4

Crack Confusion Matrix

		Actual	
		Positive	Negative
Predicted	Positive	40	17
	Negative	8	309

In the crack confusion matrix shown in Table 4, positive means the image contains or the system detects crack. Negative means the image does not contain or the system does not detect crack. Accuracy is 93.32 percent.

Table 5

Shattered Glass Confusion Matrix

		Actual	
		Positive	Negative
Predicted	Positive	69	8
	Negative	2	295

In the shattered glass confusion matrix shown in Table 5, positive means the image contains or the system detects shattered glass. Negative means the image does not contain or the system does not detect shattered glass. Accuracy is 97.33 percent.

Table 6

Broken Lamp Confusion Matrix

		Actual	
		Positive	Negative
Predicted	Positive	55	12
	Negative	10	297

In the broken lamp confusion matrix shown in Table 6, positive means the image contains or the system detects a broken lamp. Negative means the image does not contain or the system does not detect a broken lamp. Accuracy is 94.12 percent.

Table 7

Flat Tire Confusion Matrix

		Actual	
		Positive	Negative
Predicted	Positive	29	8
	Negative	2	335

In the flat tire confusion matrix shown in Table 7, positive means the image contains or the system detects flat tires. Negative means the image does not contain or the system does not detect flat tires. Accuracy is 97.33 percent.

As for the results compiled for the damage uses, the scratch demonstrated the most unsatisfactory image-level results with the accuracy of only 87.17 percent. Out of 374 test images that were analyzed, there were 168 true positives, 33 false positives, 15 false negatives, and 158 true negatives regarding this damage type. The false positives refer to those images that were

considered to contain a scratch, but they did not have corresponding reference images. Thus, such cases may create extra work. The false negatives refer to those images that were marked as containing a scratch. Thus, in this case cases, the application reported that there were no scratches present in the photograph, the only relying on this model may lead to serious damages left unconsidered. As a result, the scratch presence had the precision of 83.58 percent, the recall of 91.80 percent, and the F1-score equal to 87.50 percent. In the end, the photographs are analyzed by the owner.

System Evaluation Result

The system's evaluation aimed at establishing its functions as designed as well as understanding the quality of experience described by the users, operators, or evaluators of the system. The survey consisted of 15 questions grouped into six parameters of assessment. In particular, parameters of the study included the following: functionality defined as having necessary functions to be performed; reliability defined as failure tolerance and recovery; usability defined as understanding, learning, operation, and appearance; efficiency defined as response time and resource consumption; portability defined as having browser access and getting rid of manual processes; and maintainability defined as detection of problems and verification of functions.

For purposes of the research, purposive sampling was used to receive information from 15 renters, one owner, and five evaluators or specialists in the field of information technologies. The evaluators were introduced into the system,

performed the required actions based on their roles, and filled out the same 15-question questionnaire based on their observations.

The four-point evaluation scale values the Strongly Agree (SA) option at 4, Agree (A) at 3, Disagree (D) at 2, and Strongly Disagree (SD) at 1. They signify the intensity of the respondent's agreement/disagreement with the questionnaire statement. In terms of the means calculated rounded to two decimal points, a mean score of 3.50–4.00 can be regarded as SA, 2.50–3.49 as A, 1.50–2.49 as D, and 1.00–1.49 as SD. The weighted mean for a group represents the responses of the evaluators for a particular subcriterion; Composite reflects the average data for the opinions collected from all groups which include renters, owners, and technical experts; General Weighted Mean reflects data obtained for subcriteria within the criterion.

Table 8

System Functionality Result

	FUNCTIONALITY				
Subcriterion	**Renters**	**Owner**	**Technical Evaluators**	**Composite**	**Interpretation**
Suitability	3.93	4.00	3.80	3.91	SA
Accurateness	3.93	4.00	3.80	3.91	SA
Security	3.87	4.00	3.60	3.82	SA
General Weighted Mean	3.91	4.00	3.73	3.88	SA

Table 8 reports the functionality scores. Suitability and accurateness each have a composite mean of 3.91, and security has a composite mean of 3.82. The General Weighted Mean for functionality is 3.88.

Table 9

System Reliability Result

RELIABILITY					
Subcriterion	**Renters**	**Owner**	**Technical Evaluators**	**Composite**	**Interpretation**
Fault Tolerance	3.67	3.00	3.20	3.29	A
Recoverability	3.93	4.00	3.60	3.84	SA
General Weighted Mean	3.80	3.50	3.40	3.57	SA

Table 9 reports composite means of 3.84 for recoverability and 3.29 for fault tolerance. The General Weighted Mean for reliability is 3.57.

Table 10

System Usability Result

USABILITY					
Subcriterion	**Renters**	**Owner**	**Technical Evaluators**	**Composite**	**Interpretation**
Understandability	3.33	3.00	4.00	3.44	A
Learnability	3.20	3.00	3.60	3.27	A
Operability	3.60	4.00	3.80	3.80	SA
Attractiveness	3.47	4.00	3.20	3.56	SA
General Weighted Mean	3.40	3.50	3.65	3.52	SA

From Table 10, we derive that the means for the composite aspect stood at 3.80 for operability, 3.56 for attractiveness, 3.44 for understandability, and 3.27 for learnability. The General Weighted mean for usability is 3.52.

Table 11

System Efficiency Result

EFFICIENCY					
Subcriterion	**Renters**	**Owner**	**Technical Evaluators**	**Composite**	**Interpretation**
Time Behaviour	3.87	4.00	3.40	3.76	SA
Resource Utilization	3.87	4.00	3.40	3.76	SA
General Weighted Mean	3.87	4.00	3.40	3.76	SA

Table 11 shows that the means of the composite aspect are 3.76 for time behavior and resource utilization. The General Weighted mean for efficiency is 3.76.

Table 12

System Portability Result

PORTABILITY					
Subcriterion	**Renters**	**Owner**	**Technical Evaluators**	**Composite**	**Interpretation**
Installability	3.93	4.00	3.80	3.91	SA
Replaceability	3.87	4.00	3.60	3.82	SA
General Weighted Mean	3.90	4.00	3.70	3.87	SA

From Table 12, we derive that means of 3.91 for installability and 3.82 for replaceability were obtained. The General Weighted mean for portability is 3.87.

Table 13

System Maintainability Result

MAINTAINABILITY					
Subcriterion	**Renters**	**Owner**	**Technical Evaluators**	**Composite**	**Interpretation**
Analysability	3.67	4.00	3.40	3.69	SA
Testability	3.80	4.00	3.40	3.73	SA
General Weighted Mean	3.73	4.00	3.40	3.71	SA

Table 13 presents the means of 3.73 for the testability aspect and 3.69 for the analysability aspect, thus the General Weighted mean for maintainability is 3.71

Table 14

Summary of System Evaluation Results

SYSTEM EVALUATION RESULTS		
Criterion	**General Weighted Mean**	**Interpretation**
Functionality	3.88	SA
Reliability	3.57	SA
Usability	3.52	SA
Efficiency	3.76	SA
Portability	3.87	SA
Maintainability	3.71	SA
Grand Overall Weighted Mean	3.72	SA

In Table 14, the Grand Overall Weighted mean stood at 3.72 which shows that Strongly Agree on a four-point scale. The table clarifies that the evaluators, selected by purposive sampling method, have given agreement towards the

statements made in the questionnaire, which doesn’t mean that the organization attained a 93 percent success in applying the system. 3.29 for fault tolerance (Agree) highlights interrupted or unsuccessful operations as the zone of perturbation. Understanding got a score of 3.44 and learnability got 3.27-both of which are agreeable. They flagged the need for clearly explained instructions, labels, or first-time-user guidance. Such relative scores identified priorities for improvement; the questionnaire alone doesn't diagnose a certain fault in the software.

Table 15

General Weighted Means by Criterion

Criterion	General Weighted Mean
Functionality	3.88
Reliability	3.57
Usability	3.52
Efficiency	3.76
Portability	3.87
Maintainability	3.71
Grand Overall Weighted Mean	3.72

As shown in Table 15, the calculations in Tables 8 – 13 were performed by obtaining the average ratings within each type of evaluator category and then averaging the values obtained for renters, owners, and technical evaluators in order to come up with the value for each sub-criterion Composite. In order to get the value of General Weighted Average Mean for a particular criterion, the sub-criterion Composite scores are averaged. In this case, the values obtained for the six criteria were averaged equally in obtaining Grand Overall Weighted

Mean value in the Table 14. The calculation of one of the group means of functionality, for example, will be 3.91, 4.00, and 3.73 together lead to 3.88. In the calculations of the average means, the rounded values were not made use of; that is the reason why two decimal digits are displayed in this table. Melo et al. (2024) also conducted the analysis of software package based on the quality characteristics by different evaluator groups. In this case, the use of equal weightings for all three groups and for six criteria was employed as a calculation technique in the present research.

Discussion of Findings by Research Question

Research question 1 concerns centralized rental management. The interconnected documents and two workspaces of the owner and renter shown in Figures 4-16 elaborate the type of system created, which is a web-based application that connects the information about vehicles with the needs of the renter, booking time, payment status, evidence of the condition, and history of operation. The average rating of the functionality of 3.88 indicates positive responses concerning the required-function statements. But neither the screenshots nor the rating tells whether any time was saved by the staff of the company.

Research question 2 deals with owner management and GPS tracking. In Figures 5-8, it is possible to find details about fleet inventory, pricing, and complexity of the calendar of availability, booking monitor, and GPS map. Reservation checks and the overlapping principle of the database help complete the scheduling control; verified records of payment distinguish between paid and

unpaid reservations. GPS software can show the data about the coordinates, speed, and the paths when the data is received automatically. However, researchers conducted the physical tests of GPS only on the PLP campus and in Pinagbuhatan, where they used a motorcycle. It means that no actual vehicle was rented.

Research Question 3 Transactions of renters. Figures 9-12 illustrate the various processes involved in registration, submitted ID verification, selection of schedule, payment, and confirmation. The owner will still be required to verify the identity and driving documents. The booking checks verified whether the pick-up times did not also have other conflicting holds, while the payment checks in test mode rejected invalid callbacks. A standalone lifecycle check was done with a fake signed payment callback. All of these results confirm the various steps of processes that were being trialed, but they were not able to show a live charge done through PayMongo or prove the absence of problems arising during actual transactions.

Research Question 4 Comparison of car condition. Figures 13-14 depict the relation of pre-hand-over and post-return photographs, the odometer readings and fuel levels, the checklist information, and other observations made to the booking. The CarDD investigation suggests some conclusions that the owner of the vehicle can evaluate, while the tenant receives the final report for agreement. The outcome of the scratch analysis indicates the occurrence of both false positive and missed cases when the damage of the vehicle was not labeled, thus calling for manual examination of the results.

Research Question 5 Operational and Financial Reporting. Figure 15 depicts the total earnings and exports for bookings, verified collection, completed trips, and vehicle activities based on the date and vehicle filter. Revenues are based on the verified payment records and not the value of uncollected or canceled reservations. The figure shows the reporting done on behalf of the owner; the study did not provide an independent verification of each figure shown in this report against the financial records.

Research Question 6 Insights for Decision Making. Figure 16 shows the historical data that comprises earnings trends, market shares of payment channels, vehicle comparisons, usage rates and booking patterns. This is basic descriptive analysis that helps compare data across vehicles and over time. The study neither assessed whether the owner made a different decision nor whether this resulted in increased revenues.

CHAPTER 4

CONCLUSION AND RECOMMENDATIONS

This chapter has summed up the conclusions along with the recommendations that are based on the study findings. It has detailed the ways through which the studied system has answered the issues presented in the research and suggested areas that can be practically improved and researched in the future.

Conclusion

This research sought to create a web-based management system for 1816 Car Rental Co., which included GPS tracking and IoT features. Twenty-one people, consisting of 15 renters, one owner, and five evaluators, had tested the system via a survey through Google Forms.

According to the results of the research, the following conclusions can be drawn from the findings:

The researchers were able to develop a web-based rental management system that provides renters with information about the rental process and its related stages, including vehicle information, rental record, bookings, payments, and updates regarding the rental process. The researchers managed to link all of the aforementioned factors to the process of renting a car.

The owner of the company was given the opportunity of viewing the vehicle information, bookings, etc. Thanks to the dashboard, the owner could see the location and speed when the device was connected to the tracking system.

However, the results of the actual tracking were not tested, and no effective analysis was made to check the efficiency of the system used by the company.

The system integrated the processes of registering renters and submitting and assessing identification documents with the processes of booking, payment, and confirmation of transactions. Although document validation was in the hands of authorized individuals, the operations of transaction processing were supported by the measures of access control and payment verification. The study could not identify whether these measures ensured that fraud was excluded and security of transactions was guaranteed.

The system connected records of the handover and return photographs related to the same booking along with the inspection notes and damage conclusions verified by the owner. These records served as evidence for comparing the vehicles' condition and delivering the results to the renter. The study was not able to determine whether the decision-making in terms of inspection was fair and consistent.

The system allowed generating reports about the operational performance and finances of the given activity according to the bookings, payment verification, vehicles’ use, and profits. The summaries involved the organization of bookings according to the period and the vehicle in question; besides, the recordings provided the information regarding revenues based on verified collection of money and thus making it possible to analyze transactions.

The system produced descriptive analytics by using historical revenue trends, payment-channel summaries, vehicle comparisons, and booking patterns.

This information enabled processing the recorded actions in different times and with different type of vehicles and gave the important information for making the right judgment. However, the research did not show how much exactly the features contributed to the improvement of managerial decisions and business profitability.

The system has overall Grand Weighted Mean score of 3.72, which can be interpreted as Strongly Agree for each of the six indicators of software quality at the same time. This outcome reflects the feedbacks of all the renters, the owner, and the technician respondents after completing their respective roles. Although the final evaluation can be considered positive within the framework of the research, it did not prove the system's efficiency in real rental business.

Recommendations

In light of the findings, the authors suggest the following prospects of improvements and further studies.

Suggestions to be Followed:

Further inclusion of a concise introduction and graphical references for the site owners and tenants. Such references should highlight issues of the submission of documents, bookings, confirmations of payment, perusal of the condition report, and owners' actions. User comments should inform suggested improvements of the references.

Recommendations for Improvement of System/Process:

In future designs, the user interface should improve naming, interface calls, and messages concerning the statuses. So that interface testing is carried out with first-timers.

In future practices, new solutions should improve the work with undefined or failed requests. Users should be properly warned through better notifications about retrying and ensuring consistent bookings/payments. Testing events should include loss of communication, delayed payments, and GPS output showing eventual resuming of the operations without repetitions.

In future developments, it will be necessary to broaden the payment methods available through PayMongo, including credit cards. The 1816 Car Rental Company will need to open or upgrade its PayMongo business account to the verified status, perform the other tasks, and ask for activation of the other payment methods.

Future developments may utilize machine learning technologies to conduct the automated identification activities, especially in cases of changing the documents making them unable to be recognized as authentic. In case of ambiguous results, the referrals should be performed by authorized persons. In addition, cooperation with the related governmental structures may be acquired for the purposes of the legitimization of the documents and more accurate renting verifications.

The accident reporting module may enhance the vehicle condition file by adding the data about the most recent car accidents taking place during the rental. It may allow renters to provide photos, the documents describing the incidents, and also the time and location of the events while giving the opportunity to the owner of recording all materials required.

Recommendations for Future Research

Subsequent recommendations called for further evaluation of the vehicle condition procedure based on the implementation of tasks that looked at the handover and return testing processes. It was proposed to utilize real pictures captured in various lighting situations and angles for this process. The researchers recommended making comparisons of the proposed damage findings with the analyzed source records to assess the missed damage and wrong convictions. The researchers also proposed performing repeated evaluation of the condition records to see how those were able to assist in the comparison between two phases of the inspection. The suggestion was made after concluding that the platform provided a verified procedure of evaluation, meanwhile the general survey did not provide the evaluation of fairness or validity of vehicle condition decisions.

Explicit examination of findings and analysis was suggested to assess the management information presented in the fifth and sixth conclusions. Upon checking the source bookings and payment data with known sums, future evaluation is assumed to be made. Moreover, it was suggested to give the owner

specific tasks concerning the comparing of different time periods collecting the information.

It was recommended that future study should include more rental companies, renters, and technicians over a longer period of actual use. The current study has only one company and 21 respondents, including one among the owners. A more complete research is advisable to reveal whether the same results would have been obtained by other research in other rental places and during using vehicles for longer periods. Long-venue is also suggested to monitor vehicles under actual rental conditions, including weak-connectivity periods.

REFERENCES

Alhwety, R. N. H., & Elfadil, N. (2024). Vehicle tracking system approaches: A systematic literature review. International Journal of Computer Science and Mobile Computing, 13(8), 23-31. https://doi.org/10.47760/ijcsmc.2024.v13i08.003

Ayad, M., Alkanhel, R., Saoudi, K., Benziane, M., Medjedoub, S., & Ghoneim, S. S. M. (2022). Evaluation of radio communication links of 4G systems. Sensors, 22(10), 3923. https://doi.org/10.3390/s22103923

Barroga, E., Matanguihan, G. J., Furuta, A., Arima, M., Tsuchiya, S., Kawahara, C., Takamiya, Y., & Izumi, M. (2023). Conducting and writing quantitative and qualitative research. Journal of Korean Medical Science, 38(37), e291. https://doi.org/10.3346/jkms.2023.38.e291

Basak, S. K., Neil, L., Reaves, B., & Williams, L. (2022). What are the practices for secret management in software artifacts? In 2022 IEEE Secure Development Conference (SecDev) (pp. 69-76). IEEE. https://doi.org/10.1109/SecDev53368.2022.00026

Bucko, A., Vishi, K., Krasniqi, B., & Rexha, B. (2023). Enhancing JWT authentication and authorization in web applications based on user behavior history. Computers, 12(4), 78. https://doi.org/10.3390/computers12040078

Cichosz, M., Wallenburg, C. M., & Knemeyer, A. M. (2020). Digital transformation at logistics service providers: Barriers, success factors and leading practices. The International Journal of Logistics Management, 31(2), 209-238. https://doi.org/10.1108/IJLM-08-2019-0229

Gentile, A. F., Macri, D., Carni, D. L., Greco, E., & Lamonaca, F. (2024). A network performance analysis of MQTT security protocols with

constrained hardware in the dark net for DMS. Applied Sciences, 14(18), 8501. https://doi.org/10.3390/app14188501

Golo, M. A. T., & Encarnacion, R. E. (2024). Revolutionizing the rental services in Siargao Island: Basis for developing an online vehicle rental management system. International Journal of Advanced Research in Science, Communication and Technology, 4(1), 438-439. https://doi.org/10.48175/IJARSCT-18754

Henderson, A., Cao, M., & Liu, Q. (2022). Access-based consumption, behaviour change and future mobility: Insights from visions of car sharing in Greater London. Future Transportation, 2(1), 216-236. https://doi.org/10.3390/futuretransp2010011

Ishak, S. A., Din, R., & Hasran, U. A. (2021). Defining digital game-based learning for science, technology, engineering, and mathematics: A new perspective on design and developmental research. Journal of Medical Internet Research, 23(2), e20537. https://doi.org/10.2196/20537

Islam Akanda, N., Hossain, M. A., & Fahad, M. M. I. (2022). Cost-effective and user-friendly vehicle tracking system using GPS and GSM technology based on IoT. Indonesian Journal of Electrical Engineering and Computer Science, 28(3), 1826-1833. https://doi.org/10.11591/ijeecs.v28.i3.pp1826-1833

Jangam, S. K. (2023). Importance of encrypting data in transit and at rest using TLS and other security protocols and API security best practices. International Journal of AI, BigData, Computational and Management Studies, 4(3), 82-91. https://doi.org/10.63282/3050-9416.IJAIBDCMS-V4I3P109

Jara Ochoa, H. J., Pena, R., Ledo Mezquita, Y., Gonzalez, E., & Camacho-Leon, S. (2023). Comparative analysis of power consumption between MQTT and HTTP protocols in an IoT platform designed and implemented for

remote real-time monitoring of long-term cold chain transport operations. Sensors, 23(10), 4896. https://doi.org/10.3390/s23104896

Kambala, G. (2023). Designing resilient enterprise applications in the cloud: Strategies and best practices. World Journal of Advanced Research and Reviews, 17(3), 1078-1094. https://doi.org/10.30574/wjarr.2023.17.3.0303

Kashmim, A. K. S., Rani, M. F. C. A., & Jegatheswaran, R. (2023). Vehicles agency rental online system. Journal of Applied Technology and Innovation, 7(4), 1-8. https://doi.org/10.65136/jati.v7i1.144

Khan, S. U., Alam, N., Jan, S. U., & Koo, I. S. (2022). IoT-enabled vehicle speed monitoring system. Electronics, 11(4), 614. https://doi.org/10.3390/electronics11040614

Lombardi, M., Pascale, F., & Santaniello, D. (2021). Internet of Things: A general overview between architectures, protocols and applications. Information, 12(2), 87. https://doi.org/10.3390/info12020087

Marth, S., Hartl, B., & Penz, E. (2022). Sharing on platforms: Reducing perceived risk for peer-to-peer platform consumers through trust-building and regulation. Journal of Consumer Behaviour, 21(6), 1255-1267. https://doi.org/10.1002/cb.2075

Martirez, K. C. (2023). Compliance on Data Privacy Act of private higher educational institutions (HEIs) in the province of Albay: Basis for information governance model. International Journal of Education, Business and Economics Research, 3(3), 26-36. https://ijeber.com/uploads2023/ijeber_03_100.pdf

Melo, W. S. de, Braga, H. F. G. M., Cardoso, M. V. L. M. L., Melo, E. S. J., & Monteiro, F. P. M. (2024). Software evaluation on infant development to support teaching and professional training. Revista Latino-Americana de Enfermagem, 32, e4284. https://doi.org/10.1590/1518-8345.7248.4284

Nazer, N., Prakash, P., Ridhunanda, N., Sasi, S., & Girija, V. R. (2025). Vehicle rental website. International Journal of Science and Research Archive, 15(3), 1272-1279. https://doi.org/10.30574/ijsra.2025.15.3.1813

Palmese, F., Redondi, A. E. C., & Cesana, M. (2022). Adaptive quality of service control for MQTT-SN. Sensors, 22(22), 8852. https://doi.org/10.3390/s22228852

Paris, I. L. B. M., Habaebi, M. H., & Zyoud, A. M. (2023). Implementation of SSL/TLS security with MQTT protocol in IoT environment. Wireless Personal Communications, 132(1), 163-182. https://doi.org/10.1007/s11277-023-10605-y

Piastou, M. (2023). Comprehensive performance and scalability assessment of front-end frameworks: React, Angular, and Vue.js. World Journal of Advanced Engineering Technology and Sciences, 9(2), 366-376. https://doi.org/10.30574/wjaets.2023.9.2.0153

Pratama, I., & Raharja, I. (2023). Node.js performance benchmarking and analysis at VirtualBox, Docker, and Podman environment using Node-Bench method. JOIV: International Journal on Informatics Visualization, 7(4), 2240-2246. https://doi.org/10.62527/joiv.7.4.1762

Rashid, T., & Mustafa, S. (2023). A review on IoT: Layered architecture, security issues and protocols. IJCSNS International Journal of Computer Science and Network Security, 23(9), 100-110. https://doi.org/10.22937/IJCSNS.2023.23.9.13

Reiher, L., Lampe, B., Woopen, T., van Kempen, R., Beemelmanns, T., & Eckstein, L. (2022). Enabling connectivity for automated mobility: A novel MQTT-based interface evaluated in a 5G case study on edge-cloud LiDAR object detection. In Proceedings of the 2022 International Conference on Electrical, Computer, Communications and Mechatronics Engineering (ICECCME) (pp. 1-6). IEEE. https://doi.org/10.1109/ICECCME55909.2022.9987813

Republic of the Philippines. (2012). Republic Act No. 10173: Data Privacy Act of 2012. The Lawphil Project. https://lawphil.net/statutes/repacts/ra2012/ra_10173_2012.html

Saad Salih, S., & Alsaedi, M. A. (2023). Developed smart vehicle tracking system using GPS and GSM modem. Al-Iraqia Journal for Scientific Engineering Research, 2(1), 9-12. https://doi.org/10.58564/IJSER.2.1.2023.56

Salunke, S. V., & Ouda, A. (2024). A performance benchmark for the PostgreSQL and MySQL databases. Future Internet, 16(10), 382. https://doi.org/10.3390/fi1610382

Samizadeh Nikoui, T., Rahmani, A. M., Balador, A., & Haj Seyyed Javadi, H. (2021). Internet of Things architecture challenges: A systematic review. International Journal of Communication Systems, 34(4), e4678. https://doi.org/10.1002/dac.4678

Seoane, V., Garcia-Rubio, C., Almenares, F., & Campo, C. (2021). Performance evaluation of CoAP and MQTT with security support for IoT environments. Computer Networks, 197, 108338. https://doi.org/10.1016/j.comnet.2021.108338

Shahri, E., Pedreiras, P., & Almeida, L. (2022). Extending MQTT with real-time communication services based on SDN. Sensors, 22(9), 3162. https://doi.org/10.3390/s22093162

Skanda, C., Srivatsa, B., & Premananda, B. S. (2022). Secure hashing using BCrypt for cryptographic applications. In 2022 IEEE North Karnataka Subsection Flagship International Conference (NKCon) (pp. 1-6). IEEE. https://doi.org/10.1109/NKCon56289.2022.10126956

Wong Wei Min, J., & Jali, N. (2021). Vehicle tracking system. International Journal of Scientific & Technology Research, 10(9), 10-17. https://www.ijstr.org/paper-references.php?ref=IJSTR-0921-50854

APPENDIX A : Operational Definition of Terms

OPERATIONAL DEFINITION OF TERMS

The following operational and technical terms are defined according to their use in the developed Web-Based Car Rental Management System with Real-Time IoT-Based GPS Tracking for 1816 Car Rental Co.

Application Programming Interface (API). Refers to the system endpoints through which the website exchanges information with the Fastify server. These endpoints process registration, bookings, payments, condition records, and tracking requests. Protected endpoints check the account's identity, role, and permitted records before allowing access.

Audit History. Refers to the stored records of user and system actions associated with rental operations. Entries identify recorded actions, actors, times, and relevant references, allowing the owner to trace booking changes, payment activities, document-review decisions, and administrative actions.

Booking. Refers to a renter's reservation of a specific vehicle for selected pickup and return dates and times. The system records the rental type, destination, price, and transaction status, while availability checks prevent conflicting reservations for the same vehicle.

CarDD. Refers to the car-damage image dataset associated with the pretrained YOLOv8s model used in the study. Its annotated test images provide reference damage labels for model evaluation, covering dents, scratches, cracks, shattered glass, broken lamps, and flat tires.

Condition Report. Refers to the booking-linked record of vehicle photographs, odometer and fuel readings, checklist answers, observations, and

owner-reviewed findings before handover or after return. The owner finalizes the report for renter acknowledgment. The report documents condition without automatically determining damage liability.

Data Analytics. Refers to the owner's use of summaries, trends, and comparisons generated from stored rental records. The system presents booking demand, payment methods, vehicle activity, and revenue for selected periods to support operational decisions; these displays describe recorded activity rather than predict future outcomes.

Fleet Utilization. Refers to the extent to which the company's vehicles are used for rentals. The overview relates active rentals to the recorded fleet size, while the analytics view presents vehicle booking activity within a selected period to support comparisons of vehicle use.

Geofencing. Refers to the configured geographic boundaries against which the system checks received GPS positions. It supports identifying movement outside the operating area and presenting boundary alerts to authorized management users. A recorded boundary alert does not automatically establish renter misconduct.

Global Positioning System (GPS). Refers to the satellite-based positioning information obtained by the study's tracking prototype. Recorded coordinates and timestamps support location display and route history in the application. The reported hardware tests were researcher-operated prototype activities, rather than monitoring during actual company rentals.

Handover. Refers to the owner or staff's release of the reserved vehicle to the renter. In the system, this occurs after the required payment and pre-handover condition checks, including renter acknowledgment of the finalized report, and changes the booking to an active rental.

Identity Verification. Refers to the authorized review of a renter's submitted identification document, personal details, and captured live selfie. The review supports account and rental eligibility decisions. OCR suggestions and the guided selfie activity assist the process but do not independently authenticate the person's identity.

Instance Segmentation. Refers to the damage model's identification of separate damaged regions within a vehicle photograph using region masks. These masks represent the shapes of individual findings, alongside damage classes and confidence scores, to assist the owner's review of the submitted condition evidence.

Internet of Things (IoT). Refers to the connection between the physical tracking prototype and the web-based rental system. The prototype collects position and related readings, transmits them through the available network, and supplies information that the application stores and displays for authorized monitoring.

Machine Learning (ML). Refers to the use of a pretrained image-analysis model to suggest visible vehicle damage from uploaded photographs. Its findings assist the owner's condition assessment. The model does not decide renter liability, calculate repair costs, or automatically impose damage charges.

Message Queuing Telemetry Transport (MQTT). Refers to the publish-subscribe communication protocol used to transmit tracking readings from the IoT prototype through a broker to the server. The application receives and stores these messages for location displays, recorded routes, and boundary evaluation when communication is available.

Open Neural Network Exchange (ONNX). Refers to the file format of the pretrained vehicle-damage model executed by the system. ONNX Runtime runs this model on the server to produce image-analysis findings for review; this execution performs inference rather than training a new model.

Optical Character Recognition (OCR). Refers to the Tesseract-based reading of text from uploaded identification images. The system uses the extracted text to suggest document type and personal or license details during registration. Recognizing printed text does not establish that an identification document is genuine or linked to official LTO records.

Payment Confirmation. Refers to the system's verification that a booking payment was received through PayMongo. The server checks a signed payment notification or verified checkout-session status against the booking before recording the payment, confirming the reservation, and making the corresponding receipt available.

Role-Based Access. Refers to the assignment and enforcement of permissions for renter, staff, and owner accounts. Renters access their own transactions, staff handle authorized operational tasks, and the owner controls

restricted business functions. The server checks these permissions before providing protected data or accepting actions.

Telemetry. Refers to the device-recorded tracking information received and stored by the system, including location, speed, recording time, and delayed-transmission status. These records support position displays, route history, and boundary checks. Buffered readings received after reconnection represent historical activity rather than a current location fix.

APPENDIX B : Evaluation Sheets

Pamantasan ng Lungsod ng Pasig
Alkalde Jose St. Kapasigan, Pasig City
College of Computer Studies

Evaluator Information

Name of Evaluator	_____
Title of the Study/System	Web-Based Car Rental Management System with Real-Time IoT-Based GPS Tracking for 1816 Car Rental Co.
Evaluator type	□ Renter □ Owner □ Technical evaluator or IT professional
Date evaluated	_____
Device and browser	_____
Assigned test account	□ Renter □ Owner □ Staff □ Other _____

Instructions

1. Complete the test scenarios assigned to your evaluator type before answering.
2. Rate every statement based only on the system behavior you observed during the evaluation.
3. Place one mark in the rating column that best represents your assessment.
4. Use the comments section to identify the task, screen, or condition related to any problem.

Rating Scale

4	3	2	1
Strongly Agree	Agree	Disagree	Strongly Disagree

System Quality Ratings

No.	Indicator	Statement	4	3	2	1
Functionality						
1	Suitability	The system provides the functions needed to complete the tasks assigned to my user role.				
2	Accurateness	The system displays correct information and produces the expected result after I complete an action.				

No.	Indicator	Statement	4	3	2	1
3	Security	The system prevents users from accessing information and functions that are not permitted for their assigned role.				
Reliability						
4	Fault tolerance	When an invalid input or error occurs, the system remains usable and provides a clear message explaining what to do.				
5	Recoverability	After a page refresh or temporary interruption, saved records remain available and the user can continue working.				
Usability						
6	Understandability	The system's labels, instructions, buttons, and status messages are easy to understand.				
7	Learnability	A first-time user can learn how to perform the main tasks without extensive training.				
8	Operability	The main tasks can be completed without unnecessary steps or excessive effort.				
9	Attractiveness	The system's interface is organized, readable, and visually appropriate.				
Efficiency						
10	Time behaviour	The system responds to user actions within an acceptable amount of time during normal use.				
11	Resource utilization	The system operates without excessive slowing, freezing, or unresponsiveness on the device used for testing.				
Portability						
12	Installability	The system can be accessed through a supported web browser without complicated installation or setup.				
13	Replaceability	The system can reduce or replace the manual processes previously used for rental operations.				
Maintainability						
14	Analysability	When a task or function fails, the displayed message or system status makes it possible to identify what went wrong.				
15	Testability	Each main system function can be tested, and its successful or unsuccessful result can be clearly verified.				
Evaluator Comments						

Which system function worked best during your evaluation

_____

_____

Which task or screen caused the most difficulty

_____

_____

What specific improvement do you recommend

_____

_____

APPENDIX C : Project Timeline

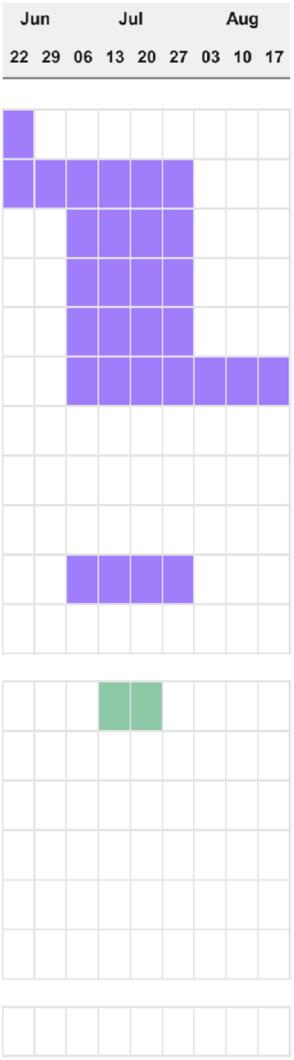
1816 Car Rental Co.

Project Gantt Chart

Project members
CA Junio, Carl AJ G.
JE Martinez, John Edrian V.
RN Navarro, Rex S.

25 June - 30 October 2026

Task / milestone	Members
System development	
1. Application and repository foundation	CA
2. Renter, owner and staff interfaces; fleet management	CA, JE, RN
3. Bookings, availability and schedule validation	CA, JE, RN
4. Payment checkout, balances and settlement	CA, RN
5. Messaging, notifications and transaction emails	CA, JE, RN
6. Registration, identity checks and access controls	CA, JE, RN
7. Vehicle inspections, damage review and handovers	CA, RN
8. GPS tracking, route history and service boundaries	CA, JE, RN
9. Refunds, late returns and operational rules	CA, RN
10. Revenue analytics, exports and audit history	JE, RN
11. Deployment, performance and demonstration data	CA, JE, RN
Paper and revisions	
12. Chapters 1-2: initial draft and revisions	CA, JE, RN
13. Chapters 1-2: scope, methodology, ethics and privacy	CA, JE, RN
14. Chapter 3: project capabilities (Figures 4-16)	JE, RN
15. Chapter 3: damage-model results and confusion matrices	CA
16. Chapter 3: evaluation tables, composites and formulas	RN
17. Chapters 3-4: panel revisions, SOP alignment and references	RN
Final supplements	
18. Work assignments, definition of terms and Gantt chart	RN



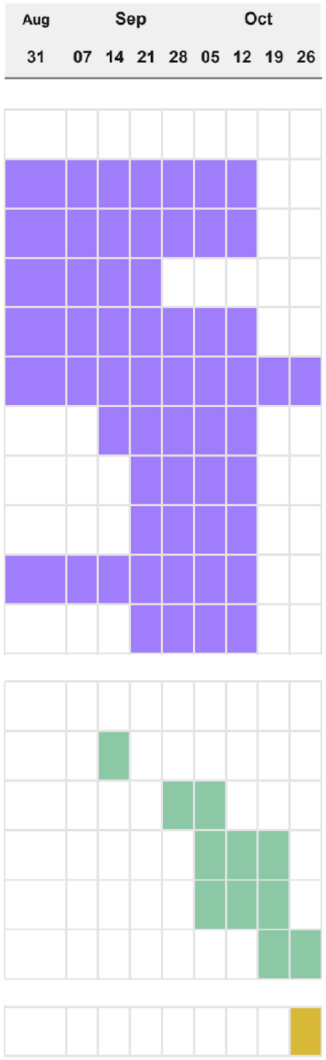
1816 Car Rental Co.

Project Gantt Chart

Project members
CA Junio, Carl AJ G.
JE Martinez, John Edrian V.
RN Navarro, Rex S.

25 June - 30 October 2026

Task / milestone	Members
System development	
1. Application and repository foundation	CA
2. Renter, owner and staff interfaces; fleet management	CA, JE, RN
3. Bookings, availability and schedule validation	CA, JE, RN
4. Payment checkout, balances and settlement	CA, RN
5. Messaging, notifications and transaction emails	CA, JE, RN
6. Registration, identity checks and access controls	CA, JE, RN
7. Vehicle inspections, damage review and handovers	CA, RN
8. GPS tracking, route history and service boundaries	CA, JE, RN
9. Refunds, late returns and operational rules	CA, RN
10. Revenue analytics, exports and audit history	JE, RN
11. Deployment, performance and demonstration data	CA, JE, RN
Paper and revisions	
12. Chapters 1-2: initial draft and revisions	CA, JE, RN
13. Chapters 1-2: scope, methodology, ethics and privacy	CA, JE, RN
14. Chapter 3: project capabilities (Figures 4-16)	JE, RN
15. Chapter 3: damage-model results and confusion matrices	CA
16. Chapter 3: evaluation tables, composites and formulas	RN
17. Chapters 3-4: panel revisions, SOP alignment and references	RN
Final supplements	
18. Work assignments, definition of terms and Gantt chart	RN



APPENDIX D : Work Assignments

WORK ASSIGNMENTS

Junio, Carl Aj. : System Integration and Documentation

Specific Tasks

Documentation Development

Chapter No.	Chapter Title	Module Title	Functionality/ies
1	Introduction: Jointly prepared the Statement of the Problem and Scope and Limitation of the Study.	Accounts and Verification	Implemented account registration, email OTP verification, JWT-based role access, and live-selfie capture with protected identity-document submission.
1	Introduction: Reviewed and synthesized technical literature on IoT architecture, MQTT, system development, database security, and 4G reliability (pp. 8-17).	Reservations and Payments	Integrated booking creation, rate snapshots, availability and overlap validation, PayMongo checkout, webhook payment confirmation, and receipt generation.
2	Methodology: Prepared the chapter introduction, Research Design, and Architectural Framework.	Vehicle Condition Analysis	Integrated CarDD-based damage inference and analysis queues, inspection evidence capture, owner review of findings, and finalized condition reports.
2	Methodology: Prepared the Algorithm Discussion and Software Engineering Paradigm.	GPS Tracking and Boundaries	Integrated tracker telemetry, start/stop tracking sessions, live-map routes, operational geofencing, configurable garage location, and vehicle proximity indicators.
3	Results and Discussions: Prepared the chapter introduction, Project Capabilities, and testing process details, including the confusion matrix.	Handover and Return	Implemented handover and return safeguards, physical vehicle receipt, late-fee cutoffs, queue pagination and search, and condition-history navigation.

4	Conclusion and Recommendations: Jointly formulated the conclusions and recommendations.	Backend and Deployment	Maintained Fastify API and PostgreSQL integrations, prepared Railway build and deployment configuration, and resolved payment, authentication, and system integration defects across renter, owner, and staff workflows.

Martinez, John Edrian V. : Interface Development and Documentation

Specific Tasks

Documentation Development

Chapter No.	Chapter Title	Module Title	Functionality/ies
1	Introduction: Jointly prepared the Statement of the Problem and Scope and Limitation of the Study.	Renter Portal	Developed fleet and vehicle-detail pages, booking calendars, renter navigation, profile controls, responsive layouts, and scrolling fixes.
1	Introduction: Reviewed and synthesized the first five studies on rental management and fleet tracking, before Internet of Things Architecture (pp. 2-7).	Management Dashboards	Developed owner and staff interfaces for overview, fleet inventory, pricing and availability, handover operations, navigation sidebars, dashboard animations, and layout refinements.
2	Methodology: Prepared the Level 0 Data Flow Diagram and explanations of the system process, external entities, and data flows.	Eligibility and Booking Holds	Implemented separate identity and driving-credential checks for chauffeured/self-drive bookings, requirement review displays, automatic expired-hold release, and renter cancellation of unpaid holds.
2	Methodology: Prepared the Data Collection Methods and Instruments sections.	Notifications and Onboarding	Implemented the renter onboarding tour, requirement prompts, notification previews with read-state handling, message indicators, and automatic portal refresh after system updates.
3	Results and Discussions: Jointly prepared module screenshots and discussions connecting project capabilities to the research questions.	Tracking History and Audit	Implemented saved tracking-session history browsing and readable audit descriptions; corrected owner analytics, message visibility, revenue displays, and odometer validation.

4	Conclusion and Recommendations: Jointly formulated the conclusions and recommendations.	Performance Optimization	Optimized dashboard reads, booking search and pagination, inspection summaries, protected previews, receipts, and refresh scheduling; verified a 150-booking workload.

Navarro, Rex S. : Workflow Development and Documentation

Specific Tasks

Documentation Development

Chapter No.	Chapter Title	Module Title	Functionality/ies
1	Introduction: Prepared The Problem and its Background and Significance of the Study.	Registration and Live Selfie	Improved ID, phone, and password validation, document-field checks, face-landmark capture stages, and confirmation cues.
1	Introduction: Jointly prepared the Statement of the Problem and Scope and Limitation of the Study.	Booking and Availability	Improved date/time selection, availability validation, booking confirmation, and readable booking status history.
2	Methodology: Prepared Population and Sampling, including the population and sampling technique.	Payments and Refunds	Implemented balance-payment recording, payment evidence, calamity refund eligibility, and manual refund settlement.
2	Methodology: Prepared ethical considerations covering privacy/confidentiality, human review and fair decisions, GPS/IoT use, vehicle-condition evidence, payments and third-party services, data retention, and user rights.	Condition Review and Returns	Improved owner finding review, renter acknowledgment, inspection context, and return and balance-payment safeguards.
3	Results and Discussions: Jointly prepared module screenshots and discussions connecting project capabilities to the research questions.	Live Updates and Communication	Implemented event-driven booking, payment, and inspection refresh; improved renter notifications and transactional email templates.
3	Results and Discussions: Prepared the System Evaluation Result and Discussion of Findings by Research Question.	Reports and Demonstration Data	Improved revenue CSV exports and booking history; prepared an isolated 150-booking demonstration dataset.

APPENDIX E : Certificate of Appreciation

CERTIFICATE

OF APPRECIATION

IS PRESENTED TO:

Carl AJ G. Junio

for your significant involvement and commitment to the development and successful implementation of the capstone study entitled **"Web-Based Car Rental Management System with Real-Time IoT-Based GPS Tracking for 1816 Car Rental Co."**

Your dedication, technical expertise, and commitment to supporting the research process greatly contributed to the effciency, organization, and succesful implementation of the study.

Given this __ day of **October, 2026** at **Cainta, Rizal**

Tricia Ann Gallanosa
Owner, 1816 Car Rental Co.

CERTIFICATE

OF APPRECIATION

IS PRESENTED TO:

John Edrian V. Martinez

for your significant involvement and commitment to the development and successful implementation of the capstone study entitled **"Web-Based Car Rental Management System with Real-Time IoT-Based GPS Tracking for 1816 Car Rental Co."**

Your dedication, technical expertise, and commitment to supporting the research process greatly contributed to the effciency, organization, and succesful implementation of the study.

Given this __ day of **October, 2026** at **Cainta, Rizal**

Tricia Ann Gallanosa
Owner, 1816 Car Rental Co.

CERTIFICATE

OF APPRECIATION

IS PRESENTED TO:

Rex S. Navarro

for your significant involvement and commitment to the development and successful implementation of the capstone study entitled **"Web-Based Car Rental Management System with Real-Time IoT-Based GPS Tracking for 1816 Car Rental Co."**

Your dedication, technical expertise, and commitment to supporting the research process greatly contributed to the effciency, organization, and succesful implementation of the study.

Given this __ day of **October, 2026** at **Cainta, Rizal**

Tricia Ann Gallanosa
Owner, 1816 Car Rental Co.

APPENDIX F : Source Code

server/src/utils/rentalEligibility.js (lines 1-40)

```
export const RENTAL_MODES = Object.freeze({
  SELF_DRIVE: 'self_drive',
  WITH_DRIVER: 'with_driver',
});

export function normalizeRentalMode(value, fallback =
RENTAL_MODES.SELF_DRIVE) {
  if (value === 'driver' || value ===
RENTAL_MODES.SELF_DRIVE) return
RENTAL_MODES.SELF_DRIVE;
  if (value === 'passenger' || value ===
RENTAL_MODES.WITH_DRIVER) return
RENTAL_MODES.WITH_DRIVER;
  return fallback;
}

export function licenseIsCurrent(licenseExpiryDate, now =
new Date()) {
  if (!licenseExpiryDate) return false;
  const expiry = new
Date(`${String(licenseExpiryDate).slice(0,
10)}T23:59:59+08:00`);
  return !Number.isNaN(expiry.getTime()) && expiry >=
now;
}

export function allowedRentalModes({ identityStatus,
selfDriveStatus, licenseExpiryDate }) {
  if (identityStatus !== 'verified') return [];
  const modes = [RENTAL_MODES.WITH_DRIVER];
  if (selfDriveStatus === 'verified' &&
licenseIsCurrent(licenseExpiryDate))
modes.unshift(RENTAL_MODES.SELF_DRIVE);
  return modes;
}

export function rentalModeEligibility({ rentalType,
identityStatus, selfDriveStatus, licenseExpiryDate }) {
  const normalized = normalizeRentalMode(rentalType,
null);
  if (!normalized) {
    return { allowed: false, code:
'INVALID_RENTAL_MODE', message: 'Choose Self-Drive
or With driver.' };
  }
  if (identityStatus !== 'verified') {
    return { allowed: false, code:
'IDENTITY_NOT_VERIFIED', message: 'Your identity
verification must be approved before booking.' };
  }
  if (normalized === RENTAL_MODES.SELF_DRIVE &&
selfDriveStatus !== 'verified') {
    return { allowed: false, code:
'SELF_DRIVE_NOT_VERIFIED', message: 'Your Driver\'s
License and LTO proof must be approved before a
Self-Drive booking.' };
  }
  if (normalized === RENTAL_MODES.SELF_DRIVE &&
!licenseIsCurrent(licenseExpiryDate)) {
    return { allowed: false, code:
'DRIVER_LICENSE_EXPIRED', message: 'Your Driver\'s
License is expired or missing an expiration date. Update it
before booking Self-Drive.' };
  }
  return { allowed: true, mode: normalized };
}
```

server/src/services/damageInference.js (lines 9-55)

```
function intersectionOverUnion(a, b) {
  const left = Math.max(a.x, b.x);
  const top = Math.max(a.y, b.y);
  const right = Math.min(a.x + a.width, b.x + b.width);
  const bottom = Math.min(a.y + a.height, b.y + b.height);
  const intersection = Math.max(0, right - left) *
Math.max(0, bottom - top);
  const union = a.width * a.height + b.width * b.height -
intersection;
  return union > 0 ? intersection / union : 0;
}

function intersectionOverSmallerBox(a, b) {
  const left = Math.max(a.x, b.x);
  const top = Math.max(a.y, b.y);
  const right = Math.min(a.x + a.width, b.x + b.width);
  const bottom = Math.min(a.y + a.height, b.y + b.height);
  const intersection = Math.max(0, right - left) *
Math.max(0, bottom - top);
  const smallerArea = Math.min(a.width * a.height, b.width
* b.height);
  return smallerArea > 0 ? intersection / smallerArea : 0;
}

export function nonMaximumSuppression(detections,
threshold) {
  const selected = [];
  const safeThreshold = Math.max(0, Math.min(1,
Number(threshold) || 0.45));
  const containmentThreshold = Math.max(0.7,
safeThreshold + 0.2);
  for (const candidate of [...detections].sort((a, b) =>
b.confidence - a.confidence)) {
    const duplicate = selected.some((item) =>
item.damageType === candidate.damageType && (
      intersectionOverUnion(item.bbox, candidate.bbox) >=
safeThreshold
      || intersectionOverSmallerBox(item.bbox,
candidate.bbox) >= containmentThreshold
    ));
    if (!duplicate) {
      selected.push(candidate);
    }
  }
  return selected;
}

export async function inspectImage(filePath) {
  const [metadata, stats] = await Promise.all([
    sharp(filePath, { failOn: 'error' }).metadata(),
    sharp(filePath, { failOn: 'error' }).stats(),
  ]);
  const warnings = [];
  if (!metadata.width || !metadata.height)
warnings.push('dimensions_unavailable');
  if (Math.min(metadata.width || 0, metadata.height || 0) <
640) warnings.push('low_resolution');
  if (Number.isFinite(stats.entropy) && stats.entropy < 3)
warnings.push('low_detail');
  return { width: metadata.width || null, height:
metadata.height || null, warnings };
}
```

src/utils/api.js (lines 14-85)

```
const BASE_URL = '/api';
const inFlightGets = new Map();

function resolveApiUrl(path) {
```

```
  if (/^https?:\/\//i.test(path)) return path;
  const normalized = path.startsWith('/') ? path : `/${path}`;
  if (normalized.startsWith('/api/') || normalized === '/api') {
    return normalized;
  }
  return `${BASE_URL}${normalized}`;
}

/**
 * Wrapper around fetch for public API calls.
 * Automatically sets JSON headers and handles non-OK
responses.
 *
 * @param {string} path - API path (e.g. '/vehicles')
 * @param {RequestInit} [options] - Fetch options
 * @returns {Promise<any>} Parsed JSON response
 */
export async function apiFetch(path, options = {}) {
  const isFormData = options.body instanceof FormData;
  const token = typeof localStorage !== 'undefined' ?
localStorage.getItem('token') : null;
  const headers = {
    ...(token && !options.headers?.Authorization ? {
Authorization: `Bearer ${token}` } : {}),
    ...(options.headers || {}),
  };

  // Only set application/json if we are sending a JSON
body
  if (options.body && !isFormData) {
    headers['Content-Type'] = 'application/json';
  }

  const url = resolveApiUrl(path);
  const canShare = !options.body && !options.signal &&
    (options.method || 'GET').toUpperCase() === 'GET' &&
    Object.keys(options).every((key) => key === 'headers' ||
key === 'method') &&
    Object.keys(headers).every((key) =>
key.toLowerCase() === 'authorization');
  const key = canShare ? `${url}\n${headers.Authorization
|| headers.authorization || ''}` : null;
  if (key && inFlightGets.has(key)) return
inFlightGets.get(key);

  const request = (async () => {
    const res = await fetch(url, {
      ...options,
      headers,
      body: options.body
        ? (isFormData || typeof options.body === 'string' ?
options.body : JSON.stringify(options.body))
        : undefined,
    });
    const data = await res.json().catch(() => ({}));
    if (!res.ok) {
      const msg = data.message || data.error || `HTTP
${res.status}`;
      const err = new Error(msg);
      err.status = res.status;
      err.data = data;
      err.code = data.code;
      err.booking_id = data.booking_id;
      err.is_my_hold = data.is_my_hold;
      throw err;
    }
    return data;
  })();

  if (key) {
    inFlightGets.set(key, request);
    request.then(
      () => { if (inFlightGets.get(key) === request)
inFlightGets.delete(key); },
      () => { if (inFlightGets.get(key) === request)
inFlightGets.delete(key); },
    );
  }
  return request;
}
```

server/src/routes/bookings.js (lines 111-131)

```
export function parseBookingWindow(body = {}) {
  const startAt = toManilaDateTime(body.start_date,
body.start_time, body.start_at);
  const endAt = toManilaDateTime(body.end_date,
body.end_time, body.end_at);

  if (!startAt || !endAt || Number.isNaN(startAt.getTime()) ||
Number.isNaN(endAt.getTime())) {
    return { error: 'start date/time and end date/time are
required.' };
  }
  if (startAt.getTime() <= Date.now()) {
    return { error: 'Pick-up must be scheduled for a future
date and time. Choose a later pick-up time.' };
  }
  if (endAt <= startAt) {
    return { error: 'End date/time must be after start
date/time.' };
  }

  return {
    startAt: startAt.toISOString(),
    endAt: endAt.toISOString(),
    startDate: manilaDateString(startAt),
    endDate: manilaDateString(endAt),
  };
}
```

server/src/routes/conditionChecks.js (lines 507-589)

```
fastify.post('/:id/checks/:checkId/damage-assessment/finali
ze', { preHandler: [authenticate, roleGuard('owner')] },
async (request, reply) => {
    const client = await pool.connect();
    try {
      await client.query('BEGIN');
      const result = await client.query(
        `SELECT da.*, vc.booking_id, vc.check_type,
vc.status AS check_status, b.status AS booking_status
         FROM damage_assessments da
         JOIN vehicle_checks vc ON vc.check_id =
da.check_id
         JOIN bookings b ON b.booking_id = vc.booking_id
         WHERE da.check_id = $1 AND vc.booking_id = $2
         FOR UPDATE OF da, vc`,
        [request.params.checkId, request.params.id]
      );
      const assessment = result.rows[0];
      if (!assessment) {
        await client.query('ROLLBACK');
        return reply.status(404).send({ message: 'Damage
assessment not found.' });
      }
      if (assessment.check_status !== 'draft' || ['finalized',
'manual_fallback'].includes(assessment.status)) {
        await client.query('ROLLBACK');
```

```
      return reply.status(409).send({ message: 'This
assessment has already been finalized.' });
    }
    if (!['review_required',
'failed'].includes(assessment.status)) {
      await client.query('ROLLBACK');
      return reply.status(409).send({ message: 'Wait for
analysis to finish, or retry it if it failed.' });
    }


    const modelResult = await client.query(
      `SELECT finding_id FROM damage_findings
       WHERE assessment_id = $1 AND source = 'model'
AND run_id = $2`,
      [assessment.assessment_id,
assessment.active_run_id]
    );
    const payload =
validateAssessmentPayload(request.body, {
      checkType: assessment.check_type,
      modelFindingIds: modelResult.rows.map((row) =>
Number(row.finding_id)),
      failed: assessment.status === 'failed',
    });
    const submittedIds = payload.findings.filter((item) =>
item.findingId).map((item) => item.findingId);
    if (new Set(submittedIds).size !== submittedIds.length)
{
      await client.query('ROLLBACK');
      return reply.status(400).send({ message: 'A finding
can only be reviewed once.' });
    }


    const mediaResult = await client.query('SELECT
media_id FROM check_media WHERE check_id = $1',
[request.params.checkId]);
    const mediaIds = new
Set(mediaResult.rows.map((row) =>
Number(row.media_id)));
    await client.query(`DELETE FROM damage_findings
WHERE assessment_id = $1 AND source = 'manual'`,
[assessment.assessment_id]);
    for (const item of payload.findings) {
      if (item.mediaId && !mediaIds.has(item.mediaId)) {
        throw Object.assign(new Error('A finding references
evidence from another check.'), { statusCode: 400 });
      }
      if (item.findingId) {
        const update = await client.query(
          `UPDATE damage_findings SET
             damage_type = $3, disposition = $4, severity =
$5, affected_area = $6, notes = $7,
             reviewed_by = $8, reviewed_at = NOW()
           WHERE finding_id = $1 AND assessment_id = $2
AND source = 'model' AND run_id = $9
           RETURNING finding_id`,
          [item.findingId, assessment.assessment_id,
item.damageType, item.disposition, item.severity,
           item.affectedArea, item.notes,
request.user.user_id, assessment.active_run_id]
        );
        if (!update.rows[0]) throw Object.assign(new
Error('A model finding is no longer available.'), {
statusCode: 409 });
      } else {
        await client.query(
          `INSERT INTO damage_findings
            (assessment_id, media_id, source,
damage_type, bbox_normalized, disposition, severity,
             affected_area, notes, reviewed_by, reviewed_at)
           VALUES
($1,$2,'manual',$3,$4::jsonb,$5,$6,$7,$8,$9,NOW())`,
          [assessment.assessment_id, item.mediaId,
item.damageType,
           item.bboxNormalized ?
JSON.stringify(item.bboxNormalized) : null,
item.disposition,
           item.severity, item.affectedArea, item.notes,
request.user.user_id]
        );
      }
    }


    const finalStatus = assessment.status === 'failed' ?
'manual_fallback' : 'finalized';
    await client.query(
      `UPDATE damage_assessments SET status = $2,
owner_summary = $3, manual_fallback_reason = $4,
        reviewed_by = $5, reviewed_at = NOW(),
updated_at = NOW()
       WHERE assessment_id = $1`,
      [assessment.assessment_id, finalStatus,
payload.summary, payload.fallbackReason || null,
request.user.user_id]
    );
    await client.query(
      `UPDATE vehicle_checks SET status = 'submitted',
submitted_at = NOW() WHERE check_id = $1`,
```

server/src/routes/payments.js (lines 475-560)

```
  fastify.post('/balance', { preHandler: [authenticate,
roleGuard('owner', 'staff')] }, async (request, reply) => {
    let fields = {};
    let savedRelative = null;


    if (request.isMultipart?.()) {
      for await (const part of request.parts()) {
        if (part.file) {
          if (part.fieldname === 'proof_image') {
            try {
              savedRelative = await saveProofFile(part, 'tmp');
            } catch (err) {
              return reply.status(err.statusCode || 500).send({
message: err.message });
            }
          } else {
            part.file.resume();
          }
          continue;
        }
        fields[part.fieldname] = part.value;
      }
    } else {
      fields = request.body || {};
      if (typeof fields === 'string') {
        try {
          fields = JSON.parse(fields);
        } catch {
          fields = {};
        }
      }
    }


    const bookingId = Number(fields.booking_id || fields.id);
    const paymentKind = ['balance',
'surcharge'].includes(fields.payment_kind) ?
fields.payment_kind : 'balance';
```

```
    const channel = String(fields.settlement_channel ||
fields.payment_method || fields.channel ||
'').trim().toLowerCase();
    const account = String(fields.settlement_account ||
fields.account || '').trim();
    const reference = String(fields.reference_number ||
fields.reference || '').trim();
    const notes = String(fields.notes || '').trim();
    const paidAtRaw = fields.paid_at ? new
Date(fields.paid_at) : new Date();
    let amount = Number(fields.amount);


    if (!bookingId || !channel) {
      return reply.status(400).send({
        message: 'booking_id and settlement_channel are
required.',
      });
    }
    if (!['cash', 'card', 'gcash', 'maya', 'bank_transfer',
'other'].includes(channel)) {
      return reply.status(400).send({ message: 'Invalid
settlement channel. Allowed: cash, card, gcash, maya,
bank_transfer, or other.' });
    }
    const settledAccount = account
      || (channel === 'cash' ? 'Cash on hand' : (channel ===
'card' ? 'Card / POS Terminal' : channel.toUpperCase()));


    if (Number.isNaN(paidAtRaw.getTime())) {
      return reply.status(400).send({ message: 'paid_at must
be a valid date/time.' });
    }


    const bookingResult = await query(
      `SELECT booking_id, renter_id, status, total_amount,
down_payment_amount, calamity_return_required,
actual_return_at
       FROM bookings WHERE booking_id = $1`,
      [bookingId]
    );
    const booking = bookingResult.rows[0];
    if (!booking) return reply.status(404).send({ message:
'Booking not found.' });
    if (!['confirmed', 'active',
'completed'].includes(booking.status) &&
!needsVehicleReturn(booking)) {
      return reply.status(409).send({
        message: 'Balance payments can be recorded after
the reservation deposit is confirmed.',
      });
    }


    const paidResult = await query(
      `SELECT COALESCE(SUM(amount), 0)::numeric AS
paid
       FROM payments
       WHERE booking_id = $1
         AND status = 'verified'
         AND COALESCE(payment_kind, 'reservation') <>
'surcharge'`,
      [bookingId]
    );
    let verifiedPaid = Number(paidResult.rows[0]?.paid || 0);
    let remaining = Math.max(0,
Number((Number(booking.total_amount) -
verifiedPaid).toFixed(2)));


    if (paymentKind === 'balance') {
      if (!Number.isFinite(amount) || amount <= 0) amount =
remaining;
      if (amount > remaining + 0.01) {
        return reply.status(400).send({
          message: `Amount exceeds remaining balance of
PHP ${remaining.toFixed(2)}.`,
        });
      }
```

src/App.jsx (lines 130-220)

```
  const [sidebarCollapsed, setSidebarCollapsed] =
useState(false);


  // Renter portal and management pages — no global
Navbar for these
  const isRenterPortal =
location.pathname.startsWith('/renter');
  const isManagementDashboard =
    location.pathname.startsWith('/dashboard/staff')
    || location.pathname.startsWith('/dashboard/owner')
    || location.pathname.startsWith('/owner')
    || location.pathname.startsWith('/staff');


  const isDashboardOrCheckout =
    location.pathname.includes('/dashboard')
    || location.pathname.includes('/checkout')
    || location.pathname.startsWith('/owner')
    || location.pathname.startsWith('/staff');


  const isAuthenticatedArea =
    isDashboardOrCheckout
    || isRenterPortal
    || location.pathname.startsWith('/booking')
    || location.pathname.startsWith('/bookings')
    || location.pathname.startsWith('/booking-requirements')
    || location.pathname.startsWith('/payment')
    || location.pathname.startsWith('/profile')
    || location.pathname.startsWith('/notifications');


  useEffect(() => {
    if (location.state?.authModal) {
      setAuthModal(location.state.authModal);
      navigate(location.pathname, { replace: true, state: null
});
    }
  }, [location.pathname, location.state, navigate]);


  useEffect(() => {
    const handleAuthOpen = (e) => setAuthModal(e.detail ||
'login');
    window.addEventListener('auth:open',
handleAuthOpen);
    return () => window.removeEventListener('auth:open',
handleAuthOpen);
  }, []);


  return (
    <div className="app">
      <ScrollToTop />


      {/* Show unified 1816 topbar on all public pages */}
      {!(isRenterPortal || isManagementDashboard) && (
        <Navbar
          onLoginClick={() => setAuthModal('login')}
          onRegisterClick={() => setAuthModal('register')}
          sidebarCollapsed={sidebarCollapsed}
```

```
        setSidebarCollapsed={setSidebarCollapsed}
      />
    )}

    <AuthModal
      isOpen={!!authModal}
      initialView={authModal || 'login'}
      onClose={() => setAuthModal(null)}
    />

    <div className={isRenterPortal || location.pathname
=== '/' ? '' : 'app__content'}>
      <Routes>
        {/* ── Public routes ── */}
        <Route path="/"
element={<PublicOnlyRoute><Home onAuth={(m) =>
setAuthModal(m)} /></PublicOnlyRoute>} />
        <Route path="/vehicles" element={<Vehicles />} />
        <Route path="/vehicles/:id" element={<VehicleDetail
/>} />
        <Route path="/workflow"
element={<PublicWorkflowIndex />} />
        <Route path="/terms" element={<LegalPage
policy="terms" />} />
        <Route path="/privacy" element={<LegalPage
policy="privacy" />} />

        {/* ── Renter portal (all pages within RenterLayout
sidebar) ── */}
        <Route path="/renter"
element={<RenterPortalRoute><Navigate
to="/renter/home" replace /></RenterPortalRoute>} />
        <Route path="/renter/home"
element={<RenterPortalRoute><RenterDashboard
/></RenterPortalRoute>} />
        <Route path="/renter/messages"
element={<RenterPortalRoute><RenterInbox
/></RenterPortalRoute>} />
        <Route path="/renter/fleet"
element={<RenterPortalRoute><RenterFleet
/></RenterPortalRoute>} />
        <Route path="/renter/receipts"
element={<RenterPortalRoute><RenterReceipts
/></RenterPortalRoute>} />
        <Route path="/renter/inbox"
element={<RenterPortalRoute><RenterInbox
/></RenterPortalRoute>} />
        <Route path="/renter/bookings"
element={<RenterPortalRoute><RenterBookings
/></RenterPortalRoute>} />
        <Route path="/renter/bookings/:bookingId"
element={<RenterPortalRoute><RenterBookingDetail
/></RenterPortalRoute>} />
        <Route path="/renter/requirements"
element={<RenterPortalRoute><RenterRequirements
/></RenterPortalRoute>} />
        <Route path="/renter/notifications"
element={<RenterPortalRoute><Navigate
to="/renter/inbox" replace /></RenterPortalRoute>} />
        <Route path="/renter/profile"
element={<RenterPortalRoute><RenterProfile
/></RenterPortalRoute>} />
        <Route path="/renter/terms"
element={<RenterPortalRoute><LegalPage
policy="terms" renter /></RenterPortalRoute>} />
        <Route path="/renter/privacy"
element={<RenterPortalRoute><LegalPage
policy="privacy" renter /></RenterPortalRoute>} />
        <Route path="/renter/vehicle/:id"
element={<RenterPortalRoute><RenterVehicleDetail
/></RenterPortalRoute>} />
        <Route path="/renter/booking/:id"
element={<RenterPortalRoute><RenterBookingPage
/></RenterPortalRoute>} />
        <Route path="/renter/checkout/:id"
element={<RenterPortalRoute><LegacyCheckoutRedirect
/></RenterPortalRoute>} />
        <Route path="/renter/payment/:bookingId"
element={<RenterPortalRoute><RenterPaymentPage
/></RenterPortalRoute>} />

        {/* Legacy renter paths → redirect to portal */}
        <Route path="/dashboard/renter"
element={<Navigate to="/renter" replace />} />
        <Route path="/bookings"
element={<ProtectedRoute roles={['renter']}><Navigate
to="/renter/bookings" replace /></ProtectedRoute>} />
```

src/context/AuthContext.jsx (lines 1-150)

```
import { createContext, useCallback, useContext,
useState, useEffect } from 'react';
import { BASE_URL } from '../utils/api';

const AuthContext = createContext();

export const useAuth = () => useContext(AuthContext);

function normalizeUser(user) {
  if (!user) return null;
  return {
    ...user,
    full_name: user.full_name || user.name || user.email ||
'User',
  };
}

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
  const [authUnavailable, setAuthUnavailable] =
useState(false);

  // Initialize auth state
  useEffect(() => {
    const initAuth = async () => {
      const token = localStorage.getItem('token');
      if (token) {
        try {
          const res = await fetch(`${BASE_URL}/auth/me`, {
            headers: {
              Authorization: `Bearer ${token}`
            }
          });
          if (res.ok) {
            const data = await res.json();
            setUser(normalizeUser(data.user));
            setAuthUnavailable(false);
          } else if (res.status === 401 || res.status === 404) {
            localStorage.removeItem('token');
          } else {
            setAuthUnavailable(true);
          }
        } catch (error) {
          console.error('Auth check failed', error);
          setAuthUnavailable(true);
        }
```

```
      }
      setLoading(false);
    };

    initAuth();
  }, []);

  const parseJsonResponse = async (res) => {
    const text = await res.text();
    if (!text) return {};
    try {
      return JSON.parse(text);
    } catch {
      throw new Error('Server returned an invalid response.
Make sure the backend is running on port 5000.');
    }
  };

  const login = async (email, password) => {
    let res;
    try {
      res = await fetch(`${BASE_URL}/auth/login`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ email: email.trim(), password
})
      });
    } catch {
      throw new Error('Cannot reach the API server. Start it
with: cd server && npm run dev');
    }

    const data = await parseJsonResponse(res);

    if (!res.ok) {
      throw new Error(data.message || data.error || 'Login
failed');
    }
    data.user = normalizeUser(data.user);
    localStorage.setItem('token', data.token);
    setUser(data.user);
    setAuthUnavailable(false);
    return data;
  };

  const register = async (profile) => {
    const body = new FormData();
    Object.entries(profile).forEach(([key, value]) => {
      if (value !== undefined && value !== null && value !==
'') {
        body.append(key, value);
      }
    });

    const res = await fetch(`${BASE_URL}/auth/register`, {
      method: 'POST',
      body
    });

    const data = await parseJsonResponse(res);

    if (!res.ok) {
      const error = new Error(data.message || data.error ||
'Registration failed');
      error.code = data.code;
      throw error;
    }
    data.user = normalizeUser(data.user);
    localStorage.setItem('token', data.token);
    setUser(data.user);
    setAuthUnavailable(false);
    return data;
  };

  const logout = () => {
    try {
      sessionStorage.clear();
    } catch {}
    localStorage.removeItem('token');
    setUser(null);
    setAuthUnavailable(false);
  };

  const refreshUser = useCallback(async () => {
    const token = localStorage.getItem('token');
    if (!token) return null;

    let res;
    try {
      res = await fetch(`${BASE_URL}/auth/me`, {
        headers: { Authorization: `Bearer ${token}` },
      });
    } catch (error) {
      setAuthUnavailable(true);
      throw error;
    }

    if (res.status === 401 || res.status === 404) {
      localStorage.removeItem('token');
      setUser(null);
      setAuthUnavailable(false);
      return null;
    }

    if (!res.ok) {
      setAuthUnavailable(true);
      throw new Error('Unable to refresh your account right
now.');
    }

    const data = await res.json();
    const nextUser = normalizeUser(data.user);
    setUser(nextUser);
```

src/components/renter/RenterBookingFlow.jsx (lines 150-260)

```
        setRequirement(data.requirement);
        if (requirementStatus(data.requirement,
user?.preferred_rental_mode === 'with_driver' ?
'with_driver' : 'self_drive') === 'verified') {
          setStep(1); // Auto-advance verified renters directly
to Trip details
        }
      })
      .catch(() => setRequirement(null))
      .finally(() => setRequirementsLoading(false));
  }, [checkout, user?.preferred_rental_mode]);

  useRenterAutoRefresh(() => {
    if (!checkout) return;
    authFetch('/requirements/me')
      .then((data) => {
```

```
        setRequirement(data.requirement);
        if (requirementStatus(data.requirement,
tripForm.rental_type) === 'verified') setStep((current) =>
current === 0 ? 1 : current);
      })
      .catch(() => {});
    if (checkout.vehicle?.id) {

apiFetch(`/bookings/availability/${checkout.vehicle.id}`)
        .then((data) => setBlockedDates(data.blocked_dates
|| []))
        .catch(() => {});
    }
  });


  useEffect(() => {
    if (!booking?.payment_due_at) {
      setHoldSeconds(0);
      return undefined;
    }
    const tick = () => {
      const remaining = Math.max(0, Math.floor((new
Date(booking.payment_due_at).getTime() - Date.now()) /
1000));
      setHoldSeconds(remaining);
    };
    tick();
    const timer = setInterval(tick, 1000);
    return () => clearInterval(timer);
  }, [booking?.payment_due_at]);


  useEffect(() => {
    if (!checkout) return;
    setCheckoutForm((prev) => ({
      ...prev,
      full_name: user?.full_name || prev.full_name,
      email: user?.email || prev.email,
      phone_number: user?.phone_number ||
prev.phone_number,
    }));
  }, [checkout, user?.full_name, user?.email,
user?.phone_number]);


  async function createBookingHold() {
    if (!vehicle || !pickupParts || !retParts) return null;
    const policyError = bookingPolicyError(checkoutForm);
    if (policyError) throw new Error(policyError);
    const result = await authFetch('/bookings', {
      method: 'POST',
      body: {
        vehicle_id: Number(vehicle.id),
        start_date: pickupParts.date,
        end_date: retParts.date,
        start_time: pickupParts.time,
        end_time: retParts.time,
        rental_type: tripForm.rental_type,
        destination: formatBookingDestination(tripForm.city,
tripForm.province),
        terms_accepted: checkoutForm.terms_accepted,
        privacy_acknowledged:
checkoutForm.privacy_acknowledged,
        terms_version: checkoutForm.terms_version,
        privacy_version: checkoutForm.privacy_version,
      },
    });
    return result.booking;
  }
```

```
  async function handleContinue() {
    setMessage('');
    if (step === 0 && (requirementsLoading || reqStatus !==
'verified')) {
      setMessage(reqStatus === 'pending'
        ? 'Your requirements are still being verified.'
        : 'Complete booking requirements before reserving.');
      return;
    }
    if (step === 1) {
      if (!isValidBookingSchedule(pickup, ret)) {
        setMessage('Choose a future pick-up time and a
return time after it.');
        setShowDateEdit(true);
        return;
      }
      const policyError =
bookingPolicyError(checkoutForm);
      if (policyError) { setMessage(policyError); return; }
      if (reqStatus !== 'verified') {
        setStep(0);
        setMessage(tripForm.rental_type === 'self_drive'
          ? 'Complete Self-Drive verification before choosing
this rental type.'
          : 'Your identity verification must be approved before
booking.');
        return;
      }
      if (!tripForm.province || !tripForm.city) {
        setMessage('Choose a province and city or
municipality before reserving.');
        return;
      }
      if (`${retParts?.date}T${retParts?.time}` <=
`${pickupParts?.date}T${pickupParts?.time}`) {
        setMessage('Return must be after pick-up.');
        return;
      }
      if (totalDays < 1) {
        setMessage('Rental duration must be at least 1
calendar day.');
        return;
      }


      // If user already created a booking hold for this
session and returned to step 1 to modify:
      if (booking?.booking_id) {
        const currentStartAt = booking.start_at ? new
Date(booking.start_at).getTime() : null;
        const newStartAt = new
Date(`${pickupParts.date}T${pickupParts.time ||
'10:00'}:00+08:00`).getTime();
```

src/pages/Dashboard/Renter/RenterBookingDetail.jsx (lines 580-690)

```
          </div>
        </div>


        {refundMessage && <p className="text-sm
text-ink-soft">{refundMessage}</p>}
        <Button type="submit" className="font-semibold
shadow-xs" disabled={refundSaving ||
refundReason.trim().length < 10}>
          {refundSaving ? 'Submitting…' : 'Submit Refund
Request'}
        </Button>
      </form>
    );
  }
```

```
  if (eligibility?.in_calamity_period) {
    return (
      <div className="rounded-2xl border
border-amber-300/70 bg-amber-50 p-5" role="status">
        <div className="flex items-start gap-2.5">
          <Info width={18} height={18} className="mt-0.5
shrink-0 text-amber-800" />
          <div>
            <h2 className="font-display text-base
font-semibold text-amber-950">Calamity window covers
this booking</h2>
            <p className="mt-1 text-[13px] leading-relaxed
text-amber-950/80">
              {label}{range ? ` (${range})` : ''}. {eligibility.reason
|| 'Refund request is not available yet for this booking.'}
            </p>
          </div>
        </div>
      </div>
    );
  }

  return null;
}

export function CheckCard({ check, onAcknowledge,
saving, feedback }) {
  const [media, setMedia] = useState([]);
  const [reportSaving, setReportSaving] = useState(false);
  const [reportError, setReportError] = useState('');
  const [selectedPhoto, setSelectedPhoto] =
useState(null);
  const [zoomed, setZoomed] = useState(false);

  useEffect(() => {
    if (selectedPhoto === null) return undefined;
    const onKey = (event) => {
      if (event.key === 'Escape') setSelectedPhoto(null);
      if (event.key === 'ArrowRight')
setSelectedPhoto((index) => (index + 1) % media.length);
      if (event.key === 'ArrowLeft') setSelectedPhoto((index)
=> (index - 1 + media.length) % media.length);
    };
    window.addEventListener('keydown', onKey);
    return () => window.removeEventListener('keydown',
onKey);
  }, [selectedPhoto, media.length]);

  useEffect(() => {
    let cancelled = false;
    const urls = [];
    Promise.all((check.media || []).map(async (item) => {
      const blob = await authBlobFetch(item.url);
      const url = URL.createObjectURL(blob);
      urls.push(url);
      return { ...item, blobUrl: url };
    })).then((items) => { if (!cancelled) setMedia(items);
}).catch(() => setMedia([]));
    return () => { cancelled = true; urls.forEach((url) =>
URL.revokeObjectURL(url)); };
  }, [check.check_id, check.media]);

  const downloadReport = async () => {
    setReportSaving(true);
    setReportError('');
    try {
      const blob = await
authBlobFetch(`/bookings/${check.booking_id}/checks/${c
heck.check_id}/damage-report.pdf`);
      if (blob.size < 5 || (blob.type && blob.type !==
'application/pdf')) throw new Error('The report could not be
generated as a PDF.');
      const url = URL.createObjectURL(blob);
      const link = document.createElement('a');
      link.href = url;
      link.download =
`booking-${check.booking_id}-${check.check_type}-conditi
on-report.pdf`;
      link.click();
      window.setTimeout(() => URL.revokeObjectURL(url),
1000);
    } catch (err) {
      setReportError(err.message || 'Could not download
condition report PDF. Please retry.');
    } finally {
      setReportSaving(false);
    }
  };

  const isPre = check.check_type === 'pre';

  return (
    <article className="booking-inspection">
      <div className="flex flex-wrap items-center
justify-between gap-2 border-b border-line pb-3">
        <div>
          <h3 className="text-sm font-semibold text-ink">
            {isPre ? 'Pre-handover inspection' : 'Return
inspection'}
          </h3>
          <p className="text-xs text-ink-soft">
            {isPre ? 'Conducted before key release.' :
'Conducted when the vehicle was returned.'}
          </p>
        </div>
        <span className={`rounded-full px-2.5 py-0.5
text-xs font-medium capitalize ${
          check.status === 'acknowledged' ?
'booking-check-status is-signed' : 'booking-check-status
is-pending'
        }`}>
          {check.status === 'acknowledged' ? 'Acknowledged'
: 'Awaiting Sign-off'}
        </span>
      </div>

      {/* Metrics Row */}
      <div className="booking-inspection__metrics">
        <div>
          <span className="booking-field-label">Odometer
Reading</span>
          <div className="font-mono font-semibold text-ink
mt-0.5">
            {check.odometer_km != null ?
`${Number(check.odometer_km).toLocaleString()} km` :
'—'}
          </div>
        </div>
        <div>
          <span className="booking-field-label">Fuel
Level</span>
          <div className="font-mono font-semibold text-ink
mt-0.5">
```

```
          {check.fuel_level_percent != null ?
`${check.fuel_level_percent}% Full` : '—'}
```

server/src/routes/auth.js (lines 1-65)

```
import { authenticate } from '../middleware/auth.js';
import { roleGuard } from '../middleware/roleGuard.js';
import {
  register,
  login,
  getMe,
  markRenterTourSeen,
  updateRenterProfile,
  sendPasswordOtp,
  verifyPasswordOtpAndChange,
  sendRegistrationOtp,
} from '../controllers/authController.js';
import { scanId } from '../controllers/ocrController.js';

/**
 * Auth routes plugin for Fastify.
 * Prefix: /api/auth
 */
export default async function authRoutes(fastify) {
  // POST /api/auth/scan-id - OCR ID text extraction
  fastify.post('/scan-id', scanId);

  // POST /api/auth/send-registration-otp - Send 6-digit
OTP code to verify prospective renter's email
  fastify.post('/send-registration-otp', sendRegistrationOtp);

  // POST /api/auth/register - renter self-registration with
primary ID upload and OTP verification
  fastify.post('/register', register);

  // POST /api/auth/login - all roles
  fastify.post('/login', {
    schema: {
      body: {
        type: 'object',
        required: ['email', 'password'],
        properties: {
          email: { type: 'string' },
          password: { type: 'string' },
        },
      },
    },
  }, login);

  // GET /api/auth/me - get current user (authenticated)
  fastify.get('/me', {
    preHandler: [authenticate],
  }, getMe);

  fastify.patch('/me/onboarding-tour', {
    preHandler: [authenticate, roleGuard('renter')],
  }, markRenterTourSeen);

  fastify.patch('/me', {
    preHandler: [authenticate, roleGuard('renter')],
  }, updateRenterProfile);

  // POST /api/auth/send-password-otp - Send 6-digit OTP
code to authenticated user's email
  fastify.post('/send-password-otp', {
    preHandler: [authenticate],
  }, sendPasswordOtp);

  // POST /api/auth/change-password-otp - Verify OTP and
update password
  fastify.post('/change-password-otp', {
    preHandler: [authenticate],
  }, verifyPasswordOtpAndChange);
}
```

server/src/controllers/authController.js (lines 166-260)

```
export async function register(request, reply) {
  let parsed;
  try {
    parsed = await parseRegisterRequest(request);
  } catch (error) {
    return reply.status(error.statusCode || 500).send({
      error: error.statusCode ? 'Validation Error' : 'Upload
Error',
      message: error.message,
    });
  }

  const { fields, pendingFile, pendingBackFile,
pendingSelfieFile } = parsed;
  const consentError = registrationConsentError(fields);
  if (consentError) {
    await Promise.all([pendingFile, pendingBackFile,
pendingSelfieFile].filter(Boolean).map((file) =>
fs.promises.rm(file.path, { force: true })));
    return reply.status(400).send({ error: 'Validation Error',
message: consentError });
  }
  const { email, password, phone_number } = fields;
  const idFields = normalizeIdFields(fields);
  const normalizedEmail = email?.trim().toLowerCase();
  const normalizedName = idFields.fullName;
  const normalizedPhone =
normalizeRenterMobile(phone_number);

  // Emergency contact fields
  const emergencyName =
fields.emergency_contact_name?.trim() || null;
  const emergencyPhone =
fields.emergency_contact_phone?.trim() || null;
  const emergencyRel =
fields.emergency_contact_relationship?.trim() || null;

  // Driver qualification & expiration
  const licenseExpiry = idFields.licenseExpiryDate;
  const dlCodes = idFields.dlCodes;
  const dlConditions = idFields.dlConditions;
  const rentalMode = idFields.rentalMode;
  const isExpired = licenseExpiry ?
!licenseIsCurrent(licenseExpiry) : false;

  // Validate required fields
  const livenessPassed = fields.liveness_passed === true
|| fields.liveness_passed === 'true';
  const livenessMotionScore =
Number(fields.liveness_motion_score);
  let livenessChallenge = {};
  try {
    livenessChallenge = fields.liveness_challenge ?
JSON.parse(fields.liveness_challenge) : {};
  } catch {
    livenessChallenge = {};
  }
```

```
  if (!normalizedName || !normalizedEmail || !password ||
!idFields.primaryIdType || !idFields.primaryIdNumber ||
!pendingFile || !pendingSelfieFile) {
    return reply.status(400).send({
      error: 'Validation Error',
      message: 'full_name, email, password, primary ID
type, primary ID number, uploaded ID, and a live selfie are
required.',
    });
  }
  if (!normalizedPhone) {
    await Promise.all([pendingFile, pendingBackFile,
pendingSelfieFile].filter(Boolean).map((file) =>
fs.promises.rm(file.path, { force: true })));
    return reply.status(400).send({ error: 'Validation Error',
message: 'Enter an 11-digit mobile number starting with
09.' });
  }
  if (!livenessPassed ||
!Number.isFinite(livenessMotionScore) ||
livenessMotionScore < 0.16) {
    return reply.status(400).send({
      error: 'Validation Error',
      message: 'Complete the live-camera movement
challenge before creating your account.',
    });
  }


  const numberError =
idNumberError(idFields.primaryIdType,
idFields.primaryIdNumber);
  if (numberError) {
    return reply.status(400).send({ error: 'Validation Error',
message: numberError });
  }
  if (idFields.primaryIdType === 'drivers_license' &&
(!pendingBackFile || !idFields.licenseExpiryDate)) {
    return reply.status(400).send({ error: 'Validation Error',
message: 'Driver license registration requires a back
photo and expiration date.' });
  }
  if (rentalMode === RENTAL_MODES.SELF_DRIVE &&
idFields.primaryIdType !== 'drivers_license') {
    return reply.status(400).send({
      error: 'Validation Error',
      message: 'Choose a Driver\'s License as your primary
ID for Self-Drive registration, or select Chauffeured.',
    });
  }
  if (rentalMode === RENTAL_MODES.SELF_DRIVE &&
isExpired) {
    return reply.status(400).send({
      error: 'Validation Error',
      message: 'Self-Drive registration requires a Driver\'s
License that has not expired.',
    });
  }


  // Validate email format
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!emailRegex.test(normalizedEmail)) {
    return reply.status(400).send({
      error: 'Validation Error',
      message: 'Invalid email format.',
    });
  }
```

```
  const passwordError =
registrationPasswordError(password);
  if (passwordError) {
    return reply.status(400).send({
      error: 'Validation Error',
```

server/src/routes/bookings.js (lines 456-555)

```
  fastify.post('/', { preHandler: [authenticate,
roleGuard('renter')] }, async (request, reply) => {
    const policyError = bookingPolicyError(request.body ||
{});
    if (policyError) return reply.status(400).send({ message:
policyError });
    await releaseExpiredHolds();
    const [requirements, renterContact] = await
Promise.all([
      query(
        `SELECT br.status AS identity_status,
br.self_drive_status,
          COALESCE(br.driver_license_expiry_date,
iv.license_expiry_date) AS license_expiry_date
          FROM booking_requirements br
          LEFT JOIN identity_verifications iv ON iv.user_id =
br.user_id
          WHERE br.user_id = $1`,
        [request.user.user_id]
      ),
      getUserContact(request.user.user_id),
    ]);


    const { vehicle_id, rental_type, destination = '' } =
request.body || {};
    const window = parseBookingWindow(request.body);
    if (!vehicle_id || !rental_type || window.error) {
      return reply.status(400).send({
        error: 'Validation Error',
        message: window.error || 'vehicle_id and rental_type
are required.',
      });
    }


    const eligibility = rentalModeEligibility({
      rentalType: rental_type,
      identityStatus: requirements.rows[0]?.identity_status,
      selfDriveStatus:
requirements.rows[0]?.self_drive_status,
      licenseExpiryDate:
requirements.rows[0]?.license_expiry_date,
    });
    if (!eligibility.allowed) {
      return reply.status(403).send({
        error: 'Forbidden',
        code: eligibility.code,
        message: eligibility.message,
      });
    }


    const validDestination =
validateBookingDestination(destination);
    if (!validDestination) {
      return reply.status(400).send({
        error: 'Validation Error',
        message: 'Choose a city or municipality and province
within the supported travel area, up to La Union.',
      });
    }


    const vehicleResult = await query('SELECT * FROM
vehicles WHERE vehicle_id = $1', [vehicle_id]);
```

```
    const vehicle = vehicleResult.rows[0];
    if (!vehicle) {
      return reply.status(404).send({ error: 'Not Found',
message: 'Vehicle not found.' });
    }


    const rateSnapshot = Number(rental_type ===
'with_driver' ? vehicle.rate_with_driver :
vehicle.rate_self_drive);
    // Outside-NCR daily surcharge removed from booking
totals.
    // Past La Union boundary fee (PHP 1000) is collected
on return if applicable — see
paymentPolicy.LA_UNION_BOUNDARY_FEE.
    const surchargeSnapshot = 0;
    const daysResult = await query('SELECT ($2::date -
$1::date + 1)::int AS total_days', [window.startDate,
window.endDate]);
    const totalDays =
Number(daysResult.rows[0].total_days);
    if (totalDays < 1) {
      return reply.status(400).send({ message: 'Rental
duration must be at least 1 calendar day.' });
    }
    const totalAmount = rateSnapshot * totalDays;
    const downPayment = await getDefaultDownPayment();


    const client = await pool.connect();
    let booking;
    try {
      await client.query('BEGIN');
      const calamity = await findCalamityWindow(client,
window.startDate, window.endDate);
      if (calamity) {
        await client.query('ROLLBACK');
        return reply.status(409).send({ code:
'CALAMITY_PERIOD', message: 'Reservations are
unavailable during the owner-declared calamity period.
Choose dates outside this period.' });
      }


      await client.query('SELECT vehicle_id FROM vehicles
WHERE vehicle_id = $1 FOR UPDATE', [vehicle_id]);


      const ownHold = await client.query(
        `SELECT booking_id FROM bookings WHERE
vehicle_id = $1 AND renter_id = $2
          AND status = 'awaiting_payment' AND
payment_due_at > NOW() ORDER BY created_at DESC
LIMIT 1`,
        [vehicle_id, request.user.user_id]
      );
      if (ownHold.rows.length) {
        const holdBookingId = ownHold.rows[0].booking_id;
        const overlap = await findOverlappingBooking(client,
{
          vehicleId: vehicle_id,
          excludeBookingId: holdBookingId,
          startAt: window.startAt,
          endAt: window.endAt,
          statuses: HOLDING_BOOKING_STATUSES,
        });
        if (overlap) {
          await client.query('ROLLBACK');
          return reply.status(409).send({
            error: 'Conflict',
            code: 'OVERLAPPING_BOOKING',
            booking_id: overlap.booking_id,
            message: overlapConflictMessage(),
          });
        }
```

server/src/routes/bookings.js (lines 987-1070)

```
  fastify.post('/:id/checkout', { preHandler: [authenticate,
roleGuard('renter')] }, async (request, reply) => {
    const policyError =
bookingPolicyError(request.body?.checkout_details ||
request.body || {});
    if (policyError) return reply.status(400).send({ message:
policyError });
    const client = await pool.connect();
    let booking;
    let checkoutBooking;
    try {
      await client.query('BEGIN');
      const current = await client.query(
        `SELECT b.*, v.make, v.model
         FROM bookings b
         JOIN vehicles v ON v.vehicle_id = b.vehicle_id
         WHERE b.booking_id = $1
         FOR UPDATE OF b`,
        [request.params.id]
      );
      booking = current.rows[0];
      if (!booking) {
        await client.query('ROLLBACK');
        return reply.status(404).send({ message: 'Booking
not found.' });
      }
      if (booking.renter_id !== request.user.user_id) {
        await client.query('ROLLBACK');
        return reply.status(403).send({ message: 'You can
only pay for your own bookings.' });
      }
      if (booking.status !== 'awaiting_payment') {
        await client.query('ROLLBACK');
        return reply.status(409).send({ message: 'This
booking is not awaiting payment.' });
      }
      if (booking.payment_due_at && new
Date(booking.payment_due_at) < new Date()) {
        await client.query('ROLLBACK');
        return reply.status(409).send({ message: 'The
10-minute reservation hold has expired. Please select
your dates again.' });
      }
      if (new Date(booking.start_at).getTime() <=
Date.now()) {
        await client.query('ROLLBACK');
        return reply.status(409).send({ message: 'The
pick-up time has passed. Choose a later schedule before
paying.' });
      }


      const renterContact = await
getUserContact(request.user.user_id);
      await addBookingEvent(client, {
        bookingId: booking.booking_id,
        actorId: request.user.user_id,
        eventType: 'booking_policies_accepted',
        fromStatus: booking.status,
        toStatus: booking.status,
        details: BOOKING_POLICY_RECORD,
      });
      checkoutBooking = booking;
      if (request.body?.checkout_details ||
request.body?.delivery_mode ||
request.body?.checkout_delivery_mode) {
```

```
      const saved = await saveCheckoutDetails(client,
booking.booking_id, { ...request.user, ...renterContact },
request.body || {});
      if (saved.error) {
        await client.query('ROLLBACK');
        return reply.status(400).send({ message:
saved.error });
      }
      checkoutBooking = { ...booking, ...saved.booking };
      await addBookingEvent(client, {
        bookingId: booking.booking_id,
        actorId: request.user.user_id,
        eventType: 'booking_checkout_details_saved',
        fromStatus: booking.status,
        toStatus: booking.status,
        details: { delivery_mode:
saved.details.deliveryMode },
      });
    }


    const paymentCheck = await client.query("SELECT 1
FROM payments WHERE booking_id = $1 AND status =
'verified'", [booking.booking_id]);
    if (paymentCheck.rows.length > 0) {
      await client.query('ROLLBACK');
      return reply.status(409).send({ message: 'A verified
payment already exists for this booking.' });
    }


    const expectedAmountCentavos =
checkoutAmountCentavos(checkoutBooking);
    const existing = await client.query(
      `SELECT * FROM paymongo_checkout_sessions
       WHERE booking_id = $1 AND status = 'created'
         AND amount_centavos = $2
         AND created_at > NOW() - interval '10 minutes'
       ORDER BY created_at DESC
       LIMIT 1`,
      [booking.booking_id, expectedAmountCentavos]
    );
    if (existing.rows[0]) {
      await client.query('COMMIT');
      return {
```

server/src/routes/paymongoWebhooks.js (lines 1-135)

```
import crypto from 'crypto';
import pool from '../config/db.js';
import { confirmPaidCheckoutSession } from
'../services/paymongoPaymentConfirmation.js';
import { emitAppEvent } from '../services/appEvents.js';
import { queueBookingLifecycleEmail,
sendBookingConfirmationEmail } from
'../services/mailService.js';
const SIGNATURE_TOLERANCE_SECONDS = 15 * 60;


function parseSignatureHeader(header = '') {
  return String(header).split(',').reduce((parts, piece) => {
    const [key, ...value] = piece.split('=');
    if (key) parts[key.trim()] = value.join('=').trim();
    return parts;
  }, {});
}


export function verifyPaymongoSignature({ header,
rawBody, secret, livemode = false, now = Date.now() }) {
  if (!header || !secret ||
secret.includes('your_webhook_secret_here')) return
false;
  const parts = parseSignatureHeader(header);
  if (!parts.t) return false;


  const timestampMs = Number(parts.t) * 1000;
  if (!Number.isFinite(timestampMs)) return false;
  if (Math.abs(now - timestampMs) / 1000 >
SIGNATURE_TOLERANCE_SECONDS) return false;


  const expected = crypto
    .createHmac('sha256', secret)
    .update(`${parts.t}.${rawBody}`)
    .digest('hex');
  const received = livemode ? parts.li : parts.te;
  if (!received || expected.length !== received.length) return
false;


  return crypto.timingSafeEqual(Buffer.from(expected),
Buffer.from(received));
}


async function handlePaidCheckout(client, event) {
  const session = event.data.attributes.data;
  return confirmPaidCheckoutSession(client, session);
}


async function processEvent(event) {
  const eventId = event.data?.id;
  const eventType = event.data?.attributes?.type;
  if (!eventId || !eventType) return;


  const client = await pool.connect();
  let confirmation;
  try {
    await client.query('BEGIN');
    const inserted = await client.query(
      `INSERT INTO paymongo_webhook_events
        (paymongo_event_id, event_type,
checkout_session_id, payload)
       VALUES ($1,$2,$3,$4::jsonb)
       ON CONFLICT (paymongo_event_id) DO NOTHING
       RETURNING event_id`,
      [
        eventId,
        eventType,
        event.data.attributes.data?.id || null,
        JSON.stringify(event),
      ]
    );
    if (!inserted.rows.length) {
      await client.query('COMMIT');
      return;
    }


    if (eventType === 'checkout_session.payment.paid') {
      confirmation = await handlePaidCheckout(client,
event);
    }


    await client.query('COMMIT');
    if (confirmation?.emailEvent === 'confirmed' &&
confirmation.payment) {
      void sendBookingConfirmationEmail({ booking:
confirmation.emailBooking, payment:
confirmation.payment })
        .catch(() => console.error('Booking confirmation
email failed.'));
```

```
    } else if (confirmation?.emailEvent === 'confirmed') {
      queueBookingLifecycleEmail({ bookingId:
confirmation.booking.booking_id, event: 'confirmed' });
    } else if (confirmation?.emailEvent) {
      queueBookingLifecycleEmail({ bookingId:
confirmation.booking.booking_id, event:
confirmation.emailEvent });
    }
    if (eventType === 'checkout_session.payment.paid') {
      emitAppEvent({ type: 'payment_updated', target_role:
'all' });
    }
  } catch (error) {
    await client.query('ROLLBACK');
    throw error;
  } finally {
    client.release();
  }
}


export default async function
paymongoWebhookRoutes(fastify) {
  fastify.removeContentTypeParser('application/json');
  fastify.addContentTypeParser('application/json', {
parseAs: 'buffer' }, (_request, body, done) => {
    done(null, body);
  });


  async function handleWebhook(request, reply, {
expectedLivemode, secretEnv }) {
    const rawBody = Buffer.isBuffer(request.body) ?
request.body : Buffer.from(String(request.body || ''));
    let event;
    try {
      event = JSON.parse(rawBody.toString('utf8'));
    } catch {
      return reply.status(400).send({ message: 'Invalid
JSON payload.' });
    }


    const livemode = event.data?.attributes?.livemode;
    if (livemode !== expectedLivemode) {
      return reply.status(401).send({ message: 'PayMongo
webhook mode does not match this endpoint.' });
    }


    const signature =
request.headers['paymongo-signature'];
    const verified = verifyPaymongoSignature({
      header: signature,
      rawBody: rawBody.toString('utf8'),
      secret: process.env[secretEnv] || '',
      livemode,
    });


    if (!verified) {
      return reply.status(401).send({ message: 'Invalid
PayMongo signature.' });
    }


    await processEvent(event);
    return reply.status(200).send({ message: 'SUCCESS' });
  }


  fastify.post('/paymongo', (request, reply) =>
handleWebhook(request, reply, {
    expectedLivemode: true,
    secretEnv: 'PAYMONGO_WEBHOOK_SECRET',
  }));
  fastify.post('/paymongo-test', (request, reply) =>
handleWebhook(request, reply, {
    expectedLivemode: false,
    secretEnv:
'PAYMONGO_TEST_WEBHOOK_SECRET',
  }));
}
```

server/src/routes/conditionChecks.js (lines 226-310)

```
export default async function
conditionCheckRoutes(fastify) {
  fastify.get('/:id/checks', { preHandler: [authenticate] },
async (request, reply) => {
    const booking = await getBooking(pool,
request.params.id);
    if (!booking) return reply.status(404).send({ message:
'Booking not found.' });
    if (request.user.role === 'renter' && booking.renter_id
!== request.user.user_id) {
      return reply.status(403).send({ message: 'You can only
view checks for your own booking.' });
    }
    const [checks, baseline] = await Promise.all([
      loadChecks(request, request.params.id),
      getOdometerBaseline(request.params.id),
    ]);
    return { checks, baseline };
  });


  fastify.post('/:id/checks', { preHandler: [authenticate,
roleGuard('owner', 'staff')] }, async (request, reply) => {
    if (!request.isMultipart?.()) return
reply.status(415).send({ message: 'Condition evidence
must use multipart form data.' });
    const booking = await getBooking(pool,
request.params.id);
    if (!booking) return reply.status(404).send({ message:
'Booking not found.' });


    const fields = {};
    const media = [];
    let checkType = null;
    try {
      for await (const part of request.parts()) {
        if (!part.file) {
          fields[part.fieldname] = part.value;
          if (part.fieldname === 'check_type') {
            checkType = part.value;
            if (!['pre', 'post'].includes(checkType)) throw
Object.assign(new Error('check_type must be pre or
post.'), { statusCode: 400 });
            const expectedStatus = checkType === 'pre' ?
'confirmed' : 'active';
            if (checkType === 'post' ?
!needsVehicleReturn(booking) : booking.status !==
expectedStatus) {
              throw Object.assign(new Error(`${checkType ===
'pre' ? 'Pre-check' : 'Post-check'} requires a
${expectedStatus} booking.`), { statusCode: 409 });
            }
          }
          continue;
        }
        if (!checkType || !['pre', 'post'].includes(checkType)) {
          part.file.resume();
          throw Object.assign(new Error('check_type must be
sent before evidence files.'), { statusCode: 400 });
```

```
      }
      media.push(await savePart(part, booking.booking_id,
checkType));
    }
  } catch (error) {
    return reply.status(error.statusCode || 500).send({
message: error.message });
  }

  if (!['pre', 'post'].includes(checkType)) return
reply.status(400).send({ message: 'check_type must be
pre or post.' });
  let checklist;
  try { checklist = JSON.parse(fields.checklist || '[]'); }
catch { return reply.status(400).send({ message: 'Checklist
must be valid JSON.' }); }
  if (!Array.isArray(checklist) || checklist.length === 0)
return reply.status(400).send({ message: 'Complete the
guided checklist.' });
  if (checklist.some((item) => !item.code || !['ok',
'damage', 'na'].includes(item.condition))) {
    return reply.status(400).send({ message: 'Every
checklist item needs a valid condition.' });
  }
  if (checklist.some((item) => item.condition === 'damage'
&& !String(item.notes || '').trim())) {
    return reply.status(400).send({ message: 'Damage
checklist items require notes.' });
  }

  const requiredAngles = CAR_ANGLES;
  const uploadedAngles = new Set(media.filter((item) =>
item.media_type === 'photo').map((item) =>
item.angle_label));
  const uploadedVideos = media.filter((item) =>
item.media_type === 'video');
  const missing = requiredAngles.filter((angle) =>
!uploadedAngles.has(angle));
  if (missing.length) {
    return reply.status(400).send({ message: `Missing
required checkpoint photos: ${missing.join(', ')}.` });
  }
  if (uploadedVideos.length > 3) {
    return reply.status(400).send({ message: 'You may
upload up to 3 walkaround videos.' });
  }

  const odoNum = Number(fields.odometer_km);
  const baseline = await
getOdometerBaseline(booking.booking_id);
  const startOdo = baseline?.handover_odometer != null
? baseline.handover_odometer : (booking.odometer_start
? Number(booking.odometer_start) : null);

  const odoValidation = validateOdometerCheck({
    checkType,
    odometerKm: fields.odometer_km,
    baseDistance: baseline?.base_distance,
    startOdo,
    discrepancyReason:
fields.odometer_discrepancy_reason,
  });

  if (!odoValidation.valid) {
    return reply.status(400).send({
      error: odoValidation.requiresReason ? 'Odometer
Discrepancy' : 'Validation Error',
      message: odoValidation.error,
      requires_reason: !!odoValidation.requiresReason,
```

server/src/routes/telemetry.js (lines 210-320)

```
fastify.post('/tracking-sessions/start', {
  preHandler: [authenticate, roleGuard('owner', 'staff')],
  schema: {
    body: {
      type: 'object', required: ['vehicle_id'],
additionalProperties: false,
      properties: { vehicle_id: { type: 'integer', minimum: 1 }
},
    },
  },
}, async (request, reply) => {
  const client = await pool.connect();
  try {
    await client.query('BEGIN');
    const vehicleResult = await client.query(
      `SELECT v.vehicle_id, v.make, v.model,
v.plate_number,
              d.device_id, d.device_serial, d.is_active
       FROM vehicles v
       JOIN iot_devices d ON d.device_id = v.device_id
       WHERE v.vehicle_id = $1
       FOR UPDATE OF v, d`,
      [request.body.vehicle_id]
    );
    const vehicle = vehicleResult.rows[0];
    if (!vehicle) throw Object.assign(new Error('Assign an
active tracker to this vehicle before starting tracking.'), {
statusCode: 409 });
    if (!vehicle.is_active) throw Object.assign(new
Error('Activate the assigned tracker before starting
tracking.'), { statusCode: 409 });

    const activeResult = await client.query(
      'SELECT * FROM tracking_sessions WHERE
vehicle_id = $1 AND ended_at IS NULL FOR UPDATE',
      [vehicle.vehicle_id]
    );
    if (activeResult.rows[0]) {
      await client.query('COMMIT');
      return { session: activeResult.rows[0], started: false };
    }

    const bookingResult = await client.query(
      `SELECT booking_id FROM bookings
       WHERE vehicle_id = $1 AND status = 'active'
       ORDER BY updated_at DESC LIMIT 1`,
      [vehicle.vehicle_id]
    );
    const inserted = await client.query(
      `INSERT INTO tracking_sessions (vehicle_id,
device_id, booking_id, started_by)
       VALUES ($1, $2, $3, $4)
       RETURNING *`,
      [vehicle.vehicle_id, vehicle.device_id,
bookingResult.rows[0]?.booking_id || null,
request.user.user_id]
    );
    const session = inserted.rows[0];
    await addAuditLog(client, request, {
      actorId: request.user.user_id,
      actorRole: request.user.role,
      action: 'tracking_session_started',
      entityType: 'tracking_session',
      entityId: session.session_id,
      newValue: { vehicle_id: vehicle.vehicle_id, device_id:
vehicle.device_id, booking_id: session.booking_id },
```

```
    });
    await client.query('COMMIT');
    return reply.status(201).send({ session, started: true });
   } catch (error) {
    await client.query('ROLLBACK');
    if (error.statusCode) return
reply.status(error.statusCode).send({ message:
error.message });
    throw error;
   } finally {
    client.release();
   }
  });


  fastify.post('/tracking-sessions/:id/stop', {
   preHandler: [authenticate, roleGuard('owner', 'staff')],
   schema: {
    params: { type: 'object', required: ['id'], properties: { id:
{ type: 'integer', minimum: 1 } } },
   },
  }, async (request, reply) => {
   const client = await pool.connect();
   try {
    await client.query('BEGIN');
    const sessionResult = await client.query(
     'SELECT * FROM tracking_sessions WHERE
session_id = $1 FOR UPDATE',
     [request.params.id]
    );
    const session = sessionResult.rows[0];
    if (!session) throw Object.assign(new Error('Tracking
session not found.'), { statusCode: 404 });
    if (session.ended_at) {
     await client.query('COMMIT');
     return { session, stopped: false };
    }
    const updated = await client.query(
     `UPDATE tracking_sessions
      SET ended_at = NOW(), ended_by = $2
      WHERE session_id = $1
      RETURNING *`,
     [session.session_id, request.user.user_id]
    );
    await addAuditLog(client, request, {
     actorId: request.user.user_id,
     actorRole: request.user.role,
     action: 'tracking_session_stopped',
     entityType: 'tracking_session',
     entityId: session.session_id,
     oldValue: { ended_at: null },
     newValue: { ended_at: updated.rows[0].ended_at },
    });
    await client.query('COMMIT');
    return { session: updated.rows[0], stopped: true };
   } catch (error) {
    await client.query('ROLLBACK');
    if (error.statusCode) return
reply.status(error.statusCode).send({ message:
error.message });
    throw error;
   } finally {
    client.release();
   }
  });
```

server/src/services/telemetryService.js (lines 152-230)

```
export async function ingestTelemetry(payload, topic =
null) {
  if (!validMessage(payload)) throw Object.assign(new
Error('Invalid telemetry payload.'), { statusCode: 400 });
  const deviceResult = await pool.query(
   `SELECT d.*, v.vehicle_id, v.make, v.model
    FROM iot_devices d
    LEFT JOIN vehicles v ON v.device_id = d.device_id
    WHERE d.device_serial = $1`,
   [payload.device_serial]
  );
  const device = deviceResult.rows[0];
  if (!device) throw Object.assign(new Error('Unknown or
unassigned telemetry device.'), { statusCode: 404 });


  let speedKmh = resolveSpeedKmh(payload.speed_kmh,
payload, null);
  if (speedKmh === null) {
   const previousResult = await pool.query(
    `SELECT latitude, longitude, recorded_at
     FROM telemetry
     WHERE device_id = $1 AND recorded_at < $2
     ORDER BY recorded_at DESC
     LIMIT 1`,
    [device.device_id, new Date(payload.recorded_at)]
   );
   speedKmh = resolveSpeedKmh(payload.speed_kmh,
payload, previousResult.rows[0]);
  }


  const result = await pool.query(
   `INSERT INTO telemetry
    (vehicle_id, device_id, message_id, latitude, longitude,
speed_kmh, recorded_at, is_backfilled,
tracking_session_id)
    VALUES ($1,$2,$3,$4,$5,$6,$7,$8,
     (SELECT session_id
      FROM tracking_sessions
      WHERE vehicle_id = $1 AND device_id = $2
       AND started_at <= $7
       AND (ended_at IS NULL OR ended_at >= $7)
      ORDER BY started_at DESC
      LIMIT 1))
    ON CONFLICT (device_id, message_id) WHERE
device_id IS NOT NULL AND message_id IS NOT NULL
DO NOTHING
    RETURNING *`,
   [device.vehicle_id, device.device_id,
String(payload.message_id), Number(payload.latitude),
Number(payload.longitude),
    speedKmh, new Date(payload.recorded_at),
Boolean(payload.is_backfilled)]
  );
  if (!result.rows[0]) return { duplicate: true };


  await pool.query('UPDATE iot_devices SET last_seen_at
= NOW() WHERE device_id = $1', [device.device_id]);
  const settingsResult = await pool.query('SELECT *
FROM system_settings ORDER BY setting_id LIMIT 1');
  const settings = settingsResult.rows[0];
  if (settings) {
   const evaluation =
evaluateGeofence(Number(payload.latitude),
Number(payload.longitude), settings);
   const outside = evaluation.outside;


   if (outside && !device.currently_outside_geofence &&
settings.geofence_alert_on_exit !== false) {
    const activeBooking = await pool.query("SELECT
booking_id FROM bookings WHERE vehicle_id = $1 AND
```

```
status = 'active' ORDER BY updated_at DESC LIMIT 1",
[device.vehicle_id]);
      await pool.query(
        `INSERT INTO geofence_alerts (vehicle_id,
booking_id, latitude, longitude, distance_km)
         VALUES ($1,$2,$3,$4,$5)`,
        [device.vehicle_id,
activeBooking.rows[0]?.booking_id || null, payload.latitude,
payload.longitude, evaluation.distance]
      );
      await pool.query(
        `INSERT INTO notifications (user_id, title, message)
         SELECT user_id, 'Geofence alert', $1 FROM users
WHERE role IN ('owner','staff') AND is_active = true`,
        [`${device.make || 'Vehicle'} ${device.model || ''}
geofence alert: ${evaluation.reason}`]
      );
    } else if (!outside &&
device.currently_outside_geofence) {
      await pool.query(
        `UPDATE geofence_alerts SET resolved_at = NOW()
         WHERE vehicle_id = $1 AND resolved_at IS NULL`,
        [device.vehicle_id]
      );
    }
    if (outside !== device.currently_outside_geofence) {
      await pool.query('UPDATE iot_devices SET
currently_outside_geofence = $1 WHERE device_id = $2',
[outside, device.device_id]);
    }
  }


  const event = { ...result.rows[0], make: device.make,
model: device.model, last_seen_at: new
Date().toISOString() };
  telemetryEvents.emit('location', event);
  return event;
}
```

server/src/routes/operations.js (lines 134-230)

```
  fastify.patch('/refunds/:id', { preHandler: [authenticate,
roleGuard('owner')] }, async (request, reply) => {
    const { decision, refund_amount = null, reason = '' } =
request.body || {};
    if (!['approved', 'denied'].includes(decision)) {
      return reply.status(400).send({ message: 'Decision
must be approved or denied.' });
    }

    const result = await withTransaction(async (client) => {
      const updateResult = await client.query(
        `UPDATE refund_requests
         SET status = $1,
             refund_amount = COALESCE($2,
refund_amount),
             reviewed_by = $3,
             reviewed_at = NOW()
         WHERE request_id = $4 AND status = 'pending'
         RETURNING *`,
        [decision, refund_amount, request.user.user_id,
request.params.id]
      );

      if (!updateResult.rows[0]) return null;

      await addAuditEvent(client, {
        actorId: request.user.user_id,
        eventType: 'refund_decision',
        entityType: 'refund_request',
        entityId: updateResult.rows[0].request_id,
        summary: `${decision === 'approved' ? 'Approved' :
'Denied'} refund request`,
        details: {
          refund_amount:
updateResult.rows[0].refund_amount,
          reason: String(reason).trim() || null,
        },
      });

      const bookingResult = await client.query(
        'SELECT renter_id FROM bookings WHERE
booking_id = $1',
        [updateResult.rows[0].booking_id]
      );
      return { refund: updateResult.rows[0], renterId:
bookingResult.rows[0]?.renter_id };
    });

    if (!result) {
      return reply.status(409).send({ message: 'Refund
request is unavailable or already decided.' });
    }

    const normalizedRefundAmount =
result.refund.refund_amount;
    const renterId = result.renterId;
    if (renterId) {
      const amountNote = normalizedRefundAmount != null
        ? ` for PHP
${Number(normalizedRefundAmount).toLocaleString('en-
PH')}`
        : '';
      await notifyUser(pool, renterId, {
        title: decision === 'approved' ? 'Calamity refund
approved' : 'Calamity refund declined',
        message: decision === 'approved'
          ? `Your calamity refund request was
approved${amountNote}. Refunds are processed
manually outside the app.`
          : (String(reason).trim() || 'Your calamity refund
request was declined. Contact 1816 support for details.'),
        linkPath:
`/renter/bookings/${result.refund.booking_id}`,
      });
    }
    queueBookingLifecycleEmail({ bookingId:
result.refund.booking_id, event: decision === 'approved' ?
'refund_approved' : 'refund_denied' });

    return { refund: result.refund };
  });

  fastify.post('/refunds/:id/settle', { preHandler:
[authenticate, roleGuard('owner')] }, async (request, reply)
=> {
    const { settlement_reference, settlement_notes = '',
refund_amount = null } = request.body || {};
    if (!settlement_reference?.trim()) {
      return reply.status(400).send({ message: 'A
transaction settlement reference number is required.' });
    }

    const result = await withTransaction(async (client) => {
      const existing = await client.query(
```

```
      'SELECT * FROM refund_requests WHERE
request_id = $1 FOR UPDATE',
      [request.params.id]
    );
    const reqRow = existing.rows[0];
    if (!reqRow) return { error: 'NOT_FOUND', message:
'Refund request not found.' };
    if (reqRow.status !== 'approved') {
      return { error: 'INVALID_STATUS', message: 'Only
approved refund requests can be marked as settled.' };
    }

    const updateResult = await client.query(
      `UPDATE refund_requests
       SET status = 'settled',
           settled_by = $1,
           settled_at = NOW(),
           settlement_reference = $2,
           settlement_notes = $3,
           refund_amount = COALESCE($4,
refund_amount)
       WHERE request_id = $5
       RETURNING *`,
      [request.user.user_id, settlement_reference.trim(),
settlement_notes?.trim() || null, refund_amount,
request.params.id]
    );

    await addAuditEvent(client, {
      actorId: request.user.user_id,
      eventType: 'refund_settled',
      entityType: 'refund_request',
      entityId: updateResult.rows[0].request_id,
```

server/src/routes/operations.js (lines 542-610)

```
  fastify.get('/audit', { preHandler: [authenticate,
roleGuard('owner')] }, async (request) => {
    const page = Math.max(1, parseInt(request.query.page,
10) || 1);
    const limit = Math.min(100, Math.max(1,
parseInt(request.query.limit, 10) || 20));
    const offset = (page - 1) * limit;
    const { action, entity_type, from, to, search } =
request.query;

    const conditions = [];
    const params = [];

    // This endpoint is owner-only. The owner needs to see
staff, renter,
    // webhook, and system actions as well as actions they
performed directly.

    if (action) {
      conditions.push(`al.action = $${params.length + 1}`);
      params.push(action);
    }
    if (entity_type) {
      conditions.push(`al.entity_type = $${params.length +
1}`);
      params.push(entity_type);
    }
    if (from) {
      conditions.push(`al.created_at >= $${params.length +
1}::timestamptz`);
      params.push(from);
    }
    if (to) {
      conditions.push(`al.created_at <= $${params.length +
1}::timestamptz`);
      params.push(to);
    }
    if (search) {
      conditions.push(`(al.action ILIKE $${params.length +
1} OR al.entity_type ILIKE $${params.length + 1} OR
al.entity_id ILIKE $${params.length + 1} OR
al.new_value::text ILIKE $${params.length + 1})`);
      params.push(`%${search}%`);
    }

    const whereClause = conditions.length ? `WHERE
${conditions.join(' AND ')}` : '';

    const countRes = await query(
      `SELECT count(*)::int AS total FROM audit_logs al
${whereClause}`,
      params
    );
    const total = countRes.rows[0]?.total || 0;

    const logsRes = await query(
      `SELECT al.id, al.actor_id, al.actor_role, al.action,
al.entity_type, al.entity_id,
              al.old_value, al.new_value, al.ip_address,
al.user_agent, al.created_at,
              COALESCE(u.full_name, CASE WHEN
al.actor_id IS NULL THEN INITCAP(al.actor_role) ELSE
'Deleted user' END) AS actor_name
       FROM audit_logs al
       LEFT JOIN users u ON u.user_id = al.actor_id
       ${whereClause}
       ORDER BY al.created_at DESC
       LIMIT $${params.length + 1} OFFSET
$${params.length + 2}`,
      [...params, limit, offset]
    );

    return {
      logs: logsRes.rows,
      total,
      page,
      limit,
      totalPages: Math.ceil(total / limit) || 1,
      // Backward-compatible events structure
      events: logsRes.rows.map((l) => ({
        id: l.id,
        event_type: l.action,
        entity_type: l.entity_type,
        entity_id: l.entity_id,
        summary: l.new_value?.summary || `${l.action} on
${l.entity_type} ${l.entity_id || ''}`.trim(),
        details: l.new_value,
        created_at: l.created_at,
        actor_name: l.actor_name,
```

src/pages/Dashboard/Owner/OwnerDashboard.jsx (lines 30-125)

```
export function OwnerDashboard() {
  const { user, logout } = useAuth();

  const [view, setView] = useState(() => {
    const requested = new
URLSearchParams(window.location.search).get('view');
    return ['bookings', 'tracking',
'garage'].includes(requested) ? requested : 'overview';
  });
```

```
  const [focusBookingId, setFocusBookingId] = useState(() =>
new
URLSearchParams(window.location.search).get('booking')
);
  const [mobileNav, setMobileNav] = useState(false);
  const [isNavigating, setIsNavigating] = useState(false);
  const [paletteOpen, setPaletteOpen] = useState(false);


  const { vehicles, bookings, telemetry, requirements,
setSettings, settings,
    pendingDamage, pendingChecks, pendingRefunds,
pendingVerif, dataError, loadAllData } =
useManagementData(view, false);


  // Unread messages count for badges
  const [unreadTotal, setUnreadTotal] = useState(0);


  // Notifications dropdown state
  const [notifOpen, setNotifOpen] = useState(false);
  const [notifications, setNotifications] = useState([]);
  const notifRef = useRef(null);


  const unreadNotifCount = useMemo(
    () => notifications.filter((n) => n && !n.is_read).length,
    [notifications],
  );


  // Load unread count for badges
  const loadConversations = async () => {
    try {
      const res = await
authFetch('/api/messages/conversations');
      if (res?.conversations) {
        const unread = res.conversations.reduce((acc, c) =>
acc + (Number(c.unread_count) || 0), 0);
        setUnreadTotal(unread);
      }
    } catch (err) {
      console.warn('Could not fetch conversations:', err);
    }
  };


  // Load notifications
  const loadNotifications = async () => {
    try {
      const res = await authFetch('/notifications');
      if (res?.notifications) {
        setNotifications(res.notifications);
      } else {
        setNotifications([]);
      }
    } catch (err) {
      console.warn('Could not fetch notifications:', err);
    }
  };


  useAppEvents();


  useEffect(() => {
    let timer;
    const refreshBadges = () => {
      if (document.visibilityState === 'hidden') return;
      loadConversations();
      loadNotifications();
    };
    const schedule = () => { clearTimeout(timer); timer =
setTimeout(refreshBadges, 200); };
    refreshBadges();
    const interval = setInterval(refreshBadges, 15_000);
    window.addEventListener('1816:app-event', schedule);
    document.addEventListener('visibilitychange',
refreshBadges);
    return () => {
      clearInterval(interval); clearTimeout(timer);
      window.removeEventListener('1816:app-event',
schedule);
      document.removeEventListener('visibilitychange',
refreshBadges);
    };
  }, []);


  // ⌘K / Ctrl+K keyboard shortcut
  useEffect(() => {
    const onKey = (e) => {
      if ((e.metaKey || e.ctrlKey) && e.key.toLowerCase()
=== 'k') {
        e.preventDefault();
        setPaletteOpen((prev) => !prev);
        setNotifOpen(false);
      }
      if (e.key === 'Escape') {
        setPaletteOpen(false);
        setNotifOpen(false);
      }
    };
    window.addEventListener('keydown', onKey);
    return () => window.removeEventListener('keydown',
onKey);
  }, []);


  // Click outside listener for notifications
  useEffect(() => {
    function handleClickOutside(event) {
```

src/pages/Dashboard/Staff/StaffDashboard.jsx (lines 25-115)

```
export function StaffDashboard() {
  const { user, logout } = useAuth();


  const [view, setView] = useState(() => {
    const requested = new
URLSearchParams(window.location.search).get('view');
    return ['bookings', 'tracking', 'checks',
'live-board'].includes(requested) ? requested : 'overview';
  });
  const [focusBookingId, setFocusBookingId] = useState(()
=> new
URLSearchParams(window.location.search).get('booking')
);
  const [mobileNav, setMobileNav] = useState(false);
  const [isNavigating, setIsNavigating] = useState(false);


  const { vehicles, bookings, telemetry, requirements,
refunds, settings,
    pendingDamage, pendingChecks, pendingRefunds,
pendingVerif, summary, dataError, loadAllData } =
useManagementData(view, true);


  // Global search state
  const [searchQuery, setSearchQuery] = useState('');
  const [searchResults, setSearchResults] = useState([]);
  const [isSearching, setIsSearching] = useState(false);
```

```
  const [showSearchResults, setShowSearchResults] =
useState(false);
  const searchRef = useRef(null);

  // Unread messages count for badges
  const [unreadTotal, setUnreadTotal] = useState(0);

  // Notifications dropdown state
  const [notifOpen, setNotifOpen] = useState(false);
  const [notifications, setNotifications] = useState([]);
  const notifRef = useRef(null);

  const unreadNotifCount = useMemo(
    () => notifications.filter((n) => n && !n.is_read).length,
    [notifications]
  );

  // Load unread count for badges
  const loadConversations = async () => {
    try {
      const res = await
authFetch('/api/messages/conversations');
      if (res?.conversations) {
        const unread = res.conversations.reduce((acc, c) =>
acc + (Number(c.unread_count) || 0), 0);
        setUnreadTotal(unread);
      }
    } catch (err) {
      console.warn('Could not fetch conversations:', err);
    }
  };

  // Load notifications
  const loadNotifications = async () => {
    try {
      const res = await authFetch('/notifications');
      if (res?.notifications) {
        setNotifications(res.notifications);
      } else {
        setNotifications([]);
      }
    } catch (err) {
      console.warn('Could not fetch notifications:', err);
    }
  };

  useAppEvents();

  useEffect(() => {
    let timer;
    const refreshBadges = () => {
      if (document.visibilityState === 'hidden') return;
      loadConversations();
      loadNotifications();
    };
    const schedule = () => { clearTimeout(timer); timer =
setTimeout(refreshBadges, 200); };
    refreshBadges();
    const interval = setInterval(refreshBadges, 15_000);
    window.addEventListener('1816:app-event', schedule);
    document.addEventListener('visibilitychange',
refreshBadges);
    return () => {
      clearInterval(interval); clearTimeout(timer);
      window.removeEventListener('1816:app-event',
schedule);
      document.removeEventListener('visibilitychange',
refreshBadges);
    };
  }, []);

  // Click outside listener for search and notifications
  useEffect(() => {
    function handleClickOutside(event) {
      if (searchRef.current &&
!searchRef.current.contains(event.target)) {
        setShowSearchResults(false);
      }
      if (notifRef.current &&
!notifRef.current.contains(event.target)) {
        setNotifOpen(false);
      }
```

src/pages/Dashboard/Owner/views/LiveTrackingView.jsx (lines 180-245)

```
  // Default to first active unit if none selected
  const activeUnits = trackedUnits;
  const sel = selectedVehicleId ??
activeUnits[0]?.vehicle_id ?? null;
  const selectedUnit = activeUnits.find((u) => u.vehicle_id
=== sel) || null;

  const loadRouteHistory = useCallback(async (vehicleId,
sessionId = null, { silent = false } = {}) => {
    if (!vehicleId) return;
    const requestId = routeRequestRef.current + 1;
    routeRequestRef.current = requestId;
    if (!silent) setRouteState({ loading: true, error: '' });
    try {
      const query = sessionId ?
`?session_id=${encodeURIComponent(sessionId)}` : '';
      const response = await
authFetch(`/telemetry/vehicles/${vehicleId}/history${query}
`);
      if (requestId !== routeRequestRef.current) return;
      const history = Array.isArray(response?.history) ?
response.history : response?.telemetry;
      if (Array.isArray(history)) {
        setRouteHistory(history);
        setHistoryMeta({
          pointCount: Number(response?.point_count ??
history.length),
          sampled: Boolean(response?.sampled),
        });
      }
      setRouteState({ loading: false, error: '' });
    } catch (error) {
      if (requestId !== routeRequestRef.current) return;
      if (!silent) setRouteHistory([]);
      setRouteState({ loading: false, error: error.message ||
'Could not load route history.' });
    }
  }, []);

  const loadTrackingSessions = useCallback(async
(vehicleId, { cursor = null, append = false } = {}) => {
    if (!vehicleId) return;
    const requestVehicle = String(vehicleId);
    const requestId = sessionListRequestRef.current + 1;
    sessionListRequestRef.current = requestId;
    sessionRequestVehicleRef.current = requestVehicle;
    setSessionListState((current) => ({
      loading: append ? current.loading : true,
      loadingMore: append,
      error: '',
```

```
    }));
    try {
      const params = new URLSearchParams({ limit: '20' });
      if (cursor) params.set('cursor', String(cursor));
      const response = await
authFetch(`/telemetry/vehicles/${vehicleId}/tracking-sessio
ns?${params.toString()}`);
      if (sessionRequestVehicleRef.current !==
requestVehicle || sessionListRequestRef.current !==
requestId) return;
      const sessions = Array.isArray(response?.sessions) ?
response.sessions : [];
      setSessionHistory((current) => {
        if (!append) return sessions;
        const known = new Set(current.map((session) =>
String(session.session_id)));
        return [...current, ...sessions.filter((session) =>
!known.has(String(session.session_id)))];
      });
      setSessionPagination({
        hasMore:
Boolean(response?.pagination?.has_more),
        nextCursor: response?.pagination?.next_cursor ||
null,
      });
      setSessionListState({ loading: false, loadingMore:
false, error: '' });
    } catch (error) {
      if (sessionRequestVehicleRef.current !==
requestVehicle || sessionListRequestRef.current !==
requestId) return;
      setSessionListState({
        loading: false,
        loadingMore: false,
        error: error.message || 'Could not load route history.',
      });
    }
  }, []);
```

APPENDIX G : Plagiarism Check Results

John Edrian Martinez

Web-Based-Car-Rental-Management-System-with-Real-Time-IoT-Bc (1).pdf

Submission of Final Documentation

BSIT Capstone Project Advisee

Pamantasan ng Lungsod ng Pasig

Document Details

Submission ID
trn:oid:::1:3692992365

Submission Date
Oct 28, 2026, 3:22 AM GMT+8

Download Date
Oct 28, 2026, 6:53 AM GMT+8

File Name
Web-Based-Car-Rental-Management-System-with-Real-Time-IoT-Based-GPS-Tracking-for-1816-Ca....pdf

File Size
6.0 MB

100 Pages

22,401 Words

122,246 Characters

29/10/24

10% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- **90** Not Cited or Quoted **7%**
  Matches with neither in-text citation nor quotation marks
- **12** Missing Quotations **1%**
  Matches that are still very similar to source material
- **12** Missing Citation **2%**
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted **0%**
  Matches with in-text citation present, but no quotation marks

Top Sources

8% Internet sources
6% Publications
8% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

10/29/24

APPENDIX H : AI Generated Content Check Results

John Edrian Martinez

Web-Based-Car-Rental-Management-System-with-Real-Time-IoT-Bc (1).pdf

Submission of Final Documentation
BSIT Capstone Project Advisee
Pamantasan ng Lungsod ng Pasig

Document Details

Submission ID
trn:oid:::1:3692992365

Submission Date
Oct 28, 2026, 3:22 AM GMT+8

Download Date
Oct 28, 2026, 6:53 AM GMT+8

File Name
Web-Based-Car-Rental-Management-System-with-Real-Time-IoT-Based-GPS-Tracking-for-1816-Ca....pdf

File Size
6.0 MB

100 Pages

22,401 Words

122,246 Characters

10/29/24

turnitin Page 1 of 102 - Cover Page Submission ID trn:oid:::1:3692992365

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

10/29/24

APPENDIX I : Thesis and Capstone Project Agreement (AI Disclosure)

PAMANTASAN NG LUNGSOD NG PASIG
College of Computer Studies
Alkalde Jose St. Kapasigan Pasig City, Philippines 1600

Thesis and Capstone Project Agreement

This Agreement outlines the responsibilities, commitments, and academic integrity expectations between the **Student Researchers** and the **College of Computer Studies** (Through the group's Adviser) in relation to the conduct, completion, and submission of the Thesis or Capstone Project.

1. Purpose

This Agreement ensures that all student researchers understand the academic standards, deadlines, integrity policies, and consequences associated with their Thesis or Capstone Project.

2. Parties Involved

- **Student Researchers**: All officially **enrolled** students completing the Thesis/Capstone subject.
- **Adviser**: Faculty members assigned to guide, evaluate, and monitor the progress of the project.

3. Responsibilities of the Student Researchers

1. Conduct all project-related tasks honestly, independently, and in compliance with institutional policies.
2. Attend consultations, defense schedules, and all required activities.
3. Submit deliverables (e.g., chapters, documents, source code, system prototype, deployment materials) on or before the prescribed deadlines.
4. Ensure that all work—programming, system development, documentation writing, data gathering, analysis, and deployment—is completed by the members of the group.

4. Submission and Deadline Policy

- All project deliverables must be submitted on or before the deadlines set by the College Dean.
- **Late Submission Penalty:**
    - A **deduction of 3 points** from the **subject grade** will be imposed for any late submission (no more than 5 days) of required outputs, regardless of reason, unless officially excused or approved by the Dean.

  Any submission made after the 5-day late submission deadline will be deemed unacceptable and result in failure.

5. Academic Integrity and Prohibited Actions

To preserve academic honesty and uphold research standards, the following additional guidelines apply:

5.1 Similarity Index Requirement

- All written components of the Thesis/Capstone Project (Chapters 1–4, documentation, technical papers, appendices) must comply with the **generally accepted similarity threshold** based on institutional policy.
- When checked using tools such as **Turnitin**, the similarity report must fall within the acceptable limit **not exceeding 20% overall similarity** - excluding preliminaries, references, tables, and properly quoted material.
- Students must revise and resubmit documents that exceed allowable similarity limits.

5.2 AI-Generated Content Policy

- The use of AI tools (e.g., ChatGPT, Gemini, Copilot, AI paraphrasers, code generators) **must be fully disclosed**. * *The Student Artificial Intelligence (AI) Disclosure Form serves as an official appendix to this agreement.*
- Any AI-generated text, code, images, or analysis must comply with institutional guidelines and must **not replace the students' own work or critical thinking**.
- AI-generated content found to be excessive—particularly in excess of 20%— may be treated as an academic integrity violation, subject to penalty based on the Student Manual.
- The College reserves the right to require:
    - AI usage logs or statements of use
    - Revisions or oral validation (defense-style questioning) to verify authorship

5.3 Prohibited Actions

To preserve the integrity of academic research and system development, the following actions are strictly prohibited:

1. **Recruiting or paying outside individuals** (freelancers, professionals, alumni, non-members, external developers) to perform any of the following:
    - Programming or system development
    - Documentation writing (Chapters 1–4)

- Data analysis
- System deployment or uploading to hosting platforms/servers
- Any substantial part of the project

2. Submitting work that is not originally produced by the group.
3. Misrepresenting externally created work as the group's own.

Penalty for Violations:

- Upon investigation and confirmation of violation, the group will automatically **receive a failing grade (5.0)** in the Thesis/Capstone subject.
- All members will be **required to re-enroll** the subject in the next available term. If the subject is not offered during the immediate succeeding term, students may opt to file for a **special schedule**, which may include additional fees as determined by the school.
- Additional disciplinary actions may be imposed based on institutional policies.

6. Adviser, Coordinator, and Dean Responsibilities

1. The **Adviser** must first review, approve, and sign the final Thesis/Capstone document before it is forwarded to the Thesis/Capstone Coordinator.
2. The **Thesis/Capstone Coordinator** will review formatting, compliance with guidelines, and completeness before signing.
3. The **College Dean** will **only sign the final document** once both the Adviser and the Coordinator have signed and all requirements have been fully met.
4. The Dean reserves the right to reject or return any document that does not comply with standards, including formatting, completeness, or integrity requirements.

7. Formatting and Submission Compliance

To maintain academic and professional quality, all submitted documents must strictly follow the official Thesis/Capstone formatting guidelines, including:

- Font style
- Font size
- Line spacing and margins
- Section structure, pagination, and labeling
- Required templates and formatting standards issued by the College

PAMANTASAN NG LUNGSOD NG PASIG
College of Computer Studies
Alkalde Jose St. Kapasigan Pasig City, Philippines 1600

Failure to comply with these formatting guidelines may result in the document being **rejected**, **returned for correction**, or marked as **non-acceptable** for that submission cycle.

8. Revision Responsibilities and Deadlines

1. All **revisions—major or minor, including updates to both the developed system and the complete written document—must be completed within the deadlines** set by the College Dean, with a minimum requirement that all revisions be submitted within one (1) week after the defense.

2. Failure to submit revisions on time will result in the grade deductions or penalties specified in this Agreement, with deductions applied for every instance of late submission. For the purposes of this Agreement, a submission is considered 'late' **only if it is submitted within two (2) days after the deadline**.

3. Revisions submitted beyond the deadline may not be accepted, and the corresponding deliverable may be marked as late or incomplete.

9. Acknowledgment and Agreement

By signing this Agreement, the Student Researchers and Adviser affirm that they:

- Understand all policies stated above;
- Agree to comply with deadlines, academic integrity rules, and project requirements;
- Accept the corresponding penalties should they violate any provisions of this Agreement.

PAMANTASAN NG LUNGSOD NG PASIG
College of Computer Studies
Alkalde Jose St. Kapasigan Pasig City, Philippines 1600

APPENDIX: STUDENT ARTIFICIAL INTELLIGENCE (AI) DISCLOSURE FORM

(In compliance with CHED Memorandum Order No. 21, s. 2026)

INSTRUCTIONS TO STUDENTS:

In accordance with CHED Memorandum Order No. 21, s. 2026 and institutional policy, student researchers must transparently disclose any use of Artificial Intelligence (AI) tools, models, or systems utilized during their thesis or capstone project.

- **Mandatory Requirement:** Complete this form and attach it to your final manuscript.
- **Core Principle:** AI tools must augment human intelligence and must not replace independent scholarly contribution, critical thinking, or intellectual rigor. Students retain full accountability for the accuracy and originality of their work.

PART I: PROJECT & STUDENT INFORMATION

- **Title of Thesis / Capstone Project:** Web-Based Car Rental Management System with Real-Time IoT-Based GPS Tracking for 1816 Car Rental Co.
- **Degree Program:** Bachelor of Science in Information Technology
- **Name of Student Researchers:**
  1. Junio, Carl AJ G. (ID No: 23-00220)
  2. Martinez, John Edrian V. (ID No: 23-00259)
  3. Navarro, Rex S. (ID No: 23-00186)
- **Adviser Name:** Federico G. Nueva, MT

PART II: AI TOOL UTILIZATION CHECKLIST

Check all applicable boxes indicating how AI was utilized across your project lifecycle:

- [x] **Ideation & Topic Refinement:** Brainstorming research questions, narrowing scope, or scoping objectives.
- [x] **Literature Review & Organization:** Searching, summarizing, or formatting academic literature.
- [x] **System Development / Coding:** Utilizing AI coding assistants (e.g., GitHub Copilot, ChatGPT) for code generation, debugging, or optimization.
- [x] **Data Gathering & Analysis:** Assisting in data processing, simulation setups, or quantitative/qualitative analysis.
- [x] **Drafting & Language Editing:** Assisting in grammar checking, translation, or structural formatting of documentation.
- [x] Others (Please specify): Testing, performance checks, deployment assistance, and integration of pretrained AI features.

PAMANTASAN NG LUNGSOD NG PASIG
College of Computer Studies
Alkalde Jose St. Kapasigan Pasig City, Philippines 1600

PART III: DETAILED LOG OF AI USAGE

Provide a detailed breakdown of how specific AI tools were used.

Research / Development Phase	AI Tool(s) Used	Specific Prompt / Purpose / Function	Extent of Human Review & Validation
1. Proposal / Ideation	ChatGPT / Codex	Clarified rental scope, user roles, workflow requirements and adviser feedback.	Researchers supplied client/adviser requirements and requested changes; confirm final scope before signing.
2. Literature Review	ChatGPT / Codex	Organized Chapter 2 material and assisted reference and literature formatting.	Researchers provided revised text. Original sources and citation accuracy was verified by the researchers.
3. Coding / System Dev.	ChatGPT / Codex	Generated and revised React/Fastify code; debugged booking, payment, tracking and inspection flows.	Researchers set requirements and requested fixes. Researchers verified the tests as the AI works through the development.
4. Data Analysis / AI Features	ChatGPT / Codex; CarDD YOLOv8s; Tesseract OCR; MediaPipe	Prepared damage-evaluation scripts, confusion matrices and a validation workbook; assisted OCR and selfie-check integration.	374 test images were cross-checked automatically with the researchers' guidance and was approved for testing of confusion matrix.
5. Writing & Editing	ChatGPT / Codex	Drafted and revised manuscript sections, including Chapter 4; edited language, figures, tables, pagination and signature layout.	Researchers supplied material and requested corrections.
6. Testing / Deployment	ChatGPT / Codex	Prepared synthetic test data, performance checks and Railway deployment configuration; assisted deployment and troubleshooting.	Automated and selected live checks were recorded. Synthetic records were identified separately from real rental activity as advised by the panelists.

PAMANTASAN NG LUNGSOD NG PASIG
College of Computer Studies
Alkalde Jose St. Kapasigan Pasig City, Philippines 1600

PART IV: STUDENT CERTIFICATION & RESPONSIBILITY

By signing below, we certify that:

1. We have exercised meaningful human review, critical thinking, and intellectual oversight over all outputs assisted by AI tools.
2. We take full academic responsibility for the accuracy, data integrity, and originality of this work, ensuring that AI usage does not exceed allowable thresholds or violate plagiarism and integrity standards.
3. All AI contributions have been fully documented above in compliance with CMO No. 21, s. 2026.

_____

Junio, Carl AJ G.
Signature over Printed Name of Student Researcher

_____

Martinez, John Edrian V.
Signature over Printed Name of Student Researcher

_____

Navarro, Rex S.
Signature over Printed Name of Student Researcher

PART V: ADVISER ENDORSEMENT

I have reviewed this AI disclosure declaration and confirm that the students' use of AI tools complies with institutional policies and academic integrity standards.

_____

Federico G. Nueva, MT
Signature over Printed Name of Research Adviser

Date: 10/30/26

APPENDIX J : Extended Abstract

Web-Based Car Rental Management System with Real-Time IoT-Based GPS Tracking for 1816 Car Rental Co.

Carl AJ G. Junio
College of Computer Studies
Pamantasan ng Lungsod ng Pasig
Pasig City, Philippines
junio_carlaj@plpasig.edu.ph

John Edrian V. Martinez
College of Computer Studies
Pamantasan ng Lungsod ng Pasig
Pasig City, Philippines
martinez_johnedrian@plpasig.edu.ph

Rex S. Navarro
College of Computer Studies
Pamantasan ng Lungsod ng Pasig
Pasig City, Philippines
navarrojr_rex@plpasig.edu.ph

***Abstract*— 1816 Car Rental Co. managed reservations through Facebook Messenger and maintained transaction records in Google Sheets, requiring manual coordination of availability, payments, and vehicle records. This study developed a web-based car rental management system with Internet of Things based Global Positioning System tracking for the company's five-car fleet. An applied developmental approach and iterative Agile process connected renter document review, booking and availability checks, reservation payments, handover and return evidence, tracking, and management reports. Vehicle-condition assessment used a pretrained CarDD YOLOv8s model, with findings subject to owner review. Twenty-one purposively selected participants, comprising 15 renters, one owner, and five technical evaluators, assessed 15 indicators across six software-quality criteria. Equal weighting of the three evaluator groups produced an overall mean of 3.72 on a four-point scale, interpreted as Strongly Agree. Fourteen targeted backend checks passed. Image-presence evaluation on 374 CarDD test images produced class accuracies from 87.17 to 97.33 percent. GPS hardware testing was limited to researcher-operated prototype activities, and payment verification used test-mode or simulated callbacks. The findings support the implemented workflows and favorable participant assessments within the study setting; they do not establish fleet-wide tracking reliability or improvements in business performance.**

Keywords—car rental management, IoT, GPS tracking, vehicle-condition assessment, web-based system

I. INTRODUCTION

Small car rental businesses must coordinate vehicle availability, renter eligibility, payments, handovers, and returns using records that remain consistent throughout a rental. Online rental-management research has identified booking difficulties and fragmented fleet records as reasons to centralize these activities [1], [2]. A reservation interface alone, however, does not provide a complete record of what happens after a vehicle is released.

1816 Car Rental Co. was established in Marikina City in February 2025 and operated five cars at the time of the study. Reservations were arranged through Facebook Messenger and rental-related Facebook groups, while booking and revenue information was maintained in Google Sheets. This arrangement required manual coordination of customer inquiries, vehicle schedules, payment confirmation, and rental records. The business reported difficulties checking availability, preventing conflicting reservations, preparing rental documentation, and responding to customers.

This study developed a web-based system with Internet of Things (IoT) based Global Positioning System (GPS) tracking that connects these activities to a shared booking record. The objectives were to centralize rental operations; support management of vehicles, availability, payments, and active-rental location data; organize renter transactions from registration to confirmation; document vehicle condition before handover and after return; and provide operational reports and descriptive analytics. Access was separated among renters, staff, and the owner.

Targeted software checks, a vehicle-damage benchmark, and participant assessments evaluated the integrated workflows for one small rental operator.

II. REVIEW OF RELATED LITERATURE

Golo and Encarnacion [1] described operational problems in manual vehicle-rental services in Siargao and proposed an online management system. Nazer et al. [2] presented a rental website combining vehicle search, reservations, payment functions, and an administrative interface. These studies support integrating customer booking and fleet records, but their contexts and reported capabilities do not establish how the present system performs at 1816 Car Rental Co.

Tracking addresses an additional need once a vehicle leaves the rental premises. Wong Wei Min and Jali [3] described an IoT vehicle-tracking system using GPS and Global System for Mobile Communications (GSM) technology with a web

application. Alhwety and Elfadil [4] reviewed active and passive tracking approaches and discussed location monitoring, geofencing, and limitations associated with GPS reception. This literature supports incorporating tracking into rental operations while recognizing that structures and weak reception can affect positional accuracy.

IoT architectures commonly separate data acquisition, communication, and application services [5]. This arrangement informed the present design: a tracker produces readings, a cellular connection and Message Queuing Telemetry Transport (MQTT) broker transport them, and the application stores and presents available telemetry. Comparative work on MQTT and Hypertext Transfer Protocol (HTTP) supports considering communication overhead in constrained devices [6]. Studies of MQTT under adverse network and security conditions also show that delivery mechanisms involve trade-offs [7]. These findings justify the communication approach but provide no guaranteed delivery time for the prototype.

III. METHODOLOGY

A. Research Design and Development

The researchers used an applied developmental approach to identify an operational problem, develop a system, and assess its functions. An online interview with the owner established the existing booking process, document requirements, payment practices, vehicle release and return procedures, recordkeeping needs, and operational difficulties. These findings guided the system requirements.

Development followed iterative Agile stages of planning, design, development, testing, deployment, and review. Account access, vehicle records, and scheduling were established before dependent payment, condition, tracking, and reporting functions. Interface and database revisions were made as modules were reviewed. Deployment made the application available for evaluation; it did not constitute a longitudinal study of routine business use.

B. System Architecture

The architecture connected a React browser interface to a Node.js and Fastify application programming interface (API) and a PostgreSQL database hosted by Supabase. The API applied authentication, role restrictions, booking validation, payment verification, and access controls for protected records. PostgreSQL stored accounts, vehicles, bookings, payments, condition reports, and telemetry. Sensitive identity and condition media were accessed through protected file handling. Fig. 1 summarizes the principal components and exchanges.

External services supported specific operations. PayMongo provided hosted reservation checkout and payment-status information. Email services delivered verification codes and rental notices. Map services supplied styles and tiles; displayed routes came from the system's recorded GPS points. The tracking prototype sent coordinates, speed, and recording times through MQTT for backend processing and storage.

The Renter Portal presented vehicle information, booking status, receipts, finalized condition reports, messages, and eligible active-trip location information. Staff functions supported document review and rental operations. Owner functions additionally covered pricing, final condition assessment, refund decisions, business reports, and audit history. Role restrictions were applied before protected data or actions were made available.

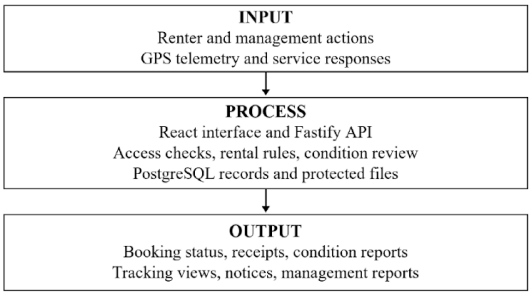


Fig. 1. System architecture summarized as inputs, processing, and outputs.

C. Rental and Vehicle-Condition Workflows

Registration collected account details, identification documents, and a guided live selfie for manual review. The camera sequence required the renter to face forward, turn left, return forward, turn right, and return forward before capture. Extracted document information assisted review but did not authenticate an identity document. Identity approval supported chauffeured rentals; Self-Drive access required separately reviewed driving documents and a valid license. The system did not connect to government identity or Land Transportation Office databases.

A booking included the vehicle, pickup and return schedule, rental type, destination, and contact or delivery details. The server checked the requested dates and vehicle availability against existing reservations and unpaid holds. Database overlap protection provided an additional constraint against conflicting live bookings for the same vehicle. Rates were stored with each booking so later price changes did not alter an existing reservation's recorded charges.

The booking workflow created a ten-minute payment hold and directed the renter to PayMongo for a PHP 1,000 reservation deposit. Payment confirmation depended on validated provider information, including the signature, amount, currency, and booking availability. A return from the hosted checkout page alone did not establish a verified payment. Confirmed payments updated booking status and supported receipt generation. The remaining rental balance was recorded separately and settled outside the online reservation-payment flow.

Handover and return inspections connected photographs, odometer and fuel readings, checklist responses, and notes to the booking. Exterior photographs could be analyzed for possible visible damage. The owner reviewed the suggestions, corrected or rejected them, added findings where needed, and finalized the report. The renter could review and acknowledge the finalized condition record. The workflow retained the documented authorized override and automatic-acknowledgment provisions rather than making model output the final decision.

Operational records also supported late-return monitoring, notifications, messaging, and eligible calamity-related refund requests. The owner declared the applicable calamity period and decided refund requests. Recording a refund decision or settlement reference did not itself transfer funds. Availability and reports depended on bookings entered into the platform; external inquiries did not reserve a vehicle until recorded through the system.

D. IoT Tracking Prototype

The tracking design used a LILYGO TTGO T-SIM A7670E R2 module. GPS readings and speed values were transmitted through the cellular network and an MQTT broker to the backend. Available telemetry supported the latest position, device status, recorded route, tracking-session history, and service-area alerts. During an active tracking session, the management view polled for new readings every ten seconds. A missing GPS fix was distinguished from an available position and was not presented as a valid route.

The design included flash buffering for readings collected during a network interruption and later transmission when connectivity returned. Such backfilled data could contribute to route history but could not supply a live position during the outage. Location availability still depended on power, satellite reception, and cellular connectivity. Geofencing used the configured service region and a maximum straight-line distance of 265 kilometers; this was not a guarantee of equivalent road-travel coverage.

Hardware checks were researcher-operated activities involving walking at the Pamantasan ng Lungsod ng Pasig campus and motorcycle travel in Pinagbuhatan. These activities focused on coordinates, speed readings, MQTT transmission, and connectivity. The prototype was not installed in the company's rental cars for actual customer trips, and the study did not report a fleet-wide field evaluation.

E. Vehicle-Damage Analysis

The condition-assessment module used the pretrained CarDD YOLOv8s version 2.0 model in Open Neural Network Exchange (ONNX) format, executed with ONNX Runtime [8], [9]. The model covered dent, scratch, crack, shattered glass, broken lamp, and flat tire. It was used as inspection assistance rather than as a mechanism for assigning liability, estimating repair cost, or imposing charges.

Before inference, the system checked image dimensions and visual quality. Resolution below 640 pixels and low image entropy generated warnings for review. Photographs were proportionally resized and centered in a 1024 by 1024 pixel input using gray padding with a channel value of 114. Transparency was removed and color values were normalized by division by 255. This letterbox process preserved the image proportions.

For each candidate, the highest class score determined the proposed damage category and confidence. Candidates with confidence at or below 0.25 were excluded. Predicted boxes were mapped back from the padded input to the original photograph, clipped to image boundaries, and discarded if their width or height was at most one pixel. Confidence expressed model support for a prediction and did not guarantee that the damage was present.

Class-wise non-maximum suppression reduced duplicate findings. Lower-confidence boxes were removed when their intersection-over-union with a retained box reached the default threshold of 0.45. A containment check also suppressed candidates when the shared area was at least 0.70 of the smaller box. Comparing candidates within the same class allowed different damage types to remain in overlapping regions.

Where available, segmentation masks were converted to normalized polygon boundaries with at most 64 points. A convex-hull representation simplified storage but could include space beyond an irregular damage boundary. The final output contained the damage category, confidence, normalized box, and a valid polygon when available. An area ratio described the detected box, not an exact measure of damage extent. Every finding remained subject to owner review.

F. Evaluation Design and Participants

Software verification examined booking-window validation, overlap checks, and payment-callback behavior. Separately, the damage-model evaluation used 374 images containing 785 annotated damage regions from the CarDD test split. Image identifiers, dimensions, instance counts, and categories were checked against the dataset metadata. For each class, an image was positive when the reference annotations contained that class. A prediction was positive when at least one detection exceeded the confidence threshold. Each image therefore contributed one image-presence result per class.

System assessment used purposive sampling with one owner, 15 renters, and five technical evaluators, totaling 21 respondents. Participants completed role-relevant tasks and answered the same structured Google Forms questionnaire. Fifteen indicators covered functionality, reliability, usability, efficiency, portability, and maintainability. The response values were 4 for Strongly Agree, 3 for Agree, 2 for Disagree, and 1 for Strongly Disagree. Mean ranges of 3.50-4.00, 2.50-3.49, 1.50-2.49, and 1.00-1.49 corresponded to these interpretations.

For each indicator, the renter, owner, and technical-evaluator group means were averaged equally. Thus, each group contributed one third of the composite despite differing group sizes. Criterion means were obtained by averaging their indicator composites, and the six criterion means were averaged equally for the overall mean. Intermediate values were not rounded; reported values were presented to two decimal places. These descriptive results represented the selected participants rather than estimates for a wider population.

G. Ethical and Data-Handling Considerations

The study's ethical framework included informed and voluntary participation, confidentiality, and collection limited to the stated research and rental purposes. Personal and location information required restricted access and proportionate use under the principles identified in the Data Privacy Act of 2012 [10]. Published results excluded unnecessary participant identifiers. GPS and condition findings served as supporting evidence for authorized human review. The study did not define a complete retention schedule or operational procedures for all

data-rights requests, leaving these as matters to address before broader deployment.

IV. RESULTS AND DISCUSSION

A. Implemented Rental Operations

The developed application connected vehicle information, renter requirements, availability, reservations, payments, condition records, and rental updates. Management views provided fleet inventory, pricing, a shared availability calendar, booking monitoring, handovers and returns, and tracking sessions. The owner could use date and vehicle filters to examine operational and financial records. These capabilities addressed the need for a common workflow in place of separate Messenger and spreadsheet records.

Recorded booking value and verified collections were treated as different measures. Payment-based revenue reports used verified collections, including retained deposits where applicable, rather than treating every reservation amount as received cash. Descriptive analytics displayed revenue trends, payment-channel breakdowns, vehicle comparisons, utilization, and booking patterns. The study demonstrated the availability of these views but did not independently reconcile every report against the company's financial records or measure changes in management decisions.

B. Booking and Payment Checks

All fourteen targeted backend checks passed. The booking checks covered rejection of a past pickup time, acceptance of a valid Manila schedule, rejection of invalid clock information, and detection of an existing unpaid hold for the same vehicle. PostgreSQL overlap protection supported the scheduling rules, although the reported exercise did not measure the incidence of concurrent booking conflicts during actual operations.

Payment checks accepted valid test-mode signatures and rejected invalid signatures. Booking confirmation checked payment amount, currency, and availability before updating the booking, with mismatches recorded as anomalies. A separate lifecycle exercise used a simulated signed callback rather than a real PayMongo transaction. The results support the tested validation paths; they do not establish live gateway performance or behavior under every possible payment-delay scenario.

C. Image-Level Damage Results

TABLE I. IMAGE-LEVEL DAMAGE PRESENCE RESULTS

Class	TP	FP	FN	TN	Accuracy (%)
Dent	135	17	22	200	89.57
Scratch	168	33	15	158	87.17
Crack	40	17	8	309	93.32
Shattered glass	69	8	2	295	97.33
Broken lamp	55	12	10	297	94.12
Flat tire	29	8	2	335	97.33

Note: 374 images per class; confidence > 0.25.

Table I consolidates the six image-presence confusion matrices. For each class, true positives (TP) indicate annotated presence with a positive prediction; false positives (FP) indicate an unannotated class reported by the model; false negatives (FN) indicate annotated presence missed by the model; and true negatives (TN) indicate absence in both the reference and prediction. Accuracy is the sum of TP and TN divided by 374, expressed as a percentage. Each row sums to the same 374 test images.

Scratch had the lowest image-level accuracy at 87.17 percent, with 168 true positives, 33 false positives, 15 false negatives, and 158 true negatives. These counts yielded precision of 83.58 percent, recall of 91.80 percent, and an F1 score of 87.50 percent. False positives can increase review work, while false negatives show why the owner must inspect the evidence rather than rely solely on model output.

The reported accuracies indicate whether a class was present in an image and detected at the selected threshold. They do not show that every damaged region was found or that boxes and masks matched the annotated boundaries. Instance-level localization and segmentation require separate evaluation. Likewise, benchmark results on CarDD images do not establish accuracy on photographs captured during the company's handover and return operations.

D. Participant Evaluation

Table II presents the six criterion means, all within the Strongly Agree range. The overall equal-group mean was 3.72.

TABLE II. SYSTEM EVALUATION BY QUALITY CRITERION

Criterion	Mean	Interpretation
Functionality	3.88	Strongly Agree
Reliability	3.57	Strongly Agree
Usability	3.52	Strongly Agree
Efficiency	3.76	Strongly Agree
Portability	3.87	Strongly Agree
Maintainability	3.71	Strongly Agree
Overall mean	3.72	Strongly Agree

Note: 4-point scale; evaluator groups weighted equally.

The mean represents agreement with questionnaire statements, not a 93 percent implementation success rate. Equal weighting gave the single owner's responses the same group-level contribution as all 15 renters or all five technical evaluators.

Lower indicator composites identified priorities for follow-up. Fault tolerance received 3.29, understandability 3.44, and learnability 3.27, all interpreted as Agree. These relative results support clearer instructions, more understandable status messages, first-use guidance, and closer examination of interrupted or unsuccessful operations. The questionnaire alone does not diagnose a specific software defect or measure actual recovery time.

V. STUDY LIMITATIONS

The study concerned one five-car rental business and a purposive sample of 21 participants. Its findings are descriptive and cannot be generalized to all rental companies or users. There was no controlled comparison of the former process and the developed platform. Reduced administrative workload, fewer real-world booking conflicts, increased profitability, and improved customer outcomes were not established by the reported evaluation.

Tracking software could display received coordinates, speed, and route history, but hardware testing remained limited to researcher-operated prototype activities. Actual customer rentals, long-duration installations, vehicle-power behavior, weak-network routes, and fleet-wide GPS reliability were not evaluated. Buffering can preserve some delayed readings when locally stored and successfully retransmitted; it cannot eliminate positional errors, power failures, or periods without current location data.

The damage results were image-presence benchmarks using a pretrained model. Lighting, camera angle, image quality, and the difference between benchmark images and rental inspections can affect practical performance. The study did not establish the fairness or consistency of liability decisions, because the model did not determine responsibility and the general survey did not directly evaluate dispute outcomes. A finalized condition record documents a review; it is not proof of when or how damage occurred.

Payment evidence was limited to test-mode and simulated callbacks. Identity review remained manual and did not verify documents against government databases. Analytics depended on complete and correct source records and were descriptive rather than predictive. Formal accounting, profit calculations, market forecasting, and automated management decisions were beyond scope. Further operational procedures for retention and data-rights requests also remained necessary.

VI. CONCLUSION AND RECOMMENDATIONS

The study developed an integrated web-based rental-management system for 1816 Car Rental Co. It linked renter document review, booking and payment status, vehicle schedules, handover and return evidence, available GPS telemetry, and management reports. Fourteen targeted backend checks passed, and the 21 participants' equal-group software-quality assessment produced an overall mean of 3.72, interpreted as Strongly Agree. The damage-model benchmark supplied reviewable findings while retaining the owner as the final condition reviewer.

These results support the implemented workflows and favorable assessments within the evaluation setting. They do not establish reliable tracking during real rentals, a fraud-free transaction process, fair automated liability decisions, or improved business performance. Future work should prioritize clearer first-use guidance and status messages, recovery testing for interrupted requests and delayed payment events, and sustained tracking trials in actual rental cars with appropriate consent.

Further evaluation should compare model suggestions with reviewed handover and return photographs under varied lighting and viewpoints, reconcile reports against known booking and payment totals, and assess owner tasks involving period and vehicle comparisons. Longer studies involving additional rental operators and respondents could determine whether the reported assessments persist during routine use. Data-retention and data-rights procedures should also be completed before wider deployment.

ACKNOWLEDGMENT

We first thank our Lord and Savior Jesus Christ for His guidance and faithfulness throughout the completion of this capstone project. We are grateful to our families for their patience, encouragement, and practical support. Their care, including the meals prepared for us before and after our joint work, helped us focus on completing the project.

We express our appreciation to our adviser, Federico G. Nueva, MT, for generously sharing his time, knowledge, and experience. His guidance helped us develop the system, work through difficulties, and find practical solutions. We also thank our panelists for reviewing our work and raising questions that helped us recognize the areas that needed correction and further attention.

Finally, we thank 1816 Car Rental Co. and its owner for trusting us with this study and helping us understand the business and its rental operations. We are also grateful to the renters, owner, and technical evaluators who gave their time to assess the system and share their feedback. Their participation helped us examine the developed functions and identify opportunities for improvement.

REFERENCES

[1] M. A. T. Golo and R. E. Encarnacion, "Revolutionizing the rental services in Siargao Island: Basis for developing an online vehicle rental management system," International Journal of Advanced Research in Science, Communication and Technology, vol. 4, no. 1, Jun. 2024, doi: 10.48175/IJARSCT-18754.

[2] N. Nazer, P. Prakash, Ridhunanda N, S. Sasi, and Girija V R, "Vehicle rental website," International Journal of Science and Research Archive, vol. 15, no. 3, pp. 1272-1279, 2025, doi: 10.30574/ijsra.2025.15.3.1813.

[3] J. Wong Wei Min and N. Jali, "Vehicle tracking system," International Journal of Scientific & Technology Research, vol. 10, pp. 10-17, 2021.

[4] R. N. H. Alhwety and N. Elfadil, "Vehicle tracking system approaches: A systematic literature review," International Journal of Computer Science and Mobile Computing, vol. 13, no. 8, pp. 23-31, 2024, doi: 10.47760/ijcsmc.2024.v13i08.003.

[5] M. Lombardi, F. Pascale, and D. Santaniello, "Internet of Things: A general overview between architectures, protocols and applications," Information, vol. 12, no. 2, Art. no. 87, 2021, doi: 10.3390/info12020087.

[6] H. J. Jara Ochoa, R. Pena, Y. Ledo Mezquita, E. Gonzalez, and S. Camacho-Leon, "Comparative analysis of power consumption between MQTT and HTTP protocols in an IoT platform designed and implemented for remote real-time monitoring of long-term cold chain transport operations," Sensors, vol. 23, no. 10, Art. no. 4896, 2023, doi: 10.3390/s23104896.

[7] V. Seoane, C. Garcia-Rubio, F. Almenares, and C. Campo, "Performance evaluation of CoAP and MQTT with security support for IoT environments," Computer Networks, vol. 197, Art. no. 108338, 2021, doi: 10.1016/j.comnet.2021.108338.

[8] X. Wang, W. Li, and Z. Wu, "CarDD: A new dataset for vision-based car damage detection," IEEE Transactions on Intelligent Transportation Systems, vol. 24, no. 7, pp. 7202-7214, 2023, doi: 10.1109/TITS.2023.3258480.

[9] abdullahg7, "CarDD YOLOv8s: Car damage detection and segmentation," pretrained model, ver. 2.0, Hugging Face. Accessed: Nov. 2, 2026. [Online]. Available: https://huggingface.co/abdullahg7/cardd-yolov8s

[10] Republic of the Philippines, "Republic Act No. 10173: Data Privacy Act of 2012," 2012. [Online]. Available: https://lawphil.net/statutes/repacts/ra2012/ra_10173_2012.html

CURRICULUM VITAE

CARL AJ G. JUNIO
Pasig City, Philippines | 0961 210 4501 | carlajjunio11@gmail.com
github.com/zaantoru

EDUCATION

Pamantasan ng Lungsod ng Pasig | BS Information Technology
Expected April 2027 | Consistent President's Lister
Rizal High School | ICT Programming | 2023
High Honors; Best in Website Design, school website competition

TECHNICAL PROJECTS AND LEADERSHIP

1816 Car Rental Platform | Team Leader and Developer | Apr 2026–Present
Led architecture planning, team guidance, development, and capstone presentations. Built booking and payment workflows with React, Fastify, and PostgreSQL; deployed on Railway with Supabase PostgreSQL. Developed LILYGO A7670E GPS tracking via MQTT and integrated pretrained YOLOv8/ONNX damage analysis for owner-reviewed inspections.
Car Rental System | Section Project Manager | Oct–Dec 2025
Managed 35 students and 7 team leads for two months to deliver a functioning car rental system.
Shopee Public Review Scraper | Developer | Oct 2025
Built a scraper for public product reviews from a user-provided Shopee link.
ALPS Bus Booking Website | Academic Project | Apr 2023
Developed a booking website for an ALPS The Bus, Inc. academic use case.

CLOUD AND TECHNICAL COURSEWORK

AWS Container Deployment Lab | Oct 2026: Docker images, Amazon ECR, ECS on Fargate, IAM roles, and security-group inbound rules.
Other labs: Unreal Engine 5 runner-stage game; Cisco Packet Tracer networks; client and server administration.

COMMUNITY LEADERSHIP

Youth Leader: Christian Faith Youth Fellowship (Nov 2023–Present); Elevate PLP (Oct 2024–Present). Mentors youth through discipleship and fellowship.
Public Information Officer | Rizal High School ICT Guild | 2022–2023

SKILLS AND INTERESTS

Languages: JavaScript, TypeScript, Python, C#, C++, PHP, HTML, CSS
Tools: React.js, Bootstrap, Fastify, PostgreSQL, MySQL, Git, Docker, Unreal Engine 5
Strengths: Architecture planning, project coordination, team guidance, public speaking
TESDA Computer Systems Servicing NC II (2023) | Filipino and English: fluent
Learning: Software reverse engineering | Interests: Strength training, motorcycles, mountain travel, gaming, project ideation

JOHN EDRIAN MARTINEZ
Pasig City, Metro Manila | edrianmartinez400@gmail.com
github.com/edrianvm | linkedin.com/in/martinez-john-edrian-9b6777290

EDUCATION

Pamantasan ng Lungsod ng Pasig | BS Information Technology
Expected graduation: April 2027
Honors: Dean's List and President's List recognition

ACADEMIC PROJECTS

1816 Car Rental Platform | Co-developer | Group Capstone | 2026
github.com/zaantoru/p2p-ui
Built React booking calendars, dashboards, and notifications for renter, staff, and owner portals. Added authorized vehicle route history and improved handling of expired or cancelled unpaid reservations. Improved dashboard loading and tested booking and inspection workflows with 150 simulated bookings.
RevOn Car Rental ERP | Module Lead and Co-developer | 2025
Customer Management (CRM) | github.com/RevOn-ERP/car-rental-erp
Co-developed customer activity logs and booking-history screens in a shared ERP. Added date and status filters; corrected customer spending summaries and account-status handling.
CourseDock Courseware Monitoring System | Co-developer | 2025
Group Academic Project | github.com/ychine/CMS
Developed task assignment, curriculum management, and submission tracking with faculty role controls. Created reports with Excel downloads for course assignments and submission status. Added course and program activity logs; improved registration checks and notification indicators.

TECHNICAL SKILLS

Languages: JavaScript, PHP, SQL, HTML, CSS
Frameworks and tools: React, Node.js, Fastify, Tailwind CSS, Vite, Git, GitHub
Databases: PostgreSQL, MySQL
Development: REST APIs, responsive interfaces, debugging, automated testing

TRAINING

Arellano University – Andres Bonifacio Campus | March 19, 2023
The Role of ICT Industry in the Pandemic; Java Application Technology; Web Designing Basics and Techniques

REX NAVARRO JR.

Metro Manila, Philippines | navarrorex438@gmail.com | 0976 461 0666
linkedin.com/in/rex-navarro-997a17388 | github.com/RexNavarro7

SUMMARY

BS Information Technology student (expected April 2027) who can take a business question, document what stakeholders actually need, then turn data into reports and dashboards using Microsoft Excel, SQL (MySQL Workbench), and Power BI. On my capstone for 1816 Car Rental Co., I sat in client interviews, listed requirements as they were stated, and captured research-adviser recommendations so the team built against a written list instead of memory. I offer junior analyst teams someone who can clean and query data, support recurring reports, and explain findings in plain language to non-technical users.

EDUCATION

Bachelor of Science in Information Technology
Expected Graduation: April 2027
Pamantasan ng Lungsod ng Pasig
Consistent Dean's Lister and President's Lister
1st Place, Best Presenter, Research Congress | Research Congress, 2023

EXPERIENCE

Capstone Project | Requirements Elicitation and System Documentation | 2026 – Present
Web-Based Car Rental Management System with Real-Time IoT-Based GPS Tracking
1816 Car Rental Co.

- Interviewed clients for 1816 Car Rental Co. and listed functional and operational requirements as they were described, producing a written requirements list the team used during build and paper writing.
- Captured research-adviser recommendations during advising sessions and converted them into requirement notes so recommended changes were not lost between meetings.
- Contributed to both the system and the research paper so documented client needs and adviser inputs were reflected in the delivered car rental and GPS-tracking solution.

SKILLS

Data Analysis & Reporting: SQL, Microsoft Excel, Data Cleaning, Data Validation, Reporting, Dashboards, Ad-hoc Analysis
Business Intelligence: Power BI, Data Visualization, Looker (familiar)
Business Analysis: Requirements Gathering, Stakeholder Interviews, Requirements Documentation
Tools: MySQL Workbench, Jira (familiar)